

Sistema de Recuperación de Información Estructurada para el Parlamento de Andalucía

Memoria del Proyecto



E.T.S. Ingeniería Informática. Universidad de Granada. 2005

ALFONSO E. ROMERO LÓPEZ

E-mail: aromero@correo.ugr.es

Pág. web: <http://alfonso.2ya.com>

Hecho en $\LaTeX$

Índice general

Agradecimientos	7
Introducción	9
1. Fundamentos teóricos de la Recuperación de Información	11
1.1. Introducción	11
1.2. Recuperación de Información	12
1.2.1. Recuperación de información como un proceso	13
1.2.2. Estructuras para la Indexación	15
1.2.3. Evaluación de la recuperación	19
1.3. Modelos de recuperación de información	23
1.3.1. Notación utilizada	23
1.3.2. Modelo booleano	23
1.3.3. Modelo de espacio vectorial	24
1.3.4. Modelo probabilístico básico	25
2. Redes Bayesianas y modelos para Recuperación de Información no estructurada	29
2.1. Incertidumbre y Recuperación de Información	29
2.2. Conceptos previos sobre teoría de la probabilidad	31
2.3. Redes Bayesianas: introducción	34
2.3.1. Definición. Ejemplo	34
2.3.2. Independencia e independencia condicional	35
2.3.3. <i>d</i> -Separación y criterios gráficos de independencia	36
2.3.4. Causalidad. Razonamiento abductivo	37
2.3.5. Propagación de evidencia. Inferencia en Redes Bayesianas	39
2.4. Modelos basados en Redes Bayesianas	40
2.4.1. Primeros modelos	40
2.4.2. El modelo BNR	42

3. Información estructurada y Redes Bayesianas	51
3.1. Información estructurada: una introducción	51
3.2. Recuperación de documentos estructurados: introducción y problemas	54
3.3. Lenguajes para información estructurada	58
3.3.1. Introducción. XML como herramienta para información estructurada	58
3.3.2. Breve descripción de la sintaxis de XML	59
3.4. Redes Bayesianas para documentos estructurados: el modelo BNR-SD	62
3.4.1. Definición del modelo	62
3.4.2. Inferencia	63
4. Desarrollo de un Sistema para Recuperación de Documentos Estructurados	65
4.1. Introducción	65
4.2. Estudio de Requisitos	66
4.2.1. Requisitos genéricos	66
4.2.2. Requisitos detallados sobre la información a almacenar	66
4.2.3. Requisitos detallados sobre las operaciones	70
4.3. Diseño del Sistema: Nivel lógico	73
4.3.1. Introducción	73
4.3.2. Niveles y vistas del Sistema de RI	73
4.3.3. Nivel conceptual	75
4.3.4. Nivel de estructuras de datos	75
4.3.5. Nivel físico	76
4.4. Identificación de las clases del sistema	77
4.4.1. Clases del nivel conceptual	77
4.4.2. Clases que representan abstracción sobre estructuras del sistema	77
4.5. El nivel físico	79
4.5.1. Introducción	79
4.5.2. Relaciones entre términos y unidades estructurales	79
4.5.3. Estructuras en memoria	87
4.5.4. Índice estructural	90
4.5.5. Posibles optimizaciones	92
4.5.6. Tabla-Resumen sobre las distintas estructuras	95
4.6. Diseño de las clases (nivel lógico)	96
4.6.1. Introducción	96
4.6.2. Atributos y operaciones	96
4.6.3. Especificación de los algoritmos requeridos	102

5. Diseño de los algoritmos	107
5.1. Introducción	107
5.2. Diseño de un algoritmo de indexación para documentos estructu- rados	108
5.2.1. Operaciones necesarias: programas de indexación	108
5.2.2. Métodos de inversión	108
5.2.3. Adaptación de un algoritmo de indexación para el caso estructurado	112
5.3. Diseño de un algoritmo de recuperación para documentos estruc- turados	116
5.3.1. Introducción	116
5.3.2. Implementación del modelo BNR-SD	116
6. Aplicación al caso del Parlamento de Andalucía	123
6.1. Introducción	123
6.2. Los Diarios de Sesiones del Parlamento de Andalucía	124
6.3. Estructura de las sesiones plenarias. Adaptación de la colección	126
6.3.1. Estructura de las sesiones	127
6.3.2. Adaptación de la colección	129
6.4. Implantación del sistema	130
6.4.1. Localización del sistema y la colección	130
6.4.2. Interacción con los usuarios: interfaz del sistema	132
6.4.3. Experimentación y pruebas	132
Conclusión y trabajo futuro	135

Agradecimientos

Hace ya más de un año que pedí al Dr. Luis M. de Campos Ibáñez, del Departamento de Ciencias de la Computación e Inteligencia Artificial, la posibilidad de que me dirigiera el trabajo en la realización de una beca de iniciación a la investigación de la Universidad de Granada. Desde el principio, su interés en mí y su disposición personal fueron bastante grandes (aun cuando no se conocía la resolución), y fueron pasando por mí numerosos libros y artículos relacionados con el tema que él me había planteado: Recuperación de Información (en particular, la implementación de un sistema concreto para el Parlamento de Andalucía). Quisiera agradecerle toda la confianza que ha puesto en mí para la realización de este trabajo, además del material y tiempo prestado (que no ha sido poco).

Además, también quisiera agradecer al Dr. Juan Huete Guadix y al Dr. Juan M. Fernández Luna su ayuda en las interminables reuniones que, junto con el director de esta beca pasamos comentando detalles sobre la implementación del sistema. Especialmente, quisiera agradecer a este último sus consejos sobre la implementación del sistema, y la ayuda prestada. Sin su experiencia pasada, seguramente, el desarrollo de este sistema se habría dirigido a un largo camino de final lejano, o peor aún, un camino sin salida.

Finalmente, quisiera agradecer a la Universidad de Granada, especialmente al Vicerrectorado de Investigación, la ayuda económica prestada para realizar este proyecto así como la oportunidad prestada para realizar mi primer trabajo de investigación.

Introducción

Esta memoria contiene el trabajo realizado durante el período de disfrute de la beca de Iniciación a la Investigación de la Universidad de Granada. En ella se desarrolla un sistema de recuperación de información estructurada y se aplica a un caso concreto: la colección formada por los diarios de sesiones de los plenos del Parlamento de Andalucía.

Durante el período de investigación, la primera parte fue dedicada a información y recopilación de fuentes. Debido a que esta materia no se imparte directamente en ninguna asignatura del currículum de Ingeniero en Informática, el primer capítulo surge de manera natural como recopilación de generalidades sobre la Recuperación de Información. En él se presenta este área junto a la terminología que la acompaña. Su contenido no pretende, desde luego, ser un recorrido profundo del problema de la Recuperación de Información, puesto que ya se mencionan en la bibliografía suficientes fuentes en las que encontrar más datos. Por el contrario, sí es una introducción suficientemente compacta a este problema, sin dejar de tratar nada de lo que será usado en capítulos posteriores.

El segundo capítulo, también orientado a fundamentos teóricos, da un giro totalmente distinto y presenta las Redes Bayesianas como un modelo de recuperación desde una perspectiva de modelos cronológicamente primeros (*Inference Network Model*, *Belief Network Model*...) hasta llegar al punto principal del capítulo: el modelo BNR, con sus diferentes variantes, creado por investigadores del grupo de investigación en el que se encuadra esta beca (dirigidos por el mismo director de este trabajo), y que ha dado como fruto más de diez publicaciones en revistas y congresos internacionales.

El capítulo tres sigue en la misma línea que los anteriores, esta vez ampliando el problema de la Recuperación de Información “clásica” hasta el problema de tratar con información estructurada. Es un hecho que dicho tipo de información (cuyo método de expresión es el bien conocido lenguaje XML) invade cada vez más el mundo de las tecnologías web (y también el mundo de los almacenes de información), necesitándose un marco teórico sólido para tratar con este tipo de información. Así, se presenta como broche final, el modelo BNR-SD, ampliación del anterior, cuidadosamente diseñado para tratar con datos estructurados. Dicho

modelo también ha sido obra del mismo grupo de investigación y ha sido una evolución temporal (y natural) del anterior, presentando también resultados de calidad en ciertas publicaciones.

La inexistencia de una herramienta de libre disposición para trabajar con sistemas de recuperación de información estructurada (salvo el caso quizás de *The Lemur Toolkit*¹, aunque no bajo una licencia libre, lo cuál restringe su uso) nos obligaba a diseñar y desarrollar desde cero un sistema de estas características. Los capítulos cuarto y quinto presentan el diseño de este sistema, y de los algoritmos que incorpora, prácticamente desde cero. Dicho trabajo constituye el grueso de la memoria (prácticamente la mitad de la misma) y se trata de un desarrollo totalmente novedoso, cuyo resultado es original del autor de la misma, siendo este el primer documento en el que se recopila dicho trabajo.

El fin último de este trabajo, la aplicación del sistema de recuperación al caso concreto del Parlamento de Andalucía, no ha podido ser realizado aún. El capítulo sexto comenta algunas peculiaridades de dicha colección (los diarios de sesión de los plenos del Parlamento de Andalucía) que impiden, por ahora, un trabajo con los mismos. El poco tiempo transcurrido desde la resolución de la beca hasta la entrega de esta memoria ha comprometido en gran parte, el hecho de que no se pueda terminar el trabajo al 100 %. Asimismo, el desarrollo del software tampoco ha llegado a su etapa final puesto que, como todo proceso de ingeniería, hasta que el mismo no haya sido cuidadosamente verificado y validado, no podemos estar plenamente seguros de su correcto funcionamiento. Pese a esto, el trabajo restante es una mínima parte del global, por lo que seguiré en su realización hasta obtener los resultados buscados.

¹Disponible en www.lemurproject.org

Capítulo 1

Fundamentos teóricos de la Recuperación de Información

1.1. Introducción

En este punto trataremos de echar un vistazo en cierto detalle a las diferentes áreas relacionadas con el trabajo posterior. Así, primeramente, estudiaremos generalidades sobre la *recuperación de información* (modelos clásicos, estructuras de datos, etc) con objeto de poder comprender los capítulos siguientes. Posteriormente se desarrollará un apartado sobre *redes bayesianas*, una herramienta utilizada en inteligencia artificial para representar incertidumbre, basada en el cálculo de probabilidades. Extendiendo a los modelos de recuperación de información probabilísticos, describimos los modelos de recuperación de información basados en redes bayesianas (los más sencillos). Finalmente, se introduce el problema de la información estructurada y la terminología que ella conlleva. Asociado a este problema, se presentan los modelos más avanzados cuyo uso quedará patente en la implementación del sistema.

Presuponemos que el lector está familiarizado con las disciplinas básicas de las ciencias de la computación (como la teoría de algoritmos), la matemática discreta (grafos) y posee cierto conocimiento de estadística y probabilidad básica, así como de nociones de programación y diseño de programas.

Se ha incluido, para cada uno de los puntos suficientes referencias “clásicas” para que el lector, si lo desea, pueda profundizar en cada punto y ampliar conocimiento. A pesar de esto, se ha tratado de escribir este capítulo para su fácil comprensión pudiendo prescindir de las referencias y quedando sólo como material complementario.

1.2. Recuperación de Información

Los Sistemas de Recuperación de Información¹ son un software con un determinado objetivo. Éste es, a grandes rasgos, dada una colección de documentos y una consulta formulada por un usuario en un cierto momento², el proporcionar el subconjunto de documentos que es más relevante para dicho usuario y consulta.

La caracterización formal de una técnica de recuperación de información es lo que se conoce como *modelo de recuperación de información*. Según Baeza y Ribeiro ([3]), un *modelo de recuperación de información* es un cuádrupla $(\mathcal{D}, \mathcal{Q}, \mathcal{F}, R(q_i, d_j))$ donde:

1. $\mathcal{D}$ es un conjunto formado por la representación (vistas lógicas) de los documentos.
2. $\mathcal{Q}$ es un conjunto formado por consultas, es decir la representación (vistas lógicas) de la información que el usuario necesita.
3. $\mathcal{F}$ es un marco o modelo de representación de los documentos, las consultas, y las relaciones existentes entre ellos.
4. $R(q_i, d_j)$ es una función (de clasificación o *ranking*) que asocia un número real con una consulta $q_i \in Q$ y la representación del documento $d_j \in D$, de esta forma se ordenan en una clasificación los documentos que responden a una misma consulta.

Conocido lo anterior y, sin entrar en amplio detalle, podemos decir que el funcionamiento de un SRI es el siguiente:

1. El usuario introduce una *consulta* en el sistema. Esta consulta representa sus necesidades de información.
2. El sistema *procesa* dicha consulta. Se buscan documentos que, de alguna forma, sean coincidentes con los términos que aparecen en dicha consulta.
3. El sistema muestra los documentos coincidentes con la consulta, ordenándolos de mayor a menos relevancia según el valor proporcionado por la función R .

En la literatura existen numerosos modelos de RI³. Así, en Baeza-Ribeiro ([3], capítulo 2) podemos encontrar, entre otros, el modelo booleano, el modelo de

¹En adelante, usaremos este término o bien SRI, indistintamente.

²Resumiendo sus intereses respecto de los documentos de la colección

³Recuperación de información.

espacio vectorial, el probabilista, o el basado en redes de creencia. Se mencionan especialmente estos cuatro por ser los que más relación tienen con este trabajo. Posteriormente se describirán con más detalle cada uno de estos modelos.

1.2.1. Recuperación de información como un proceso

Obviando el modelo subyacente, podemos tratar la recuperación de información de un conjunto de documentos (que llamaremos *colección*) como un proceso que consta de distintas partes. Para poder entenderlo mejor, observemos el siguiente gráfico (simplificación del aparecido en [19]):

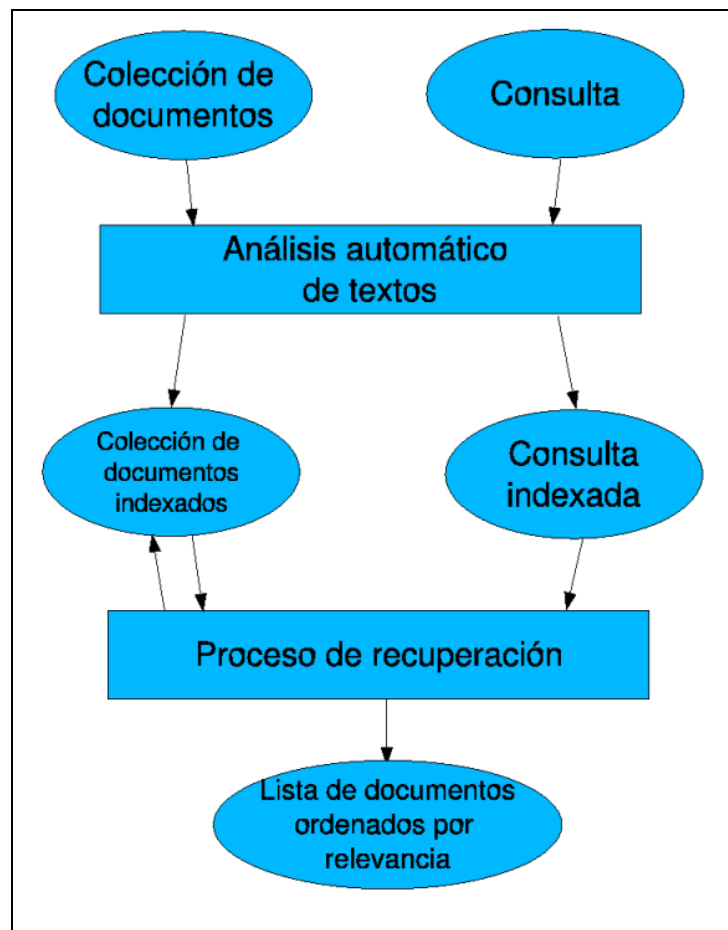


Figura 1.1: Sistema de RI típico.

Desde el punto de vista de los documentos, todo documento pasa por dos etapas: una de ellas, cuando es introducido por primera vez al sistema y se produce el proceso conocido como “análisis de textos”, donde son extraídas todas las ca-

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

racterísticas que posteriormente se usarán al documento (en ese momento el se crean una serie de índices para hacer más eficiente el acceso, siendo esto es conocido como *indexación*). En la segunda etapa, el documento es recuperado (si es conveniente) desde el motor de recuperación.

Desde el punto de vista de un usuario normal, éste formula una consulta al sistema (generalmente en lenguaje natural) que posteriormente va a pasar también por el analizador de textos (sufriendo asimismo el proceso de indexación) y el sistema, con esta consulta indexada devolverá un *resultado* conteniendo los documentos de la colección que más se ajusten a las necesidades del usuario, ordenados por relevancia (si el modelo incluye esta característica).

En ambos casos, el papel del módulo analizador de textos tiene bastante importancia. A la hora de tratar los términos uno a uno, antes de introducirlos en el sistema, este módulo puede realizar, opcionalmente, algunas de estas acciones:

- *Stemming* (ó lematización): que equivale a obtener sólo la raíz de la palabra que se trata en este momento. Al realizar esto, por ejemplo, se eliminan plurales y todas las formas verbales, quedando en sí el núcleo de la información. Desgraciadamente, no es un problema fácil y además depende del idioma en el que estemos trabajando. Para el caso concreto del inglés, existen varios algoritmos, siendo el más famoso el de Porter (*Porter stemmer*). Para el caso del español, no existe a día de hoy ningún trabajo conocido de una calidad comparable. Una comparación seria de diferentes algoritmos para realizar este proceso puede verse en [22].
- *Case Folding*: que normaliza las palabras reduciendo todas las letras a minúsculas. Además, para el caso de los idiomas con tildes, se pueden reducir todas las distintas versiones de una vocal acentuada a la versión sin acentuar (este último punto, aunque discutible, siempre va a ayudar en la búsqueda y va a ahorrar espacio).
- Eliminación de *stopwords* o “palabras vacías”: aquellas que no poseen un significado por sí solas y su frecuencia de aparición en el idioma anfitrión es bastante alta (p. ej: preposiciones, determinantes, ...). También, basándonos en la ley de Zipf (véanse [19] y [35]) podemos eliminar aquellas palabras cuya frecuencia de aparición esté por debajo o por encima de unos límites (palabras muy comunes o demasiado infrecuentes) por no aportar casi información sobre el documento en el que aparecen. Para cada idioma existen listas de *stopwords* bastante completas, así que no es necesario construir una propia.
- Eliminación de signos no alfabéticos: signos de puntuación, espacios en blanco, números (según el caso), ...

Una vez indexada una colección, podemos tener una vista lógica de ésta como un array de documentos, y de este último como un array de términos. Este será el modelo de sistema con el que trabajaremos a partir de ahora.

1.2.2. Estructuras para la Indexación

Introducción

Antes de realizar un procedimiento de recuperación de información, ésta debe estar almacenada de manera conveniente en un soporte físico en forma de *documentos*. De este modo, se podrán realizar búsquedas sobre los datos. Una forma obvia (aunque primitiva) de realizar esta búsqueda es realizar una *búsqueda secuencial*, esto es, analizar la similitud de cada uno del total de documentos con la consulta proporcionada y devolver sólo los más concordantes con aquélla.

Cuando el conjunto de documentos es pequeño y su tamaño total es de unos pocos Megabytes (p.ej: un volumen de una revista científica), esto es claramente posible. Cuando el volumen de información se mide en órdenes de magnitudes mayores (Giga o Terabytes) y el tiempo de respuesta es crítico, se necesitan otras soluciones.

Es razonable pensar entonces que dicha base de datos no va a contener la totalidad del documento en la forma en la que se concibe como tal, sino que, en su lugar, podría contener una representación del mismo (hecha de forma manual o automática).

Así, para un sistema de recuperación de información de textos, el punto del que se partirá será del conjunto de textos completos (si son pequeños), o de las partes específicas representativas en las que vamos a buscar (p. ej.: en una colección de artículos, el *abstract* suele ser bastante representativo del contenido del resto del documento) o, simplemente una lista de palabras (si el texto, p.ej. es plano desestructurado). A partir de ahí, el proceso debe producir unas estructuras de datos en una forma que el sistema pueda manejarlas correctamente. Estas estructuras de datos se conocen como *índices* e incrementan la velocidad de búsqueda sobre un conjunto de documentos, sobre todo, si la colección es *semi-estática* (soporta actualizaciones de forma regular). El proceso de construcción de estas estructuras de datos es llamado *indexación*.

Como ejemplo de estas técnicas, veremos una de las estructuras que mejor resultado ha dado en los sistemas implementados históricamente: los *archivos invertidos*

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

Archivos invertidos: introducción

Un archivo invertido es un mecanismo para indexar un documento, generalmente un texto. Está compuesto de dos partes: el *vocabulario* y las *ocurrencias*. El vocabulario es el conjunto de palabras diferentes del texto, mientras que las ocurrencias son los puntos de aparición de cada una de las palabras dentro del texto (véase la figura 1.2 para un ejemplo sencillo).

1 Verde que te quiero verde.	14 21	
27 Verde viento. Verdes ramas.	32 40 47	
56 El barco sobre la mar	71	
79 y el caballo en la montaña.	93	
108 Con la sombra en la cintura	121	
133 ella sueña en su baranda,	145	
153 verde carne, pelo verde,	166 171	
181 con ojos de fría plata.	189 194	
200 Verde que te quiero verde.	213 220	

Vocabulario	Ocurrencias
barco:	56
baranda:	145
caballo:	79
carne:	159
cintura:	121
fría:	189
montaña:	93
ojos:	181
pelo:	166
plata:	194
quiero:	14, 213
rama:	47
sombra:	108
sueña:	133
verde:	1, 21, 27, 40, 153,
viento:	27

Figura 1.2: Archivo invertido sencillo. Obsérvese que se han eliminado *stopwords* al indexar y se ha realizado un cierto nivel de *stemming* (se han eliminado algunos plurales).

El espacio requerido para el vocabulario es pequeño, creciendo a un orden $O(n^\beta)$, donde β es una constante entre 0 y 1 dependiente del texto, siendo en la práctica entre 0.4 y 0.6 en la mayoría de los casos. Para hacerse una idea de lo que supone esto, para la colección de prueba TREC-2 (ver siguiente subsección, “Evaluación de la recuperación”) de tamaño 1 Gb, el tamaño del vocabulario apenas supera los 5 Mb.

Las ocurrencias necesitan bastante más espacio. En efecto, cada palabra que aparece en el texto se referencia una vez en la estructura y el espacio requerido es $O(n)$. Incluso omitiendo la indexación de un cierto conjunto de stopwords, la tabla de ocurrencias puede llegar a tener entre un 30 % y 40 % del tamaño del texto original.

Para poder reducir esta necesidad de espacio, utilizamos *direccionamiento por bloques*. La idea es simple: el texto se divide en bloques, y la tabla de ocurrencias referencia al bloque en el que se encuentra una palabra. De esta forma, todas las apariciones de una misma palabra dentro de un bloque se colapsan a una misma referencia. Así, es posible obtener índices de tan sólo 5 % del tamaño total del texto. El precio a pagar es la necesidad de una búsqueda lineal dentro del bloque concreto si se requieren las ocurrencias exactas del término buscado. Una tabla comparativa con el tamaño completo de los índices (vocabulario y ocurrencias) según el tamaño de los bloques elegidos para varias colecciones modelo se puede encontrar en [3].

Búsqueda en archivos invertidos

¿Por qué es tan rápido buscar en un archivo invertido? La respuesta la podemos dar a partir del algoritmo de búsqueda. Éste suele constar de los siguientes tres pasos:

1. *Búsqueda de vocabulario*: las palabras o patrones presentes en la consulta se aíslan y se localizan en el vocabulario.
2. *Recuperación de ocurrencias*: la lista de ocurrencias de estas palabras se extrae del sistema.
3. *Manipulación de ocurrencias*: las ocurrencias se procesan para encontrar frases, resultados de aplicar operaciones booleanas sobre conjuntos de palabras, o palabras próximas. Si se usa direccionamiento por bloques, es posible que sea necesario la búsqueda de ocurrencias concretas dentro de cada bloque devuelto como respuesta.

La justificación de que el vocabulario resida en un fichero aparte es que las búsquedas comienzan siempre por él. Debido a que tiene además un tamaño mucho menor que la tabla de ocurrencias, incluso para colecciones de texto de tamaño grande, el vocabulario puede residir en memoria principal. Para colecciones de tamaño respetable el vocabulario no ocupa más de 40 MB (un tamaño al alcance de cualquier ordenador de hoy en día), como podemos comprobar en el capítulo tercero de [43].

Las *consultas unarias* se pueden resolver por algún método que incremente la velocidad, como por ejemplo: *hashing* o *árboles-B*. Sin embargo, almacenar las palabras en orden lexicográfico ya proporciona un tiempo $O(\log n)$ usando búsqueda binaria. Si la consulta la forman varias palabras, se devuelve la unión de las listas de ocurrencias respectivas.

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

Las consultas basadas en el contexto o proximidad son bastante más difíciles de resolver haciendo uso de índices invertidos. Cada elemento debe ser buscado por separado, generándose una lista para cada uno. Entonces, se recorre el conjunto de las listas en sincronización, buscando lugares donde aparecen palabras de varias listas en secuencias (para frases) o suficientemente cerca (para encontrar resultados próximos).

Construcción de archivos invertidos

La construcción de archivos invertidos, proceso conocido como *inversión*, no es del todo crítica, es decir, puede admitir ser realizada a una velocidad relativamente lenta, pues sólo va a lanzarse una vez por documento (al menos en principio). No obstante, para ciertos casos, interesa que su desempeño sea lo más eficiente posible, por ejemplo si estamos tratando con una colección dinámica y haya que reconstruir el índice (parcial o completamente) numerosas veces debido a que haya nuevos documentos, o algunos sean dados de baja.

En [3] se propone usar como método un *árbol trie*⁴. Conceptualmente se trata de una idea mejor que realizar la inversión *ad hoc* manteniendo el archivo invertido en memoria en su totalidad, pero no deja de ser ineficiente.

En la literatura se encuentran varias formas de realizar una inversión recopiladas en [43], capítulo quinto. Entre ellas encontramos la anteriormente mencionada que realizaba la construcción del índice completo en memoria. Este proceso llamado *inversión en memoria* resulta poco eficiente para colecciones muy grandes y suele ser desechado.

La mejor manera de realizar una inversión es quizás la inversión basada en ordenación. La idea es realizar la construcción del índice en tres etapas, obteniendo en la primera unas tripletas $\langle t, d, f_{d,t} \rangle$ ⁵, ordenándolas por valor de t en lotes, y en la tercera fase realizando un proceso de mezcla de dichos lotes. Todo el proceso apoyado, por supuesto, en el uso de archivos de disco temporales. El algoritmo es el siguiente:

1. Crear una estructura de diccionario vacía, S en disco. Crear un archivo temporal vacío en disco.
2. Para cada documento D_d de la colección, $1 \leq d \leq N$,
 - a) Leer D_d , obteniendo los términos.
 - b) Para cada término $t \in D_d$,

⁴Ésta y otras estructuras de datos relacionadas con el proceso de construcción de índices se describen en detalle en [3].

⁵Donde t es un término, d un documento y $f_{d,t}$ la frecuencia de aparición de t en d .

- 1) Sea $f_{d,t}$ la frecuencia en D_d del término t .
 - 2) Buscar a t en S .
 - 3) Si no se ha encontrado, insértese.
 - 4) Escribir un registro $\langle t, d, f_{d,t} \rangle$ al archivo temporal, donde t se representa por el identificador de término que tiene en S .
3. Sea k el número de registros que caben en memoria,
- a) Leer k registros del archivo temporal.
 - b) Ordenar por orden no decreciente de t , por valores iguales de t y por orden no decreciente de d .
 - c) Volver a escribir el lote reordenado al archivo temporal.
 - d) Repetir hasta que no queden más lotes para ordenar.
4. Mezclar lotes de dos en dos en el registro temporal hasta que esté ordenado.
5. Para cada término $1 \leq t \leq n$,
- a) Comenzar una nueva entrada de archivo invertido.
 - b) Leer todos los triples $\langle t, d, f_{d,t} \rangle$ del archivo temporal y formar la entrada del archivo invertido para el término t .
 - c) Si se requiere, aplicar compresión.
 - d) Añadir esa entrada al archivo invertido.

La aplicación del punto 5.c es opcional. Aplicar compresión puede ser útil en muchos casos (sobre todo en colecciones estáticas) para poder ahorrar espacio y almacenar el máximo número de datos en memoria (lo cual acelera el proceso global de recuperación posterior). Para una referencia bastante precisa de las diferentes técnicas de compresión, puede consultarse [43], capítulo segundo.

Por último, añadir que el resto de métodos que aparecen para la construcción de archivos invertidos sólo son mejoras de los dos presentados. Podemos destacar la *inversión comprimida en memoria*, que aplica técnicas de compresión para realizar una inversión de la colección completa sin usar archivos temporales, con el consiguiente ahorro de tiempo. Como ya se ha dicho, el capítulo quinto de [43] cubre estas y otras técnicas.

1.2.3. Evaluación de la recuperación

Introducción

El tipo de evaluación depende de los objetivos del sistema de recuperación. Inicialmente, y debido a que estamos tratando con un sistema software el primer tipo

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

de evaluación que se debe realizar es un *análisis funcional*, en el cual se comprueba su funcionalidad en todas sus partes. Este análisis también incluiría una fase de inserción y detección de errores, en las que se introducirían errores (proceso conocido como “siembra de errores”) en el sistema para intentar hacerlo fallar. Esto es útil para detectar problemas de programación y en general hacer una estimación del número de errores del sistema. En [32], en el capítulo noveno, se explican éstas y otras técnicas de prueba y estimación de fallos del sistema.

Pasada esta fase de análisis funcional, se ha de realizar una evaluación de efectividad del sistema, esto es, si realmente hace lo que se quiere.

Para el caso de algoritmos de recuperación de información, en lugar de tiempo y espacio (magnitudes clásicas en la teoría de algoritmos) hay otras medidas muy importantes. Las dos principales son la precisión y la eficacia del conjunto resultado. El método general de evaluación de la efectividad de un método de recuperación de información consiste en que dado un conjunto de documentos respuesta de una consulta, obtenidos con una estrategia, y un conjunto de documentos seleccionados como los relevantes para esa consulta, por un grupo de expertos, medimos la similitud entre ambos conjuntos, y esto nos proporciona una estimación de la bondad de dicha estrategia.

Evaluación de la eficiencia en la recuperación

Dado que hay diferentes métodos de recuperación de información (consultas simples, interacción con el usuario, combinación de ambos, ...), también hay distintos métodos de evaluación de dicha recuperación, específicos para cada método. En adelante nos centraremos en los métodos que evalúan la recuperación de información en las cuáles el usuario realiza consultas (sin interacción), para especificar los documentos deseados.

Precisión y eficacia

Para evaluar estos métodos mediante las medidas de *precisión* y *eficacia* consideramos un ejemplo de información solicitada y si conjunto R de documentos relevantes. Sea $|R|$ el número de documentos en este conjunto. Sea A el conjunto de documentos respuesta, obtenidos por la estrategia de recuperación a la solicitud. Sea $|A|$ el número de documentos en el conjunto A . Sea $|R_a|$ el número de documentos en la intersección de los conjuntos R y A ; definimos la precisión y la eficacia como:

Eficacia es la fracción de los documentos relevantes (R) que han sido recuperados:

$$\text{Eficacia} = \frac{|R_a|}{|R|}$$

Precisión es la fracción de los documentos recuperados (A) que son relevantes:

$$\text{Precisión} = \frac{|R_a|}{|A|}$$

A mayor precisión y eficacia mejor será el algoritmo, pero en estas definiciones se presupone que el usuario verá todos los documentos devueltos, y esto en realidad no suele ser así. Normalmente, el usuario sólo leerá algunos de los primeros documentos devueltos. Para poder realizar mejor el proceso de evaluación, se obtiene una curva que representa la *precisión frente a la eficacia*. Esta curva se obtiene calculando la precisión para varios valores de eficacia. Por ejemplo se escogen 10 valores de eficacia el 10 %, el 20 %, el 30 %, ... 100 %, y para ellos se calcula la precisión, que sería el número de documentos relevantes en el 10 % de los documentos devueltos, en el 20 %, ... y en el 100 %. Así pues un algoritmo con una curva *superior* (el eje Y es la precisión y el X la eficacia) a la de otro algoritmo será mejor.

En el método descrito anteriormente se supone que sólo se ha realizado una consulta. Para el caso de querer evaluar varias consultas distintas, para cada una de ellas tendríamos una curva diferente. De esta forma, se introduce una nueva medida: la media de las precisiones en cada nivel de eficacia, calculada de la siguiente forma:

$$\hat{P}(r) = \sum_{i=1}^{N_q} \frac{P_i(r)}{N_q},$$

donde $P(r)$ es la media de la precisión en el nivel de eficacia r , N_q es el número de consultas usadas y $P_i(r)$ es la precisión al nivel de eficacia r para la consulta i -ésima. Esta medida se usa normalmente para comparar gráficamente algoritmos de recuperación con tan solo comparar las curvas.

Para hacer un estudio más preciso sobre el funcionamiento de una batería de consultas y detectar anomalías en consultas concretas (en vez de observar la media) existen una serie de medidas adicionales bien descritas en el capítulo 3 de [3], como son la precisión media de documentos relevantes vistos, la R -precisión o el histograma de precisión.

Colecciones de prueba

Para la evaluación de modelos nuevos y comparar su funcionamiento con el de otros ya desarrollados se utilizan colecciones de documentos estándar para compararlos con los conjuntos respuesta del algoritmo de recuperación de información. Una colección de prueba está compuesta de tres partes:

1. Un conjunto de documentos.

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

2. Un conjunto (batería) de consultas.

3. Juicios de relevancia.

Estos conjuntos están ya determinados, siendo los más usados los siguientes:

- La colección *TREC* (TExt Retrieval Conference), disponible en http://trec.nist.gov/data/test_coll.html.
- Las colecciones *CACM* y *CISI*, disponibles en <ftp://ftp.cs.cornell.edu/pub/smart/cacm/> y <ftp://ftp.cs.cornell.edu/pub/smart/cisi/>, respectivamente.
- La colección *INEX* (Initiative for the Evaluation of XML Retrieval), especial para la recuperación de documentos estructurados (véase sección 6 de este mismo capítulo).

1.3. Modelos de recuperación de información

1.3.1. Notación utilizada

En los sucesivos ejemplos, suponemos que los modelos de recuperación asignan un valor llamado *peso* a cada uno de los términos de un documento. Así, siendo t_i un término, d_j un documento y $w_{i,j} > 0$ el peso asociado con el par (t_i, d_j) , dicho peso cuantifica la importancia del término para describir el contenido semántico del documento.

Sea T el número de términos en el sistema y t_i un término genérico. Sea $\mathcal{T} = (t_1, \dots, t_T)$ el conjunto de todos los términos. Un peso $w_{i,j} > 0$ se asocia con cada término t_i de un documento d_j . Asimismo, para un término que no aparece en el documento d_j tendremos que $w_{i,j} = 0$. Al documento d_j está asociado un vector de términos $\vec{d}_j = (w_{1,j}, w_{2,j}, \dots, w_{t,j})$. Además se define g_i como una función que devuelve el peso asociado con el término t_i .

Normalmente se asume que los pesos de los diferentes términos son *independientes*, es decir, que el conocer un peso $w_{i,j}$ asociado con un par (t_i, d_j) no nos da información sobre otro peso $w_{i+1,j}$ asociado con el par (t_{i+1}, d_j) . Esto es claramente una simplificación, ya que las ocurrencias de un término en un documento no son independientes, sin embargo, en el capítulo sexto de [35] y en otros se da como hecho base para poder tratar un sistema de estas características.

1.3.2. Modelo booleano

El modelo booleano se basa en la teoría de conjuntos y el álgebra de Boole. Por eso, proporciona un método sencillo a la hora de realizar las consultas y ha sido (y es) ampliamente utilizado en los SRI. Además, es el primero que aparece en la literatura y su formulación es bastante intuitiva.

El sistema que lo implementa recibe una consulta formada por expresiones booleanas de términos. Según esta consulta, se buscan documentos que verifiquen la expresión realizándose un criterio de decisión binario, esto es, un documento es o no es relevante para una consulta concreta.

Definición: En el modelo booleano, el valor de una variable peso de un índice $w_{i,j} \in \{0, 1\}$. Una consulta es una expresión booleana convencional. Sea $\vec{q}_{dnf}$ la forma normal disjunta para la consulta q . Y sea $\vec{q}_{cc}$ cualquiera de la componentes conjuntivas de $\vec{q}_{dnf}$. La similitud de un documento d_j con la consulta q está definida como

$$\text{sim}(d_j, q) = \begin{cases} 1 & \text{si } \exists \vec{q}_{cc} | (\vec{q}_{cc} \in \vec{q}_{dnf}) \wedge (\forall k_i, g_i(\vec{d}_j) = g_i(\vec{q}_{cc})), \\ 0 & \text{en otro caso} \end{cases}$$

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

La principal ventaja es que la semántica del álgebra booleana está completamente especificada; esto hace que su implementación sea sencilla. Desafortunadamente, hay varias desventajas entre las que se encuentran:

- El anteriormente mencionado “criterio de decisión binario” hace que un documento sea relevante o no, sin grado de valoración sobre otros. Así, resulta imposible encontrar documentos más relevantes que otros ya que todos están al mismo nivel.
- El número de resultados devuelto por el sistema, además de ser bastante alto, por el motivo anterior, suele estar desordenado respecto a la relevancia *real* de los documentos, lo cuál dificulta al usuario para seleccionar de entre los resultados mostrados aquellos que satisfagan más su necesidad.

Por estos motivos, se proponen modelos más avanzados en la línea de este mismo (modelo booleano extendido, por Shalton, Fox y Wu en 1983), o como los que describimos a continuación, algo más complejos.

1.3.3. Modelo de espacio vectorial

Es posible concluir, sin un análisis muy profundo, que normalmente, para una consulta concreta del usuario, van a existir documentos más o menos relevantes. Intuitivamente, un documento d_j puede ser más relevante que otro d_k para un término t_i si, por ejemplo, t_i aparece en el título de d_j y varias veces en su cuerpo, mientras que en d_k no apareciera ninguna vez.

La forma de obtener esta “ponderación” es la de asignar a cada par (t_i, d_j) (*término, documento*) un **peso** $w_{i,j}$, real y positivo, y mayor cuanto más relevante sea t_i en el documento d_j . Cada documento entonces, puede considerarse como un **vector** en el espacio vectorial⁶ del conjunto de todos los términos de nuestro vocabulario, donde para un documento d_j cada componente i -ésima tendrá el valor del peso $w_{i,j}$, quedando la representación de cada documento como una n -upla⁷ (donde n es el número de términos del sistema).

Este modelo permite una representación de la consulta de la misma forma que un documento. La *distancia* (o disimilitud) entre un documento y la consulta se define como el coseno del ángulo entre los dos vectores. Numéricamente:

⁶El término “espacio vectorial” está aquí usado del mismo modo que se usa en la bibliografía básica de Recuperación de Información. No obstante, el autor quiere hacer hincapié en lo desafortunado de la elección de esta nomenclatura ya que, en su opinión, no existe un verdadero espacio vectorial de documentos como tal (por ejemplo, no existe el opuesto de un vector), sino que se aprovecha la representación de n -upla que puede tener un documento en esta forma.

⁷Se sobreentiende que, $\forall t_k : t_k \notin d_j | w_{k,j} = 0$

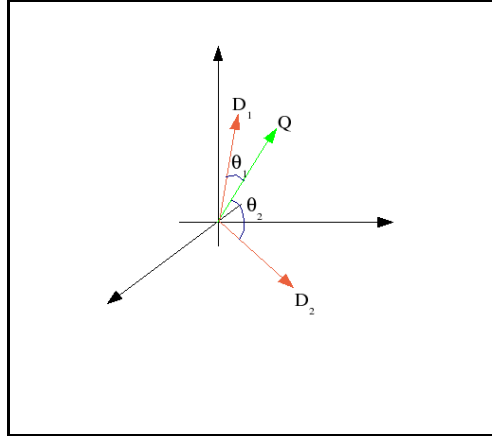


Figura 1.3: Ejemplo de modelo de espacio vectorial: dada la consulta Q , D_1 es más relevante que D_2 al ser el ángulo θ_1 menor que θ_2 .

$$\text{sim}(q, x_i) = \vec{Q} \cdot \vec{X}_i = \frac{\sum_{j=1}^m v_j \cdot w_{ij}}{\sqrt{\sum_{j=1}^m (v_j)^2 \cdot \sum_{j=1}^m (w_{ij})^2}}$$

Nótese que el coseno se calcula como el producto escalar entre los dos vectores normalizado.

El funcionamiento de un sistema que implemente este método es bastante sencillo: dada una consulta, convertirla en vector y calcular la distancia entre todos los demás vectores (documentos) de la colección. Devolver los más cercanos, ordenados por relevancia (distancia) ascendente. A menor distancia, mayor relevancia.

1.3.4. Modelo probabilístico básico

Este modelo está principalmente basado en el *principio de ordenación probabilístico*, definido por W. S. Cooper y recogido en el capítulo 6 de [35]. El principio dice así:

Si la respuesta de un SRI a cada consulta es una ordenación de documentos de la colección, en orden decreciente de la probabilidad de relevancia al usuario que ha enviado dicha consulta, y donde las probabilidades se estiman de la forma más precisa posible basándonos en cualquier dato proporcionado al sistema para este propósito, la efectividad media del sistema de cara al usuario será la mejor que se puede obtener con base en estos datos.

CAPÍTULO 1. FUNDAMENTOS TEÓRICOS DE LA RECUPERACIÓN DE INFORMACIÓN

Así, dada una consulta q , existe un subconjunto de documentos $R \subset D$ que son relevantes dada q . El principal problema es que no es sencillo calcular la membresía de dicho conjunto. Aunque podemos ver la consulta como una descripción de las propiedades de este conjunto, no sabemos exactamente qué propiedades son éstas, sino que todo lo que conocemos es que existen términos que describen estas propiedades. Por tanto, cuando se realiza la consulta, lo primero que necesitamos es obtener la descripción probabilística preliminar del conjunto R , y a partir de ella un primer conjunto de documentos.

Definición: definimos los pesos para los términos y las consultas como binarios, es decir, $w_{i,j} \in \{0, 1\}$ y $w_{i,q} \in \{0, 1\}$. Sea R el conjunto de los documentos que serán relevantes y $\bar{R}$ su complemento. Sea $P(R|\vec{d}_j)$ la probabilidad de que el documento d_j sea relevante para la consulta y $P(\bar{R}|\vec{d}_j)$ la probabilidad de que no lo sea. Se define la *similitud* como:

$$\text{sim}(d_j, q) = \frac{P(R|\vec{d}_j)}{P(\bar{R}|\vec{d}_j)}$$

Esta expresión se deriva (utilizando el Teorema de Bayes y suponiendo la independencia de los términos índice) hasta obtener una fórmula más simple que es la función de valoración en el modelo probabilístico. (para más detalles, consultar [3], capítulo segundo):

Ya que inicialmente no conocemos el conjunto R , es necesario un método que inicialice los valores de las probabilidades $P(\vec{d}_j|R)$ y $P(\vec{d}_j|\bar{R})$. A continuación veremos uno de los métodos más comunes. En él se hacen dos suposiciones: primero supone que $P(\vec{d}_j|R)$ es constante para todos los términos y segundo supone que la distribución de términos entre los documentos no relevantes es la misma que la distribución de los términos entre todos los documentos.

$$P(\vec{d}_j|R) = 0.5 \cdot P(\vec{d}_j|\bar{R}) = \frac{n_i}{N}$$

Siendo n_i el número de documentos que contienen el término t_i y N el número total de documentos.

A continuación, una vez que ya están inicializadas estas probabilidades (los documentos están ordenados en una clasificación), se sigue iterando, para obtener un conjunto respuesta mejor, como se explica a continuación. Sea V un subconjunto de los documentos recuperados inicialmente y ya ordenados, (V puede obtenerse por ejemplo eligiendo los primeros n documentos del conjunto) y sea V_i el subconjunto de V compuesto por los documentos de V que contienen el término t_i , para ir mejorando la selección hay que mejorar las probabilidades $P(\vec{d}_j|R)$

y $P(\vec{d}_j|\vec{R})$, para ello hacemos dos suposiciones: primera, que podemos aproximar $P(\vec{d}_j|R)$ a la distribución de los términos t_i entre los documentos recuperados y segunda, que podemos aproximar $P(\vec{d}_j|\vec{R})$ considerando que todos los documentos no recuperados son irrelevantes.

$$P(\vec{d}_j|R) = \frac{V_i}{V} P(\vec{d}_j|\vec{R}) = \frac{n_i - V_i}{N - V}$$

Este proceso puede ser repetido recursivamente, para mejorar la estimación de las probabilidades, pudiendo intervenir o no el usuario.

La principal ventaja del modelo probabilístico es que los documentos se ordenan en orden decreciente de su probabilidad de ser relevantes. Las desventajas son que necesitamos estimar la separación inicial del conjunto en relevantes y no relevantes, el hecho de que los pesos son binarios y la suposición de que los términos son independientes, aunque esto último es prácticamente necesario para no hacer este problema intratable.

Una panorámica más amplia de los modelos probabilísticos de recuperación de información puede encontrarse en [8].

Capítulo 2

Redes Bayesianas. Modelos basados en Redes Bayesianas para Recuperación de Información no estructurada

2.1. Incertidumbre y Recuperación de Información

¿Por qué tratar con incertidumbre? Muchas definiciones se han dado de la Recuperación de Información, pero todas coinciden en una cosa: éste no es un problema con una única solución.

Así, y como hemos visto antes, la recuperación de información se basa en el principio de obtener documentos relevantes ante la necesidad de un usuario de obtener cierta información, y no únicamente se refiere a encontrar patrones que sean correspondientes entre sí (véase [35], capítulo 1).

Para [3] y [35] este último punto es el que marca la diferencia entre lo que es la recuperación de información y la recuperación únicamente de datos. El gran acervo de información de una biblioteca digital hace que la búsqueda dentro de la misma sea una tarea difícil, ya que no se requiere únicamente saber que documentos contienen alguna palabra (recuperación de datos), sino que se busca que realmente el documento que contiene algún término de la consulta del usuario sea relevante de acuerdo a un grado de similitud que se produzca junto con los resultados (recuperación de información). De hecho, en [35] se pone bastante énfasis desde el principio en diferenciar *data retrieval* (“recuperación de datos”, proceso

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

similar al que realizaría un SGBD¹) de *information retrieval* (es decir, recuperación de información).

Ese hecho, la “relevancia” de un documento, es en sí misma un término impreciso. ¿Qué hace a un documento relevante o no, dadas diferentes consultas por un usuario? ¿Puede un mismo documento ser relevante o no para consultas parecidas? Como vemos, un criterio de decisión binario (como el del modelo de recuperación booleano) no es suficiente para representar la relevancia de un documento.

Desde la segunda mitad del S.XX en adelante, la Inteligencia Artificial [18] ha propuesto varios modelos numéricos para tratar con incertidumbre como son los siguientes:

- El modelo *probabilista clásico*, que usa la probabilidad como herramienta principal y medida de la creencia personal².
- La teoría de los *conjuntos difusos* (o lógica difusa³), surgida a partir de los trabajos de L. Zadeh [45]. En esta teoría se estudia la vaguedad de una forma en la que un elemento tiene un “grado de pertenencia” a un conjunto (un número real entre 0 ó 1), en vez del clásico “pertenece” y “no pertenece”.
- La teoría de las *redes bayesianas* propuesta por J. Pearl (véase [31]) que será la que estudiemos detalladamente. Es como generalización del modelo probabilista clásico donde se representa mediante un grafo dirigido acíclico las relaciones entre variables aleatorias de un sistema concreto.
- Otras técnicas como son la teoría de Dempster-Shafer, la teoría de la posibilidad, o el modelo de factores de certeza de MYCIN (esta última estudiada con más detalle en [18]).

Aunque sería lógico suponer que el lector conoce y maneja con fluidez los conceptos básicos de la teoría de la probabilidad, incluimos el siguiente apartado, a título de introducción, por si no se da el caso anterior. También puede servir, a modo de aclaración, como guía de los diferentes términos utilizados en secciones posteriores.

¹Sistema de Gestión de Bases de Datos

²No nos referimos a la interpretación de la probabilidad “a largo plazo” estudiada a principios del S.XX

³*Fuzzy Logic* en el original

2.2. Conceptos previos sobre teoría de la probabilidad

Sea un conjunto U de sucesos elementales y una familia de sucesos $\mathbf{B}$. Si U es finito, $\mathbf{B} = \mathcal{P}(U)$ (el conjunto de las partes de U).

Definición: una *medida de probabilidad* sobre el par $(U, \mathbf{B})$ es una aplicación $P : \mathbf{B} \rightarrow [0, 1]$ que verifica:

- $P(U) = 1$.
- Si A y C son disjuntos, $P(A \cup C) = P(A) + P(C)$.
- Si $\{A_i\}_{i \in I}$ es una familia numerable de elementos de $\mathbf{B}$, numerables dos a dos, entonces $P(\cup_{i \in I} A_i) = \sum_{i \in I} P(A_i)$.

Tras esta definición se pueden derivar varios teoremas, entre los que están:

1. $P(\emptyset) = 0$
2. Si $A_1 \subset A_2$, entonces, $P(A_1) \leq P(A_2)$ y $P(A_2 - A_1) = P(A_2) - P(A_1)$.
3. $P(\bar{A}) = 1 - P(A)$
4. Si A y B son dos eventos cualesquiera, entonces $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

Dados dos sucesos A y B , un concepto “natural” que surge es el de probabilidad condicional:

Definición: la *probabilidad condicional* $P(A|B)$ (“probabilidad del suceso A condicionada al suceso B ”) es igual a

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \forall B \neq \emptyset$$

y su interpretación es tal que simboliza la probabilidad de ocurrencia del suceso A , habiendo ocurrido el suceso B .

Los dos resultados siguientes son resultados básicos de la teoría de la probabilidad y su uso es necesario en casi cualquier problema que incorpore el concepto de probabilidad condicional:

Teorema 2.2.1 (de la probabilidad total) Si $\{H_i\}_{i \in I}$ es una colección de sucesos disjuntos dos a dos y cuya unión es el suceso seguro U , se verifica para todo suceso B que

$$P(B) = \sum_{i \in I} P(B|H_i)P(H_i)$$

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

Teorema 2.2.2 (de Bayes) Si $\{H_i\}_{i \in I}$ es una colección de sucesos disjuntos dos a dos y cuya unión es el suceso seguro U , se verifica para todo suceso B que

$$P(H_j|B) = \frac{P(H_j \cap B)}{P(B)} = \frac{P(B|H_j)P(H_j)}{P(B)} = \frac{P(B|H_j)P(H_j)}{\sum_{i \in I} P(B|H_i)P(H_i)}$$

Cuando trabajemos con redes bayesianas, hablaremos de *variables aleatorias*. Una variable aleatoria, formalmente, no es más que una aplicación de cada elemento del conjunto de sucesos en $\mathbb{R}$. Generalmente se representará como una letra mayúscula, que puede tener subíndices ($D_{10}, T_2, \dots$), y el conjunto de sus valores concretos como las correspondientes letras minúsculas ($D_1 = \{d_1, \bar{d}_1\}$). Muchas veces, más que interesarnos los valores numéricos, utilizaremos esta representación con objeto de evaluar una cierta $P(X = x)$.

Las variables aleatorias pueden tomar valores en conjuntos finitos o infinitos. En el primer caso, hablamos de *variables aleatorias discretas* y en el segundo, de *variables aleatorias continuas*. En este trabajo sólo consideraremos variables discretas, puesto que las continuas suelen tener un significado más físico.

El último concepto importante que vamos a exponer es el de *distribución de probabilidad*⁴.

Definición: una distribución de probabilidad p es una aplicación definida $\forall x \in X$ como

$$\begin{aligned} p : X &\longrightarrow [0, 1] \\ p(x) &\longmapsto P(X = x) \end{aligned}$$

Extendiendo este concepto a un conjunto formado por n variables $\{X_1, \dots, X_n\}$ podemos definir la *distribución de probabilidad conjunta* para un cierto valor $(x_1, \dots, x_n) \in \{X_1, \dots, X_n\}$.

$$\begin{aligned} p : \{X_1, \dots, X_n\} &\longrightarrow [0, 1] \\ p(x_1, \dots, x_n) &\longmapsto P(X_1 = x_1, \dots, X_n = x_n) \end{aligned}$$

Finalmente, definimos la *distribución de probabilidad condicionada* de un conjunto de variables $\{X_1, \dots, X_n\}$ al valor de una variable $Y = y$ como

$$p : \{X_1, \dots, X_n\} \longrightarrow [0, 1]$$

⁴Una distribución de probabilidad se utiliza, como va a verse, para expresar la probabilidad de que ocurra un cierto suceso. Por abuso del lenguaje, a veces escribiremos $P(x)$ (expresión que no tiene sentido alguno), donde x es un valor de X . Obviamente, la forma correcta sería $p(x)$ ó $P(X = x)$.

2.2. CONCEPTOS PREVIOS SOBRE TEORÍA DE LA PROBABILIDAD

$$p(x_1, \dots, x_n | y) \mapsto P(X_1 = x_1, \dots, X_n = x_n | Y = y)$$

Que puede interpretarse como “para cada valor y de la variable Y , doy la probabilidad de cualquier conjunto de valores del conjunto de variables $\{X_1, \dots, X_n\}$.”

Tras esta interpretación, de forma natural surge la posibilidad de condicionar a varias variables, dando, para todo valor del conjunto de variables que condicionan (los que están a la derecha de la barra “|”) la probabilidad de los distintos valores del conjunto de variables condicionadas (los que están a la derecha de la barra).

2.3. Redes Bayesianas: introducción

2.3.1. Definición. Ejemplo

Una red bayesiana ([10], [31], [24], [25]) consta de dos componentes. El primero de ellos, más cualitativo, está representado por un grafo acíclico dirigido $G = (V, E)$ donde los nodos (el conjunto finito V) son variables aleatorias del problema, y los arcos ($E \subset V \times V$) indican relaciones entre variables. El segundo de ellos, cuantitativo, se trata de un conjunto de distribuciones de probabilidad condicionadas (una por nodo) donde la distribución en cada nodo está condicionada al posible valor de cada uno de los padres.

Aunque puede pensarse que una red bayesiana de nodos $X_i, i = 1, \dots, m$ no es más que una expresión más cómoda para la distribución de probabilidad conjunta $p(x_1, \dots, x_n)$, realmente es más que eso. Como veremos, su estructura permite determinar fácilmente relaciones de independencia entre diferentes variables, lo cuál puede ser útil a la hora de realizar cálculos.

Un ejemplo de la construcción de una red bayesiana (solamente su estructura) puede ser el dado a partir del siguiente ejemplo (*icy roads*, de [24]):

El inspector de policía Smith esperaba impacientemente la llegada del Sr. Holmes y del Dr. Watson. Ambos llegan tarde, y el inspector tiene después una importante cita (el almuerzo). Mirando a la ventana, se pregunta si las carreteras están heladas. Como ambos son realmente malos conductores, si las carreteras se hielan, podrían tener un accidente.

De repente, su secretaria entra en la habitación y le comunica que Watson ha tenido un accidente de tráfico. Inmediatamente, Smith piensa que las carreteras deben estar heladas y dice “como las carreteras están heladas, Holmes probablemente habrá tenido un accidente también, por lo que me iré a comer”.

Su secretaria contesta, por su parte que “las carreteras no están heladas en absoluto” y que “han puesto sal en la mayoría”. Smith replica “está bien, Holmes puede no haber tenido un accidente; démosle un poco más de tiempo”.

En la figura 2.1, el nodo “Hielo” corresponde a una variable aleatoria (llamémosla He) cuyos valores podrían ser dos: “hay helada”, “no hay helada”. Por su parte, los nodos “Holmes” y “Watson” corresponden a dos variables aleatorias (Ho y W) cuyos valores son “tiene accidente”, “no tiene accidente”. Para completar la especificación de la red, tendríamos que proporcionar tres tablas⁵

⁵Por abuso del lenguaje y porque una forma de especificar la distribución de probabilidad de

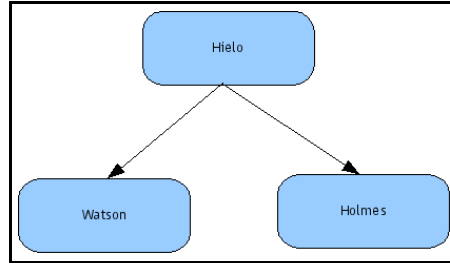


Figura 2.1: Red bayesiana del ejemplo “icy roads”.

correspondientes a $P(He)$, $P(Ho|He)$ y $P(W|He)$ para las variables He , Ho y W , respectivamente. Posteriormente volveremos a hacer mención al ejemplo para introducir nuevos términos.

2.3.2. Independencia e independencia condicional

Uno de los conceptos clásicos de la teoría de la probabilidad que no se ha mencionado en el punto anterior es el de *independencia de sucesos*; se dice que dos sucesos (representados por las variables aleatorias A y B) son independientes si $P(A = a, B = b) = P(A = a) \cdot P(B = b)$, o lo que es lo mismo, la probabilidad de que ocurra uno de los sucesos es independiente de que ocurra el otro. Dos variables aleatorias se dicen independientes si lo son para todos sus valores. Resulta fácil pensar en sucesos independientes, pues estamos rodeados de ellos. Por ejemplo, la probabilidad de que un día sea fiesta en Granada es independiente de que ese mismo día el precio del petróleo suba en Estados Unidos (siempre y cuando no exista una complicada conspiración que relacione ambos sucesos).

Para problemas con un amplio número de variables, aparece un fenómeno que relaciona a más de dos de ellas y que también es estudiado: el de *independencia condicional*. Sean tres variables aleatorias X , Y y Z . Decimos que X e Y son *condicionalmente independientes dada Z* si y sólo si todo par de valores x e y es condicionalmente independiente para cada z tal que $p(z) > 0$; es decir:

$$\forall x, \forall y, \forall z, p(z) > 0 \Rightarrow p(x, y|z) = p(x|z) \cdot p(y|z)$$

El concepto de la independencia condicional es extensible a conjuntos de variables.

Si existe una variable que está *observada* (es decir, ha tomado un cierto valor de su conjunto de valores), y queremos comprobar independencias respecto a ella, basta con comprobar la independencia con dicho valor. Generalmente, al observar

una variable discreta es mediante una tabla de valores, a veces utilizaremos el término *tabla* para referirnos en sí a una distribución, aunque no sea del todo correcto

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

una variable, pueden cambiar las relaciones de independencia de un conjunto de variables.

Si un conjunto de variables $\{X_1, \dots, X_n\}$ es independiente habiéndose observado otro conjunto $\{Y_1, \dots, Y_n\}$, lo notaremos como

$$I(X_1, \dots, X_n | Y_1, \dots, Y_n)$$

Obviamente, la independencia condicional es un caso general del condepto de independencia de variables anteriormente mencionado, ya que podemos decir que las variables del conjunto $\{X_1, \dots, X_n\}$ son independientes entre sí, si $I(X_1, \dots, X_n | \emptyset)$ (siendo esa una forma válida de notarlo).

2.3.3. *d*-Separación y criterios gráficos de independencia

Resulta bastante sencillo encontrar criterios *gráficos* mediante los cuales podemos encontrar relaciones de independencia (condicional o no) entre conjuntos de variables tan solo observando el grafo correspondiente a la red bayesiana. El más importante es el llamado criterios de *d*-separación (véase [5] para un desarrollo más profundo del que se hace aquí).

Definición: decimos que un nodo de una red bayesiana, para dos arcos concretos incidentes en él, es *cabeza-cabeza* si dichos arcos le apuntan. En caso contrario, diremos que es *no cabeza-cabeza* (ver figura 2.2 para más detalle).

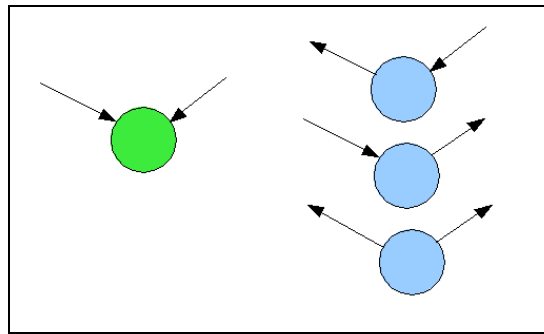


Figura 2.2: A la izquierda, nodo cabeza-cabeza. A la derecha, las tres posibles formas de nodo no cabeza-cabeza.

Definición: (*separación gráfica*) decimos que X es independiente de Y dado $Z_1, \dots, Z_k$ si todo camino (utilizando los arcos en ambas direcciones) entre X e Y está bloqueado en algún nodo debido a $Z_1, \dots, Z_k$. Las formas de bloqueo (*d*-separación) de un nodo son dos:

- Un nodo no cabeza-cabeza observado en el camino.
- Un nodo cabeza-cabeza no observado, ni ninguno de sus descendientes no observados, en el camino.

Estas reglas son demostrables a partir de la definición de independencia, pero dicha demostración no la veremos aquí, pudiendo el lector remitirse a [31] para encontrarla.

En el ejemplo anterior, *icy roads*, en la red inicial sin observaciones (figura 2.1), se puede observar que las variables “Watson” y “Holmes” no son independientes puesto que el camino entre ambas no está bloqueado. De esta forma, Smith piensa que si Watson ha tenido un accidente, existe probabilidad también de que a Holmes le haya sucedido lo mismo.

Tras el anuncio de la secretaria de que no hay una helada, conocemos el valor del nodo “Hielo” (queda observado), quedando la red correspondiente de la figura 2.3.

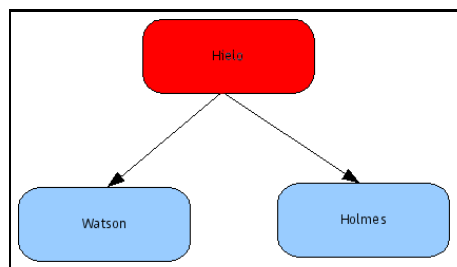


Figura 2.3: La secretaria avisa a Smith de que no hay helada, por lo que modificamos la red poniendo el nodo “Hielo” como observado (en rojo).

Como el nodo “Hielo” es no cabeza-cabeza y está observado, y bloquea todos los caminos entre “Holmes” y “Watson”, podemos decir que estos dos últimos son independientes habiéndose observado el primero.

2.3.4. Causalidad. Razonamiento abductivo

Otra de las características más potentes de las redes bayesianas es su capacidad para realizar lo que se llama *razonamiento abductivo*.

El razonamiento “clásico” que se ha venido utilizando en inteligencia artificial es inductivo (partiendo de hechos particulares, dar una conclusión general) ó deductivo (partiendo de categorías generales, dar hace afirmaciones sobre casos particulares, utilizando reglas de inferencia⁶). La investigación científica y el

⁶Generalmente *modus ponens* y *modus tollens*.

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

aprendizaje de conceptos abstractos son ejemplos de razonamiento inductivo, y el pensamiento lógico y la extracción de características de un individuo por su pertenencia a una clase son ejemplos de razonamiento deductivo.

Existe un tercer tipo de conocimiento, llamado *abductivo* que complementa al deductivo, y que ha sido frecuentemente olvidado en los sistemas basados en conocimiento. Si la inferencia deductiva podemos representarla como la regla modus ponens,

$$\frac{\alpha \longrightarrow \beta \quad \alpha}{\beta}$$

la inferencia abductiva, por su parte, opera en sentido contrario (busca explicación para las conclusiones) y podemos representarla como

$$\frac{\alpha \longrightarrow \beta \quad \beta}{\alpha}$$

Aunque esta regla no siempre es cierta a rajatabla, existen muchos casos donde tras haber encontrado una conclusión (evidencia), aumentamos la creencia de que haya podido ocurrir el antecedente.

Las redes bayesianas representan conocimiento deductivo: una red que contiene dos nodos A y B y un arco de A a B , representa de algún modo $A \Rightarrow B$ porque la probabilidad de un valor de B está *condicionada* a la de A . También representan conocimiento abductivo: si conozco la probabilidad de B , puede variar mi conocimiento sobre A (habrá un antecedente más probable que otro).

Siguiendo la dirección de los arcos, se sigue la *causalidad* de los nodos. Un nodo A es *causa* de otro B , si existe un camino (dirigido) desde A hasta B ⁷. Por este motivo, a las redes bayesianas también se les suele llamar *redes causales*.

En el ejemplo *icy roads*, el razonamiento abductivo se plasma en la conclusión (tal vez adelantada) del inspector Smith de que habría una helada sabiendo que Watson ha tenido un accidente. En este caso, decimos que *aumenta la certeza* de helada partiendo de la *evidencia* de que Watson ha tenido un accidente. Como puede comprobarse, es un razonamiento que las personas estamos acostumbradas a aplicar.

⁷No puede existir un camino en ambas direcciones porque el grafo ya no sería acíclico y estaríamos tratando con algo que no es una red bayesiana. En otras palabras, añadir ciclos eliminaría las relaciones de causalidad (lo que no nos interesa, puesto que es la mitad del conocimiento del que disponemos).

2.3.5. Propagación de evidencia. Inferencia en Redes Bayesianas

Antes de realizar ninguna observación, una red bayesiana representa nuestra conocimiento *a priori* del sistema. Cuando recibimos alguna información, tenemos un conocimiento *a posteriori*. El problema surge cuando se intenta dar un enfoque algorítmico al cálculo de la probabilidad de un nodo en función de los demás (pueden estar observados o no). Si algunos nodos están observados, se produce una “propagación de conocimiento”. Esta “propagación del conocimiento” se realiza mediante reglas bien conocidas: la *propagación de evidencia*.

La evidencia está representada por variables observadas (aquellas de las que conocemos su valor) y se plasma en una red, por el diferente color de dichos nodos. En ocasiones, se suele decir que los nodos (o las variables) están *instanciados*.

El cálculo de dichas probabilidades es, en general NP-duro ([7]) por lo que se proponen dos alternativas:

- Utilizar un algoritmo de propagación *exacto* (pero muy ineficiente debido a la dificultad del problema). Normalmente se suele guiar a estos algoritmos con buenas heurística a la hora de dar cada paso con objeto de agilizarlo.
- Utilizar un algoritmo de propagación *aproximado* a costa de perder exactitud y ganar eficiencia. Si la red es demasiado grande o el tiempo en el sistema es crítico, se puede utilizar uno de estos algoritmos, a costa de perder exactitud en los cálculos.

No entramos en detalle en dichos algoritmos puesto que no son el objetivo de esta memoria. Si el lector quiere información sobre ellos, puede referirse a [5] o a [31].

Nótese que el problema es NP-duro en el caso de una red bayesiana general. Si la red es un poliárbol⁸, el problema puede resolverse en un tiempo polinómico con el algoritmo de Pearl (véase [31] para más detalle).

El proceso de inferencia es sencillo: partiendo de $p(x_1, \dots, x_m)$, poder obtener la distribución $p(y_1, \dots, y_k | z_1, \dots, z_j)$ ⁹. Más concretamente, muchas veces interesará obtener la distribución de probabilidad de un cierto nodo, condicionado a los demás.

⁸Un poliárbol es un grafo en el que un nodo puede tener varios padres, pero en el que no pueden existir múltiples trayectorias entre dos nodos concretos.

⁹Donde obviamente $\{y_1, \dots, y_k\} \subset \{x_1, \dots, x_m\}$ y $\{y_1, \dots, y_k\} \cup \{z_1, \dots, z_j\} = \{x_1, \dots, x_m\}$.

2.4. Modelos de Recuperación de Información basados en Redes Bayesianas

2.4.1. Primeros modelos

Los dos primeros modelos recogidos en la literatura son el *Inference Network Model* de Turtle y Croft (estudiado en [42]) y el *Belief Network Model* de Ribeiro y Reis (desarrollado en [34]). El primero data de 1991, y el segundo de 1996, lo que da una idea de lo relativamente recientes que son ambos. Una revisión de la bibliografía referente a estos modelos y a otros posteriores puede encontrarse en [11].

Inference Network Model

En el *Inference Network Model*, se utilizan dos redes bayesianas: de documentos y de consultas. La red de documentos, fija durante el proceso, está compuesta básicamente por dos tipos de nodos representando documentos y términos de la colección. Cada documento se conecta con los términos indexados que contiene apuntando un arco desde él al término. La red de consultas por su parte, es variable, y representa las necesidades de información del usuario, añadiendo un nodo por cada consulta proporcionada. En la figura 2.4 podemos observar el modelo con las dos subredes bien diferenciadas: la red de documentos (*document network*) y la red de consultas (*query network*).

La red de documentos representa la colección de documentos y consiste en un nodo por cada documento¹⁰ (representado por d_i) y por cada término (representado por t_i). Se construye una sola vez para cada colección y su estructura no varía. La red de consultas, por su parte, representa necesidades concretas de información y se construye (de forma dinámica) con base en la interacción del usuario con el sistema. Sus componentes son los nodos de conceptos (si necesitamos su aparición en el problema concreto), c_i , para “afinar” la semántica de los diferentes términos, los nodos consulta q_i , y las necesidades de información i_i .

Las variables asociadas a cada nodo son binarias (valores *true* y *false*), realizándose el proceso de inferencia generalmente estimando la probabilidad $P(q_k = true | d_k = true)$.

La importancia del modelo radica en que es capaz de generalizar a muchos otros modelos clásicos, adaptando las probabilidades del *Inference Network Model* a otros.

¹⁰Por abuso del lenguaje, y de ahora en adelante, nos referiremos a la correspondencia entre entidad y nodo de una red bayesiana, aunque realmente tendríamos que corresponder cada nodo con una variable aleatoria (asociada a la entidad representada).

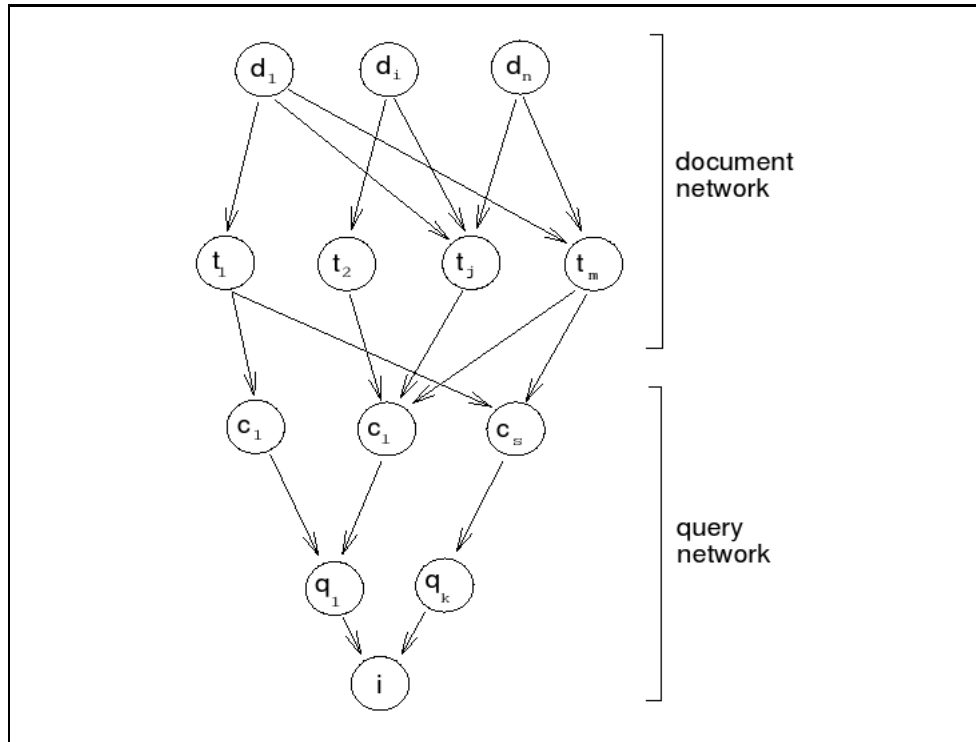


Figura 2.4: Red correspondiente al *Inference Network Model* de Turtle y Croft.

Belief Network Model

Posterior al modelo anterior, éste presenta una estructura bastante distinta a pesar también de estar basado en redes bayesianas. En este caso (véase figura 2.5) aunque se mantiene un nodo por documento (d_i) y término (k_i) y se introduce otro nodo por cada consulta (nodo q), las relaciones causales de la red están invertidas respecto al modelo anterior, siendo invertida la dirección de los arcos entre términos y documentos.

Al igual que el modelo anterior, todas las variables aleatorias son binarias. En este caso, el proceso de inferencia también realiza la estimación de $P(d_i|q)$, utilizando en este caso que (notándose en este caso los términos por t_k).

$$P(d_i|q) \propto \sum_{t_j \in d_i} P(d_i|t_j) \cdot P(q|t_j) \cdot P(t_j)$$

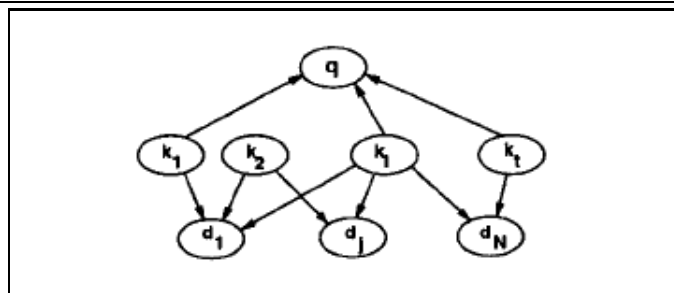


Figura 2.5: Red correspondiente al *Belief Network Model* de Ribeiro y Reis.

2.4.2. El modelo BNR

Descripción del modelo

El *Bayesian Network Retrieval Model*¹¹ ([12]) utiliza también redes bayesianas como modelo subyacente de recuperación. Las características que guían su construcción son las siguientes:

1. Hay una relación muy fuerte entre un documento y los términos que lo indexan.
2. Las relaciones entre documentos sólo están presentes por medio de los términos que comparten.
3. Varios documentos son condicionalmente independientes dados los términos por los que fueron indexados.

Las variables incluidas serán de dos tipos: T_i para los términos y D_j para los documentos. En ambos casos consideraremos dos valores para cada variables ($\{t_i, \bar{t}_i\}$ y $\{d_j, \bar{d}_j\}$) que equivalen a decir que el término (o documento) es o no relevante (respectivamente).

La estructura del modelo, de forma general, quedaría definida con el siguiente esquema (ver figura 2.6).

El modelo presenta diferentes variantes según las relaciones consideradas en cada una de las dos subredes. Así, en el modelo más simple (*Simple Bayesian Network Retrieval Model*, o SBNR), sólo se consideran las relaciones entre términos y documentos, quedando vacías las relaciones entre términos y entre documentos. Este modelo se describe en [1], y podemos observar su estructura en la figura 2.7.

¹¹También citado en este documento como “Modelo BNR” o “BNRM”.

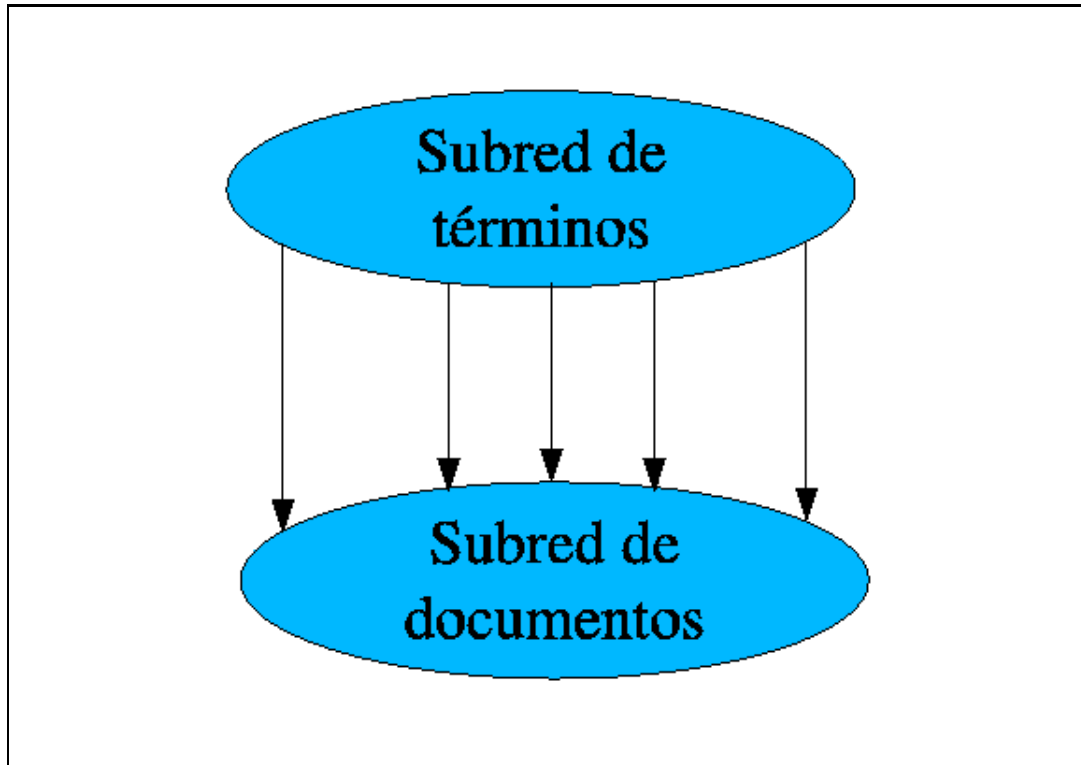


Figura 2.6: Esquema general del modelo BNR.

Dada la estructura, quedaría por especificar la parte cuantitativa de la red (esto es, las distribuciones de probabilidad). Para todos los términos, suponemos la misma distribución de probabilidad (a priori),

$$p(t_i) = \frac{1}{M}, p(\bar{t}_i) = 1 - p(t_i) = \frac{M-1}{M}$$

donde M es el número de términos del sistema.

El problema de esta estructura es especificar la distribución asociada a cada documento, ya que tiene unos requisitos muy grandes en espacio. Por poner un ejemplo, si un documento está indexado (contiene) 50 términos diferentes, especificar $p(d_j|Pa(D_j))$ ¹² implicaría almacenar 2^{50} valores distintos. Como los documentos van a tener un número de términos indexables mucho mayor (del orden de cientos o incluso miles), los requisitos de almacenamiento serían colosales. La solución dada en [12] y [1] proponer especificar las distribuciones de probabilidad de los términos **sólo en función de los términos padre que sean relevantes**.

¹²La notación $Pa(X)$ hace referencia a los padres del nodo X en la red bayesiana. Una vez más, por abuso del lenguaje se entremezclan la terminología referida a variables aleatorias y al grafo propiamente dicho.

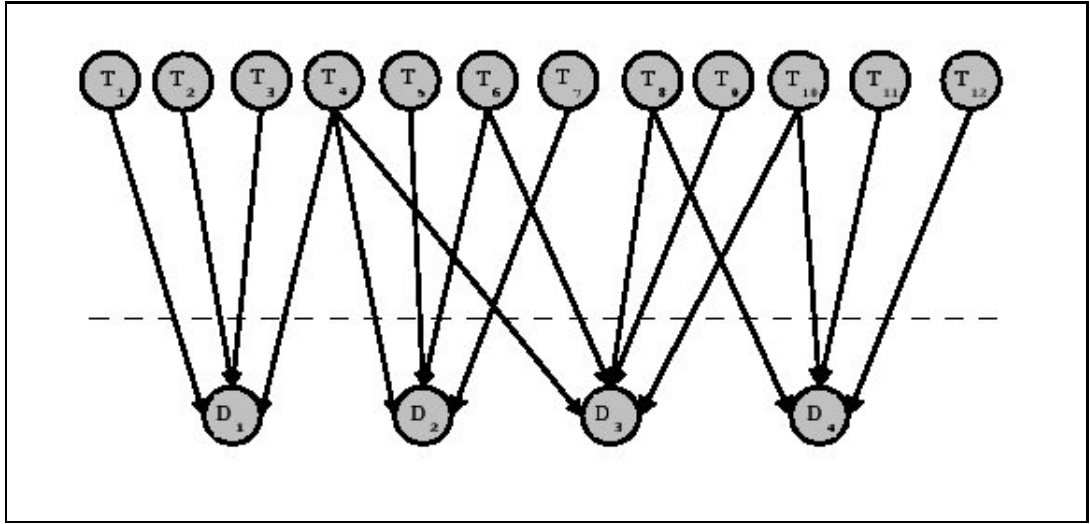


Figura 2.7: Modelo simple BNR.

Así pues, en [12] se desarrolla lo que se conoce como un *modelo canónico de interacción multicausal* o *función de probabilidad*. Este modelo canónico define, para cualquier configuración $Pa(D_j)$ la probabilidad condicional de relevancia tal y como sigue:

$$p(d_j|Pa(D_j)) = \sum_{T_i \in R_{Pa(D_j)}} w_{ij}$$

siendo $R_{\{T_i, \dots, T_x\}}$ el conjunto de términos relevantes entre $\{T_i, \dots, T_x\}$. Además, los valores w_{ij} (pesos) verifican las dos siguientes propiedades:

1. $w_{ij} \geq 0, \forall i = 1, \dots, m_j$
2. $\sum_{T_i \in D_j} w_{ij} \leq 1$

De esta forma, a más términos relevantes en D_j , mayor será la probabilidad de relevancia de D_j (algo que es obvio y no está en contra del planteamiento intuitivo del problema). Por otra parte, las necesidades de almacenamiento se ven reducidos radicalmente pasando de 2^{50} valores en el supuesto del caso anterior a 50 con el nuevo método (pasamos de un requisito *exponencial* a uno *lineal*).

Inferencia en el modelo BNRM

Construida la red bayesiana, puede ser usada para predecir el valor que una variable tomará. La inferencia en este caso será el proceso para calcular, dada una

consulta Q , el conjunto de documentos que son relevantes, ordenados por la probabilidad $p(d_j|Q), \forall D_j$.

La propagación utilizando las funciones de probabilidad anteriormente definidas es la siguiente:

$$p(d_j|Q) = \sum_{i=1}^{m_j} w_{ij} \cdot p(t_i|Q)$$

En [12] puede encontrarse la demostración de que la distribución de probabilidad $p(d_j|Pa(d_j))$ puede expresarse tan solo en función de los términos relevantes, de igual forma que se ha hecho aquí.

Para cerrar la descripción anterior, faltaría concretar el valor de los diferentes pesos w_{ij} . El valor elegido es el siguiente:

$$w_{ij} = \alpha^{-1} \frac{tf_{ij} \cdot idf_i^2}{\sqrt{\sum_{T_k \in D_j} tf_{kj} \cdot idf_k^2}}$$

donde tf_{ij} simboliza la frecuencia del término T_i en el documento D_j , idf_i es la frecuencia documental inversa¹³ del término T_i , y α es una constante para garantizar que $\sum_{T_i \in D_j} w_{ij} \leq 1$.

El modelo BNR completa su descripción con la especificación de las subredes de términos y documentos.

Subred de términos

En este caso trataremos las diferentes variantes que hay de tratar este problema, esto es, de qué formas pueden tratarse las relaciones entre términos.

En primera instancia podemos pensar en un modelo simple, como el ya presentado donde las relaciones entre términos no existan (SBNR), por lo que un término T_i sólo será relevante para una consulta Q si $T_i \in Q$. Esto tiene por qué ser del todo cierto ya que podrían existir documentos que contengan sinónimos de algún término $T_i \in Q$ y no ser relevantes en absoluto.

El primer enfoque (ver [12]) incluye la construcción de un poliárbol como red de términos. La principal ventaja es la relativa poca complejidad de la propagación en dicho tipo de red (orden polinómico), aunque el coste de construcción es algo elevado. El algoritmo de aprendizaje del poliárbol consta de los tres siguientes pasos:

¹³ $idf_i = \log \frac{N}{n_i}$ donde N es el número de términos de la colección, y n_i el número de documentos que contienen a T_i .

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

1. Cálculo de la *dependencia* entre cada dos pares de nodos:

$$\text{Dep}(T_i, T_j | \emptyset) = \sum_{\mathbf{T}_i, \mathbf{T}_j} p(\mathbf{T}_i, \mathbf{T}_j) \log\left(\frac{p(\mathbf{T}_i, \mathbf{T}_j)}{p(\mathbf{T}_i)p(\mathbf{T}_j)}\right)$$

donde $\mathbf{T}_i$ representa uno de los posibles valores de la variable T_i , y las probabilidades se estiman por medio de frecuencias. Este método se basa en la entropía cruzada de Kullback-Leibler.

2. Construcción del *esqueleto* del poliárbol, por medio del algoritmo de Prim (consiguiendo el árbol generador minimal del grafo obtenido en el paso 1).
3. Selección de la *orientación* de las aristas del árbol (estudiando como varían las dependencias entre T_i y T_j al instanciar otro término T_k). Para más detalles, consultar [12].

Estimar las distribuciones de los nodos término en el poliárbol implica hacerlo en dos tipos de nodo: nodos raíz y no-raíz. Para los nodos raíz, obviamente, la distribución es similar a la de cualquier término tal y como se comentó en la descripción del modelo simple anteriormente. Para los nodos no raíz tenemos que expresar la distribución de probabilidad en función de los distintos valores de los padres. Dado un conjunto de términos $\mathcal{T} = \{T_1, \dots, T_m\}$ definimos una *configuración* como un vector $\langle \mathbf{t}_1, \dots, \mathbf{t}_m \rangle$ donde $\mathbf{t}_i$ es una valor posible de la variable T_i . Dada una configuración C , definimos $n(C)$ como el número de documentos que contienen el conjunto de términos que son relevantes en la configuración y no contienen los que no son relevantes. De esta forma, se propone en [12] un estimador, basado en la medida de similitud de Jaccard¹⁴, para la distribución de la probabilidad de un nodo no-raíz con la siguiente forma¹⁵:

$$p(\bar{t}_i | Pa(T_i)) = \frac{n(\langle \bar{t}_i, Pa(T_i) \rangle)}{n(\langle \bar{t}_i \rangle) + n(\langle Pa(T_i) \rangle) - n(\langle \bar{t}_i, Pa(T_i) \rangle)}$$

Nótese que ahora el proceso de inferencia comprendería, como paso previo, una propagación en el poliárbol de términos para obtener las distintas $p(t_i)$.

El sistema propuesto lleva consigo dos desventajas en su aplicación sobre colecciones de gran tamaño:

- El tiempo necesario para construir la subred de términos (el poliárbol) puede ser muy grande aun cuando sólo necesitamos realizar este proceso una vez.

¹⁴Esta medida ([35]), para dos conjuntos X e Y , mide la similitud entre ambos como el cociente del cardinal de la intersección entre el cardinal de la unión.

¹⁵Nótese que $p(t_i | Pa(T_i)) = 1 - p(\bar{t}_i | Pa(T_i))$

- La propagación de evidencia, dada la consulta, a pesar de poderse realizar en tiempo polinómico (algoritmo de Pearl), puede suponer bastante esfuerzo computacional en colecciones con gran número de términos.

Por este motivo, en [13] se propone una alternativa a la construcción del poliárbol completo consistente en la partición del conjunto de términos $\mathcal{T}$ en $\mathcal{T}_b$ (*bad terms*, términos malos) y $\mathcal{T}_g$ (*good terms*, términos buenos). El poliárbol se realizará tan sólo entre los términos de $\mathcal{T}_g$, mientras que los de $\mathcal{T}_b$ quedarán enlazados a los documentos de la misma forma que en el modelo SBNR. El algoritmo utilizado para separar los términos está basado en el conocido k -medias, y puede encontrarse completamente detallado en [13]. En este caso, sólo mostramos el proceso de forma esquemática el proceso, y la red resultante (figura 2.8).

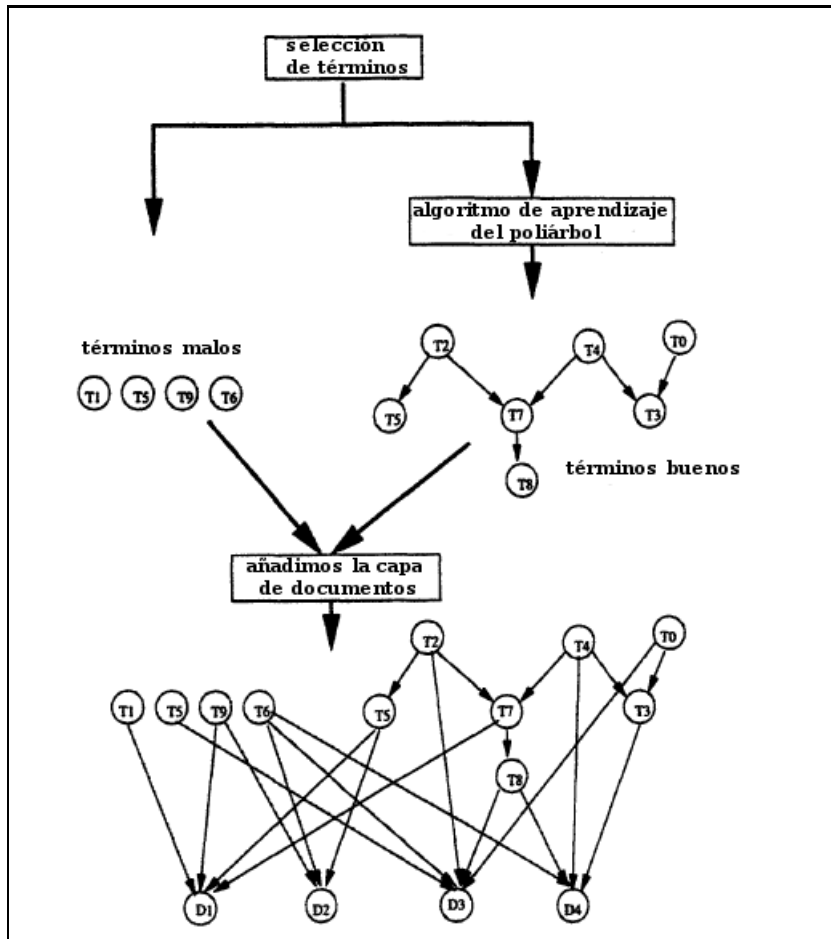


Figura 2.8: Agrupamiento de términos en el modelo BNR.

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

El enfoque más sencillo de tratar (y quizás el definitivo de entre los que se proponen) se muestra en [14] y da un giro importante a los planteamientos anteriores. El método consiste en duplicar la capa de términos, obteniendo un *grafo bipartido* como subred de términos (ver figura 2.9). El conjunto de nodos de la red bayesiana, por tanto, será el conjunto de variables $T \cup T' \cup D$, y los arcos que conectan los nodos de la capa T' y T irán de cada nodo $T'_i \in R_p(T_j)$ hasta T_j , donde la notación $R_p(T_j)$ hace referencia a los términos altamente relacionados con T_j .

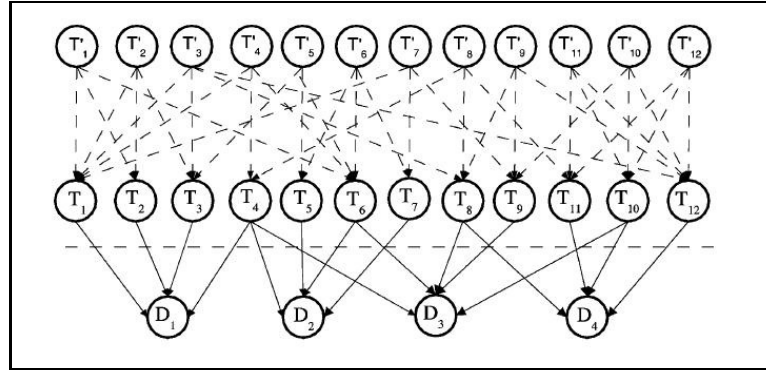


Figura 2.9: BNRM con dos capas de términos.

En este caso, el enfoque seguido para medir las relaciones entre términos es diferente, basado directamente en la frecuencia de coocurrencias de dos términos. La justificación de la forma del estimador puede consultarse en [14], mientras que nosotros daremos sólo el resultado final: la dependencia (fuerza) entre dos términos se mide como

$$\text{strength}'(T_j, T_i) = \begin{cases} 0, & \text{si } n_{t_i t_j} = 0 \\ \text{strength}(T_j, T_i), & \text{en otro caso} \end{cases}$$

$$\text{strength}(T_j, T_i) = \frac{n_{t_i t_j} + 1}{n_{t_i} + 2}$$

donde n_{t_i} es el número de documentos de la colección donde aparece t_i , y $n_{t_i t_j}$ es el número de documentos donde aparecen a la vez ambos términos.

Este enfoque realizaría un proceso de inferencia sencillo: para una consulta Q , se instanciarían los términos $T_j \in Q$ (relevantes) en la capa T' obteniéndose $p(t'_j|Q)$. Tras esto, se obtendrían las probabilidades $p(t_j|Pa(T_j))$ siguiendo un esquema similar¹⁶ al anteriormente mencionado basado en modelos canónicos. Una vez obtenidos estos, el proceso continúa de igual manera que en el modelo SBNR.

Dos detalles a tener en cuenta sobre este último enfoque:

¹⁶Para mayor detalle sobre una propuesta de los valores de los pesos puede consultarse [14].

- Los resultados proporcionados son realmente buenos, siendo competitivo con el primer enfoque basado en poliárboles.
- Tanto el proceso de aprendizaje como el de propagación (aunque sobre todo el primero) son más eficientes que en el caso del poliárbol.

Subred de documentos

En el punto previo, se trataba de modelar relaciones directas entre términos. Usando dichas relaciones, la instanciación de los términos de la consulta incrementaría la probabilidad de relevancia de otros términos, que en definitiva, incrementarían la probabilidad de relevancia de algunos documentos que los contenían. En este caso, el enfoque es parecido: incluimos en el modelo relaciones directas entre documentos, teniendo éstas un papel similar a las técnicas de clustering utilizadas en otros modelos de recuperación de información ([35]).

Los enfoques para mantener una subred de documentos no son tan variados como en el caso de la subred de términos. Esto en gran parte, es así porque las relaciones entre documentos, o son explícitas, o realmente no existen.

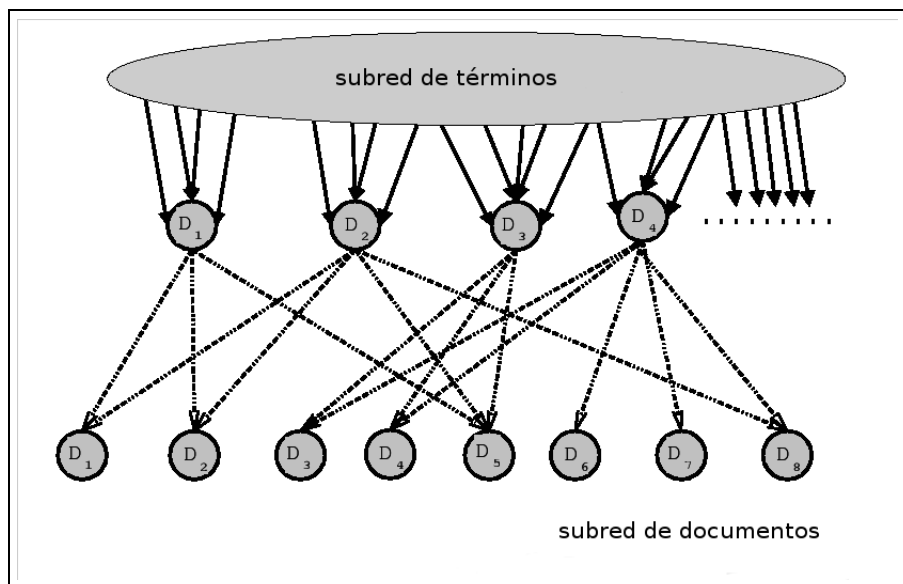


Figura 2.10: BNRM con dos capas de documentos.

En ausencia de información sobre relaciones directas y obvias entre documentos (que será el caso típico), las relaciones en este modelo se basarán en medir similitudes (de forma asimétrica) entre documentos, estimando la probabilidad condicional de relevancia de un documento, respecto a otro que suponemos relevante.

CAPÍTULO 2. REDES BAYESIANAS Y MODELOS PARA RECUPERACIÓN DE INFORMACIÓN NO ESTRUCTURADA

Estas probabilidades se calcularán usando el modelo sencillo de dos capas descrito anteriormente. Así, dado un documento D_j , si calculamos las probabilidades $p(d_j|d_i), \forall D_i \in \mathcal{D}$, entonces, aquellos documentos que den mayor valor de éstas son aquellos más relacionados con D_j (en el sentido en el que D_j tiene una alta probabilidad de ser relevante cuando conocemos que D_i es relevante para una consulta concreta). Un desarrollo más teórico (y justificado) de la elección de ésta medida de similitud entre los documentos se presenta en [1].

Sea $R_c(D_j)$ el conjunto de los c documentos más relacionados con D_j ¹⁷. Estas relaciones se presentarán en la subred de documentos como arcos yendo de los documentos $D_i \in R_c(D_j)$ al documento D'_j en la capa duplicada de documentos (véase figura 2.10 para más detalles). Obtenidos, por tanto, estos documentos más relevantes, queda definida la parte estructural (los enlaces) entre las dos capas de documentos.

Para definir las probabilidades $P(d'_j|Pa(D'_j))$ se utiliza también en este caso una función de probabilidad de la siguiente forma:

$$P(d'_j|Pa(D'_j)) = \frac{1}{S_j} \sum_{D_i \in Pa(D'_j)} p(d_i|d_j)$$

Donde $S_j = \sum_{D_k \in Pa(D'_j)} p(d_j|d_k)$, y los valores de probabilidad condicionada de un documento a otro se obtienen, sin propagación explícita, haciendo uso del modelo anterior:

$$p(d_j|d_i) = \frac{1}{M} \left(\sum_{T_k \in D_j} w_{kj} \right) + \frac{M-1}{M} \left(\frac{\sum_{T_k \in D_j \cap D_i} w_{kj} w_{ki}}{\sum_{T_h \in D_i} w_{hi}} \right)$$

(este último resultado puede obtenerse fácilmente a partir de las ecuaciones del modelo anterior).

Para más detalles sobre esta ampliación del modelo básico y algunas propuestas adicionales véase [1] y [15].

¹⁷Obviamente, D_j siempre será uno de ellos.

Capítulo 3

Recuperación de información estructurada mediante modelos basados en Redes Bayesianas

3.1. Información estructurada: una introducción

La información considerada hasta este momento son unidades indivisibles e independientes llamadas *documentos*. Estas unidades pueden representar obras literarias, artículos científicos, vídeos anotados, ... Reflexionando brevemente sobre el concepto de documento, podemos encontrar múltiples ejemplos en los que, a pesar de poder considerarse éste como una unidad indivisible, resulta más natural tratarlo como un conjunto de partes:

- Un libro podría dividirse en capítulos (si es una obra literaria). En casos de una obra de teatro tendríamos más divisiones: por ejemplo acto, escena, discurso, línea.
- Un artículo científico: que normalmente consta de *abstract*, una serie de secciones (cada una pudiendo dividirse en varias subsecciones y así sucesivamente) y una colección de referencias bibliográficas.
- Las transcripciones de un juicio, donde se suele seguir una estructura que, aunque no es rígida en el orden de las partes, sí lo es en las diferentes partes que la componen.

Este tipo de documentos presenta un panorama distinto al propuesto anteriormente. Si antes existía un conjunto de documentos $\mathcal{D} = \{D_1, \dots, D_m\}$, ahora

dichos documentos se corresponden con un conjunto de *unidades estructurales* $\mathcal{U} = \{U_1, \dots, U_n\}$. De esta forma, desaparece totalmente el concepto de documento.

El concepto de estructura es un concepto flexible. Algunos autores ([17]) llegan incluso a decir que la información estructurada es información *que se ha analizado*. Sin entrar en discusión en este punto, sí que podemos constatar que las divisiones estructurales no son tan claras como las divisiones entre documentos y que éstas pueden situarse a diferentes niveles¹ sin que ninguna de las elecciones sea la definitiva. Incluso, al nivel de más alta generalidad, podríamos considerar un documento no estructurado como uno estructurado, que consta de una sola unidad².

Representar gráficamente un documento estructurado puede ser de gran ayuda para entender de qué estamos hablando. Así, por ejemplo, una representación apropiada podría ser la que utiliza un árbol, donde cada nodo es una unidad estructural. En la figura 3.1 puede observarse un ejemplo de esta representación.

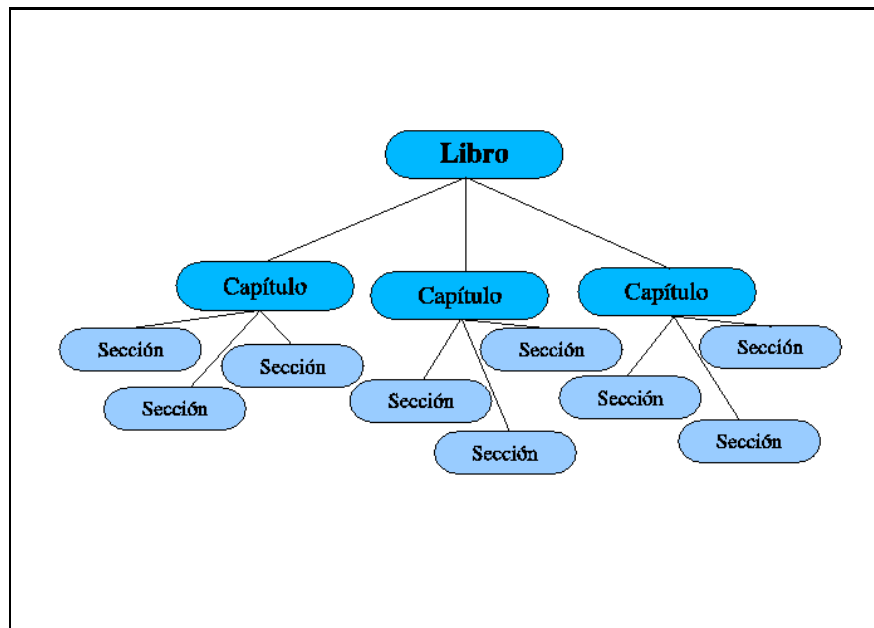


Figura 3.1: Árbol de un documento estructurado.

Se asocia un color distinto a cada nivel del árbol (que se corresponde con un

¹Por ejemplo, en la división de un libro podría considerarse como unidad más pequeña el capítulo (si estamos interesados en comercialarlo en diferentes fragmentos), o la página (si estamos interesados en publicar periódicamente su contenido en un diario).

²Obviamente, esta generalización degenera el concepto de unidad estructural y no proporciona ninguna de las ventajas de los documentos estructurados a los no estructurados, pero se puede considerar así para dar un trato homogéneo a los dos tipos de información.

tipo de unidad estructural) y el árbol tiene siempre la misma profundidad. Realmente, la estructura no sigue un formato tan rígido como en la figura 3.1, y se permite que algunos elementos sean opcionales, el número de hijos varíe, o que no existan siempre el mismo tipo de unidades estructurales a un mismo nivel (ver figura 3.2).

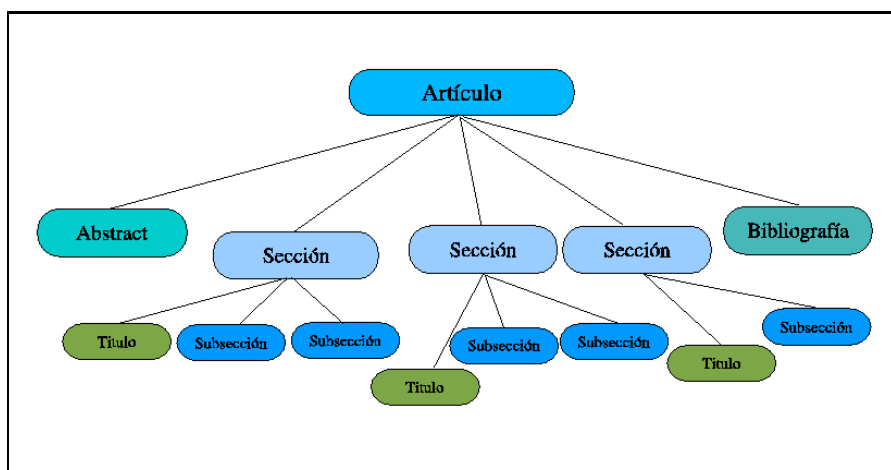


Figura 3.2: Árbol de un documento estructurado (estructura menos rígida).

En este caso, como podemos ver, a iguales niveles, se encuentran diferentes tipos de unidades estructurales, pudiendo variar el número de hijos según el caso, y siendo la profundidad del árbol variable. Esta estructura se corresponde más con la realidad de un documento estructurado que el anterior.

En lo sucesivo, el árbol representando un documento estructurado **se representará al revés** de como lo hemos venido haciendo hasta ahora. Este tipo de representación será más apropiada para nuestro trabajo posterior y no afecta a lo dicho hasta ahora.

Esta introducción, además de justificar la importancia de este tipo de información (presente por doquier), dar una serie de ejemplos de estructuración, y mostrar formas de representación gráfica, termina dando paso a los dos siguientes puntos:

1. *Recuperación de documentos estructurados*: un vistazo sobre qué tiene de peculiar el proceso de RI sobre estos documentos, y qué técnicas específicas pueden usarse.
2. *Lenguajes para documentos estructurados*: ¿qué estándares existen en la actualidad? ¿Qué limitaciones o ventajas proporcionan? ¿Cuál es el estado actual de este tipo de tecnologías?

3.2. Recuperación de documentos estructurados: introducción y problemas

El proceso clásico de RI no sirve totalmente para trabajar con documentos estructurados. En [6] se citan problemas relacionados con el proceso de interacción con el usuario (el fin último de este tipo de sistemas y por tanto un punto crítico), algunos de los cuales comentamos aquí:

1. El impacto en el proceso de *consulta*. Durante el proceso de RI tradicional, el usuario obtenía, como respuesta a su consulta, un conjunto de documentos como una lista de ítems separados. Al introducir documentos estructurados, el sistema puede responder con una lista en la que se incluyan U_i y U_j (dos unidades estructurales), pero que una de ellas esté lógicamente incluida en otra (por ejemplo, si U_j es una parte de U_i). Obviamente no puede ignorarse esta relación entre documentos. Más aún, las funciones clásicas de relevancia ordenarán ambas unidades sin tener en cuenta su relación. Por tanto, un usuario podría encontrarse en primer lugar de la respuesta a una consulta con el primer capítulo de un libro, y posteriormente, en el puesto 15, con una sección del mismo capítulo. Esto es, a todas luces inaceptable, provocando
 - *Sobrecarga cognitiva*: existen datos repetidos en distintos lugares.
 - *Desorientación* ¿Cómo puede tener la ser la misma unidad dos valores diferentes de relevancia según en qué entorno sea considerada?.
 - *Ineficiencia en la búsqueda*: el fin último del sistema, responder claramente a las consultas del usuario, no se alcanza. Éste queda empobrecido por el esfuerzo necesario por parte del usuario para interpretar los datos mostrados.
2. El impacto en el proceso de obtención de la *relevancia*. Como ya se ha comentado anteriormente, los modelos clásicos no sirven para obtener los valores de relevancia que necesitaremos en este tipo de sistemas. Dichos modelos, suelen utilizar la frecuencia del término t_i en el documento d_j (tf_{ij}) como base (definida, obviamente, como el número de veces que aparece dicho término en el documento). Esta frecuencia suele estar ponderada por la frecuencia inversa documental *idf* con objeto de obtener un valor (peso) del término en el documento. Ahora bien, ¿cómo afectaría este esquema de pesado a los documentos estructurados? Supongamos las siguientes inclusiones lógicas entre unas unidades estructurales: $U_a \subset \dots U_m \subset U_n$ ³. Supon-

³Notamos como $U_a \subset U_b$ la *inclusión lógica* de U_a en U_b (por ejemplo, que U_b sea un libro completo y U_a uno de sus capítulos).

3.2. RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS: INTRODUCCIÓN Y PROBLEMAS

gamos también que U_n tiene el valor más alto de relevancia para una cierta consulta Q . Supongamos que la única relevancia respecto a Q del resto de unidades es debida a que contienen a U_n . Sin embargo, muy posiblemente se devuelvan todas las unidades como relevantes (de U_n a U_a , seguramente con otros resultados intercalados), de forma que $U_a \dots U_m$ aparecerán, de forma redundante en la lista (y en posiciones relativamente altas) sin aportar información nueva (aprovechando la inclusión en ellas de U_n). Un usuario del sistema, no encontraría en ellas nueva información, pese a estar bien posicionadas. Una solución parcial a este problema puede ser tratar a priori de forma diferente a diferentes unidades estructurales (conocidos los tipos de estructura). Para este fin, puede consultarse [40].

3. Impacto en la *concentración* de información relevante. En la RI tradicional, si un documento era altamente relevante para una consulta, podíamos considerar que el documento contenía en casi su totalidad información interesante para el usuario. En el caso estructurado, toda la información puede estar *concentrada* en una subunidad incluida en la unidad actual devuelta en la consulta. La calidad del proceso de recuperación depende de la habilidad del usuario para realizar *decisiones* y postprocesar los resultados del sistema, con lo que la calidad de desempeño del sistema baja considerablemente.

Aunque es cierto que los tres problemas van relacionados, también lo es su importancia, y el hecho de que sólo se refieran a la interacción con el usuario. Ampliando esta revisión al resto de los aspectos de la RI, surgen nuevas trabas para la implementación de sistemas que trabajen con documentos estructurados:

- Problemas en el proceso de *indexación*. Nos centraremos en la utilización de archivos invertidos, aunque este problema puede extenderse también con el uso de otras estructuras como los archivos signatura. En el caso no estructurado, la información que debía obtenerse era sencilla: para cada término, una lista de documentos en los que aparece $(t_i, d_1, d_2, \dots, d_n)$. En el caso estructurado, un mismo término está contenido en una unidad, y todas las contenedoras de ésta. Si seleccionamos aquella de último nivel (más lejano a la raíz), no tendremos conocimiento automático de que las unidades contenedoras también incluyen a dicho término, a través de la unidad de último nivel. Para más detalles, pueden consultarse [37] y [27].
- Problemas en el proceso de *búsqueda*. Complementando al anterior, dadas dos unidades estructurales U_i y U_j , ¿cómo obtener eficientemente la relación de parentesco? ¿Es U_i contenedora de U_j ? ¿Son unidades pertenecientes a partes totalmente distintas? ¿Cuál es la unidad contenedora común más cercana a los términos? Todas estas preguntas no tienen respuesta a menos

que se utilicen estructuras de datos convenientemente apropiadas. Incluso y si no se tiene especial cuidado, con dichas estructuras, los requisitos sobre las relaciones entre unidades no podrían ser resueltos en tiempos razonables. Algunos enfoques generales sobre búsqueda en documentos estructurados se pueden encontrar en [39].

- Problemas *semánticos* sobre entidades utilizadas. En RI tradicional, el concepto de colección está claro: se trata de un conjunto de documentos $C = \{d_1, \dots, d_n\}$. En el caso estructurado, será por tanto un conjunto de unidades estructurales. El problema puede residir en que la colección al completo sea un único documento estructurado ampliamente ramificado. Por tanto, se confundirían los conceptos de colección y unidad estructural. Otro problema, comentado anteriormente, es la amplia variedad de definiciones del *idf* que podemos utilizar. Recordando que el *idf* se define como $\log \frac{N}{n_i}$. ¿Cómo definir N y n_i para el caso estructurado? Pueden tomarse distintos enfoques ($N = |\mathcal{U}|$ ó $N = |\text{unidades que sólo contienen texto}|$, $n_i = |\text{todas las unidades que contienen a } t_i|$, contando contenedoras, o sin hacerlo) aunque ninguno parece estar libre de problemas.
- Problemas en la forma de *consulta*. Más que un problema, es una ventaja ofrecida por los documentos estructurados: la posibilidad de incluir diferentes tipos de consultas. Mientras que en los sistemas clásicos, una consulta solía ser representada como una expresión booleana (generalmente conjunción) de términos (p.ej.: $t_i \wedge \dots \wedge t_k$), un documento estructurado permite refinar la consulta incluyendo precisamente algún tipo de información sobre la estructura. Así por ejemplo, si estamos consultando en una colección de artículos científicos, podríamos restringir la búsqueda de resultados de una consulta Q a una cierta unidad estructural (por ejemplo, el *abstract* de cada artículo). De esta forma, los tipos de consulta apropiados para documentos estructurados son los siguientes:
 - Consultas de tipo “sólo contenido” (CO, *content only*). Equivalentes a las consultas clásicas de los sistemas de RI, se construyen sólo con términos.
 - Consultas de tipo “contenido y estructura” (CAS, *content-and-structure*, donde la lista de documentos relevantes se dirige a un conjunto concreto de tipos de unidades estructurales. Si esta restricción es obligatoria, hablaríamos de consultas de tipo SCAS, mientras que si la restricción es vaga⁴, hablaremos de consultas de tipo VCAS.

⁴En el sentido de imprecisión

3.2. RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS: INTRODUCCIÓN Y PROBLEMAS

El problema de añadir consultas de tipo CAS es la necesidad de un lenguaje específico de consulta. Un modelo para obtener un lenguaje de consulta por contenido y estructura puede encontrarse en [28].

Otra propuesta de lenguaje de consulta, ampliamente utilizado, es el *Narrowed Extended XPath I* (NEXI), detallado en [41].

Por supuesto hay más problemas al tratar con documentos estructurados, aunque hemos tratado los más importantes. En el desarrollo de nuestro propio sistema, tendremos que tenerlos en cuenta, y proponer soluciones apropiadas.

3.3. Lenguajes para información estructurada

3.3.1. Introducción. XML como herramienta para información estructurada

Tras ser presentados los problemas de intentar usar RI clásica con documentos estructurados, estudiamos ahora las diferentes alternativas posibles para la definición y especificación de documentos estructurados. Dichas formas pasan por la utilización de lenguajes específicos para la definición de documentos estructurados.

No es ningún secreto que el lenguaje más utilizado, a día de hoy, para este tipo de tareas, es el *Extensible Markup Language* o XML⁵ ([44]).

XML fue creado al amparo del Word Wide Web Consortium (W3C) organismo que vela por el desarrollo de WWW partiendo de las amplias especificaciones de SGML. Su desarrollo se comenzó en 1996 y la primera versión salió a la luz el 10 de febrero de 1998.

Es un formato basado en texto con una sintaxis muy simple. Fue originalmente diseñado para resolver los problemas de la publicación electrónica a gran escala y tiene una gran importancia en el *intercambio de información* de un amplio tipo de datos en Internet y en otras aplicaciones.

En este apartado no entraremos a fondo en la descripción de XML, aunque sí trataremos de justificar tanto su uso en documentos estructurados como su simplicidad. Para una referencia más profunda sobre la sintaxis, APIs que permiten su manipulación y las posibles aplicaciones pueden consultarse [44] (para la especificación) y [33] o [23] para conocer sus aplicaciones y encontrar numerosos ejemplos.

Además de poder representar tanto datos estructurados como semi-estructrados, XML tiene un número de características que lo han hecho ser ampliamente utilizado como formato de representación de datos. XML es *extensible, independiente de la plataforma utilizada*, y soporta completamente *internacionalización*. El hecho de que XML sea un formato basado en texto hace que cualquiera pueda leer y editar documentos XML usando tan solo simples editores de texto.

La *extensibilidad* de XML se manifiesta de varias formas. Primeramente, a diferencia de HTML, XML no tiene un conjunto de tags fijo, esto es, las palabras clave del lenguaje no están definidas por un principio. En este lugar, uno mismo puede definir usando XML un *lenguaje* específico para aplicaciones concretas. También, las aplicaciones que procesan XML son resistentes al cambio en el sen-

⁵XML no debe confundirse con HTML. Este último es un lenguaje, también propuesto por el W3C, orientado a la definición de documentos web. Actualmente existe una evolución de HTML que implementa una sintaxis parecida a la de XML llamado XHTML, cuyo objetivo es acercar los documentos del web a los de tipo estructurado, y finalmente, sustituir al HTML.

tido de que dichos cambios son siempre aditivos. Por ejemplo, una aplicación que dependa del procesamiento de elementos de tipo <Cliente> con un subelemento <id-cliente> seguiría funcionando correctamente si añadimos otro atributo como <fecha_compra>. Esta flexibilidad no es común en otros lenguajes de programación y es un beneficio derivado de usar XML.

Por último, XML es *independiente de la plataforma* utilizada, del sistema operativo, o del fabricante de software. De hecho, es bastante posible producir (o procesar) XML utilizando una amplia variedad de productos y lenguajes de programación. Esta independencia hacen a XML ideal como un medio de alcanzar *interoperabilidad* entre plataformas diferentes de programación y sistemas operativos distintos.

3.3.2. Breve descripción de la sintaxis de XML

La sintaxis de XML es, como vamos a comprobar, extremadamente sencilla. Esta parte no es especialmente exhaustiva, sino que contiene todo lo necesario para que el lector pueda asumir la nomenclatura y las partes más utilizadas del lenguaje.

Inicialmente, todo documento XML se escribe en un documento de texto plano, incluyendo la cabecera siguiente:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
```

El 1.0 hace referencia a la versión utilizada de XML. Si se usa otra posterior, debe especificarse. El encoding hace referencia a la codificación de caracteres, y puede omitirse, tomándose como valor por defecto la codificación UTF-8.

En el resto del documento se deben escribir unidades estructurales encerradas entre **etiquetas**. Dichas etiquetas son del tipo <etiqueta> si son de inicio de una unidad, o </etiqueta> si son de fin⁶.

Las etiquetas tendrán un nombre (sensible a mayúsculas) que hará referencia al tipo de unidad estructural considerado. Dichos nombres, junto con las reglas de inclusión de etiquetas, pueden especificarse en un documento especial, con sintaxis diferente llamado DTD. Por otra parte, cualquier etiqueta puede tener atributos, siguiendo este formato (entrecomillados):

```
<etiqueta atributo1="valor1" atributo2="valor2"...>
```

Las únicas restricciones son las siguientes:

⁶XML también permite la inclusión de etiquetas representando unidades de información vacías, cuya sintaxis es <etiqueta />.

- Toda etiqueta abierta, `<etiqueta>` debe cerrarse posteriormente con su complementaria `</etiqueta>`.
- Los elementos XML *no pueden solaparse*. Si una unidad contiene a otra, los anidamientos de etiquetas deben ser de la siguiente forma:

`<etiqueta1>...<etiqueta2>...</etiqueta2>...</etiqueta1>`

No permitiéndose cosas como la siguiente:

`<etiqueta1>...<etiqueta2>...</etiqueta1>...</etiqueta2>`

- Por último, cada documento XML debe estar contenido dentro de una sola etiqueta, llamada *elemento raíz*.

Si un documento sigue estas tres indicaciones, se dice que *está bien formado* ó que es *XML bien formado*. Decimos que un documento XML es *válido* si es XML bien formado y además sigue las reglas especificadas en su propio DTD. En cualquier otro caso, diremos que el documento es *no válido*.

Por último, comentaremos brevemente la forma de definir la sintaxis de los distintos tipos de etiquetas utilizada en los DTD⁷, mediante un ejemplo (extraído de [38]):

```
<!-- direccion.dtd -->
<!ELEMENT direccion (nombre, calle, ciudad, estado, codigo-postal)>
<!ELEMENT nombre (titulo? nombre, apellidos)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT nombre (#PCDATA)>
<!ELEMENT apellidos (#PCDATA)>
<!ELEMENT calle (#PCDATA)>
<!ELEMENT ciudad (#PCDATA)>
<!ELEMENT estado (#PCDATA)>
<!ELEMENT codigo-postal (#PCDATA)>
```

Este DTD define todos los elementos usados en el documento de ejemplo. Define tres cosas básicas:

⁷Realmente, un DTD no es la única forma de definir la sintaxis particular de nuestras etiquetas, sino que existe una alternativa llamada XML Schema. Nosotros utilizaremos DTD por su antigüedad (fue desarrollado con la primera especificación de XML), y por su simplicidad, aun conscientes de algunas limitaciones del método, que sí son cubiertas por XML Schema.

- Un elemento <direccion> que contiene un <nombre>, un <calle>, un <ciudad>, un <estado>, y un <codigo-postal>. Todos estos elementos deben aparecer y deben hacerlo en ese mismo orden.
- Un elemento <nombre> contiene un elemento <titulo> opcional (la marca de interrogación indica que <titulo> es opcional), seguido de un elemento <nombre> y de un elemento <apellidos>.

Todos los demás elementos contienen texto (#PCDATA indica datos de carácter y que no pueden incluirse otros elementos dentro de éstos). Aunque son muy simples, los DTD aclaran qué combinaciones de elementos son legales. Una documento de dirección postal que presente un elemento <codigo-postal> antes de un elemento <estado> no es legal, y ningún documento que no presente el elemento <apellidos> será por tanto XML válido.

Por último, añadir que la sintaxis DTD es diferente de la sintaxis XML ordinaria⁸.

Una referencia más profunda sobre la sintaxis de los documentos DTD puede encontrarse en [33] o en [38].

⁸Los documentos de XML Schema, por el contrario, son XML ellos mismos, lo cuál tiene algunas consecuencias interesantes

3.4. Redes Bayesianas para documentos estructurados: el modelo BNR-SD

3.4.1. Definición del modelo

Presentado en [9], se trata de una extensión del modelo BNR comentado en la sección anterior, para el caso de documentos estructurados.

La idea inicial parte del aprovechamiento de la estructura en árbol del documento estructurado como parte gráfica de la red bayesiana. Para esto, representaremos el árbol de forma invertida con objeto de tener una disposición de nodos similar a la del modelo BNR⁹.

Los nodos de la red serán por una parte el conjunto de términos $\mathcal{T} = \{T_1, \dots, T_l\}$ y por otra parte, el conjunto de unidades estructurales $\mathcal{U}$. Para simplificar el modelo, consideraremos dos tipos de unidades estructurales: básicas (aquellas que sólo contienen términos) $\mathcal{U}_b = \{B_1, \dots, B_m\}$ y complejas (aquellas que contienen unidades básicas y complejas) $\mathcal{U}_c = \{S_1, \dots, S_n\}$. Por tanto, el conjunto de nodos correspondiente a las unidades estructurales será $\mathcal{U} = \mathcal{U}_b \cup \mathcal{U}_c$. U_l notará un elemento cualquiera de $\mathcal{U}$.

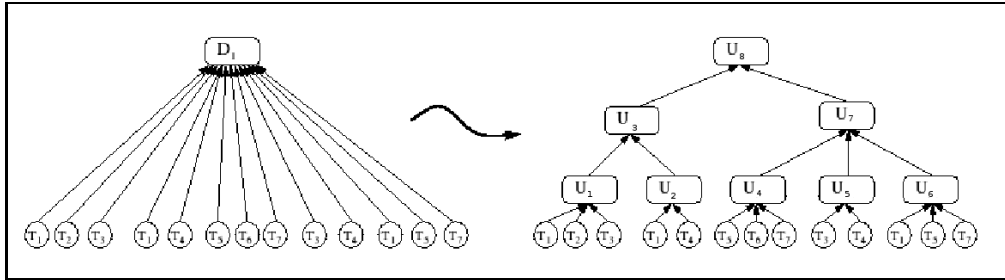


Figura 3.3: Construyendo la red bayesiana para los documentos estructurados.

Cada nodo T_k , B_i o S_j tiene asociada una variable aleatoria binaria (el ítem asociado es o no es relevante) cuyos valores notamos $\{t_k^-, t_k^+\}$, $\{b_i^-, b_i^+\}$ o $\{s_j^-, s_j^+\}$.

Faltarían para completar el modelo, la definición de las distribuciones de probabilidad condicionada $p(t_k^+)$, $p(b_i^+ | pa(B_i))$ y $p(s_j^+ | pa(S_j))$, para cada nodo en $\mathcal{T}$, $\mathcal{U}_b$ y $\mathcal{U}_c$, respectivamente, y todas las configuraciones posibles del conjunto de sus padres¹⁰.

La definición de las probabilidades para el caso de los términos es similar a la definición dada en el modelo BNR. Para el caso de las unidades básicas, utiliza-

⁹De ahí la razón de hacerlo así, como ya comentamos anteriormente

¹⁰Para un nodo X , $Pa(X)$ será el conjunto de sus padres, mientras que $pa(X)$ notará una configuración de valores concretos de $Pa(X)$.

3.4. REDES BAYESIANAS PARA DOCUMENTOS ESTRUCTURADOS: EL MODELO BNR-SD

mos un modelo canónico similar al propuesto en el BNR, cambiando documentos por unidades simples. Para el caso de unidades complejas, el mecanismo también es parecido, aunque considerando que una unidad “está indexada” por otras. Las fórmulas son las siguientes:

$$\forall B_i \in \mathcal{U}_b, p(b_i|pa(B_i)) = \sum_{T_k \in R(pa(B_i))} w(T_k, B_i)$$

$$\forall S_j \in \mathcal{U}_c, p(s_j|pa(S_j)) = \sum_{U_l \in R(pa(U_l))} w(U_l, S_j)$$

Los pesos verifican $w(T_k, B_i) \geq 0$, $w(U_l, S_j) \geq 0$, $\sum_{T_k \in Pa(B_i)} w(T_k, B_i) \leq 1$ y $\sum_{U_l \in Pa(S_j)} w(U_l, S_j) \leq 1$. Además, $R(pa(U))$ es el subconjunto de padres de U que son relevantes en la configuración $pa(U)$.

Una definición posible de los pesos de los términos en las unidades básicas es la siguiente:

$$\forall B_i \in \mathcal{U}_b, T_k \in Pa(B_i), w(T_k, B_i) = \frac{\rho(T_k, A(B_i))}{\sum_{T_h \in A(B_i)} \rho(T_h, A(B_i))}$$

donde $A(U_l)$ es el conjunto de términos contenidos en U_l (sin depender si es de tipo básico o complejo), $\rho(T_k, C)$ es $tf_{k,C} \cdot idf_k$, y $tf_{k,C}$ denota la frecuencia de T_k en el conjunto de términos C , e idf_k es la frecuencia documental inversa de T_k en la colección. Es fácilmente demostrable que este peso cumple las dos propiedades requeridas más arriba.

Alternativamente, para las unidades complejas, definimos los pesos:

$$\forall S_j \in \mathcal{U}_c, U_l \in Pa(S_j), w(U_l, S_j) = \frac{\sum_{T_k \in U_l} \rho(T_k, A(U_l))}{\sum_{T_h \in A(S_j)} \rho(T_h, A(S_j))}$$

3.4.2. Inferencia

Dada una consulta Q , El método de inferencia obtendrá las probabilidades a posteriori $p(u_l^+|Q)$, ordenando las unidades por dicho valor.

$$\forall B_i \in \mathcal{U}_b, p(b_i^+|Q) = \sum_{T_k \in Pa(B_i) \setminus Q} w(T_k, B_i) p(t_k^+) + \sum_{T_k \in Pa(B_i) \cap R(Q)} w(T_k, B_i)$$

$$\forall S_j \in \mathcal{U}_c, p(s_j^+|Q) = \sum_{U_l \in Pa(S_j)} w(U_l, S_j) p(u_l^+|Q)$$

Recordamos en este caso, que el término $p(t_k^+)$, esto es, la probabilidad a priori de relevancia términos (en este caso, que no pertenecen a la consulta) se

considera igual para todos, con un valor de $\frac{1}{M}$, de modo que puede sacarse fuera del sumatorio.

El enfoque seguido en este caso, evalúa la probabilidad de relevancia de una unidad en función de dicho valor, para sus padres. Para las unidades básicas, esto es inevitable (sus padres son los términos), pero para las unidades complejas existe una alternativa más directa. Utilizaremos la siguiente notación: $\forall S \in \mathcal{U}_c$, sea $A_b(S) = \{B_j \in \mathcal{U}_b | B_j \text{ es un ascendiente de } S\}$, $A_c(S) = \{S' \in \mathcal{U}_c | S' \text{ es un ascendiente de } S\}$, y $\forall B \in \mathcal{U}_b$, sea $D_c(B) = \{S \in \mathcal{U}_c | S \text{ es un descendiente de } B\}$. Sea $Hi(X)$ el único¹¹ hijo de un nodo X . Nótese que, para cada unidad básica B en $A_b(S)$, hay un sólo camino de B a S . Definamos el peso $w(B, S)$ como el producto de los arcos del camino de B a S . Así,

$$w(B, S) = w(B, Hi(B)) \prod_{S' \in A_c(S) \cap D_c(B)} w(S', Hi(S'))$$

Con esta definición, podemos reescribir las probabilidades a posteriori de las unidades complejas como sigue:

$$\forall S_i \in \mathcal{U}_c, p(s_i^+ | \mathcal{Q}) = \sum_{B_j \in A_b(S)} w(B_j, S_i) p(b^+ | \mathcal{Q})$$

¹¹En el caso de un nodo al final de la red, definimos $Hi(X) = \emptyset$

Capítulo 4

Desarrollo de un Sistema para Recuperación de Documentos Estructurados

4.1. Introducción

En un principio hemos encuadrado el concepto de la recuperación de información en un contexto y revisados los modelos clásicos y otros más avanzados basados en redes bayesianas. Tras esto, hemos dado el salto a la información estructurada, con su correspondiente problemática, y se ha justificado completamente el uso del lenguaje XML para representar la información.

En este punto, nos disponemos a construir un sistema de recuperación de información genérico (que trabaje con documentos XML), con objeto de que sea aplicado al caso concreto de las sesiones plenarias del Parlamento de Andalucía.

Inicialmente, necesitamos unos requisitos básicos que debe cumplir el sistema. Para ello, la experiencia del Prof. Dr. Juan M. Fernández-Luna con este tipo de sistemas ha sido de gran ayuda puesto que ha sido él quien ha confeccionado los requisitos del sistema genérico, como si de un cliente se tratara.

Un sistema que sea capaz de proporcionar la funcionalidad (genérica) que aquí se lista, estaría listo para trabajar con cualquier colección de documentos XML (sin restricciones de tamaño), por lo que podría implementarse cualquier modelo de recuperación (en especial nuestro modelo BNR-SD, u otros más avanzados que pudieran surgir en el futuro).

4.2. Estudio de Requisitos

4.2.1. Requisitos genéricos

- Almacenar una colección XML, intentando ahorrar espacio en disco y con estructuras de datos que permitan operaciones eficientes sobre la información almacenada.
- Representar documentos y sus componentes, y los términos que aparecen en la colección.
- Almacenamiento, bajo un mismo “nombre” de una colección heterogénea, es decir, diferentes colecciones con DTDs y/o XML Schemas distintos, manteniendo los requisitos anteriores de espacio y tiempo.
- Acceso eficiente a todos los elementos representados en la colección.
- Permitir la formulación de consultas por contenido (CO) y por contenido y estructura (CAS).
- Facilidad en la actualización de los diferentes elementos representados:
 - Añadir nuevos documentos o eliminación de ya existentes.
 - Actualización de la información representada e inclusión de información adicional.

4.2.2. Requisitos detallados sobre la información a almacenar

Información relevante

En una colección de documentos estructurados marcada en XML nos encontraremos los siguientes elementos relevantes para nuestros intereses, que deberán ser tenidos en cuenta en el desarrollo del sistema de recuperación de información estructurada¹:

- Colección: conjunto de documentos XML que siguen uno o varios DTDs. Dentro de la colección podemos considerar los siguientes actores:
 - Tipos de etiquetas o marcas incluidas en los DTDs de la colección.
 - Componentes, unidades o elementos (se corresponden con las marcas o etiquetas encontrados en los documentos).

¹En adelante, SRIE

- Términos: cadenas de caracteres asociados a algunos componentes.
- Relaciones:
 - Relaciones entre unidades estructurales, reflejando composición (una unidad está compuesta por otras).
 - Relaciones entre unidades estructurales (hiperenlaces).
 - Relaciones entre unidades estructurales (similitud).
 - Relaciones entre términos y unidades (aparición de los primeros en los segundos).
 - Relaciones entre términos.
 - Relaciones entre tipos de etiquetas y unidades estructurales de ese tipo.
- Otros elementos relevantes para la construcción y uso de los modelos de redes bayesianas y diagramas de influencia (pesos elaborados, probabilidades, niveles, ...).

Así, podemos dividir todos estos elementos en dos grandes áreas, con los contenidos correspondientes:

■ **Área Estructural:**

- Información sobre la colección en sí.
- Información sobre los tipos de etiquetas.
- Información sobre las unidades estructurales.
- Relaciones de composición de unidades estructurales.
- Relaciones (hiperenlaces) de unidades estructurales.

■ **Área Textual:**

- Información sobre los términos.
- Relaciones de ocurrencia de términos en unidades.

Un tercer grupo podría ser el que relacionara todo lo específico del modelo de recuperación:

■ **Área específica de recuperación:**

- Probabilidades a priori de los términos.
- Probabilidades a priori de las unidades.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- Probabilidades a posteriori de unidades.
- Valores de utilidad.
- Utilidades esperadas de las unidades.
- Pesos de unidades.
- Pesos de términos.
- Relaciones entre unidades estructurales (similitudes).
- Relaciones entre términos.

A continuación, veremos más detalladamente la información que se deberá almacenar en cada área.

Área Estructural

- Colección.
 - Identificador único de la colección.
 - Descripción de la colección, o de las subcolecciones.
 - DTD ó lista de ellos.
 - Número de documentos que la componen.
 - Número de unidades que componen los documentos.
 - Número de términos.
 - Información sobre cómo se ha hecho la indexación. Por ejemplo, si ha habido eliminación de palabras vacías, o si se han obtenido las raíces,...
- Tipos de etiquetas.
 - Identificador.
 - Cadena de caracteres con la etiqueta.
 - Grado de importancia de la etiqueta en el documento desde el punto de vista de la recuperación.
 - Tipo de etiqueta: contenedor de términos, contenedor de otras unidades, contenedor imaginario (y por tanto, de términos).
 - Relación de unidades estructurales que son de un tipo dado.
 - Si es recuperable o no la unidad.

- Unidades estructurales ²
 - Identificador.
 - Ruta XPath.
 - Tipo de etiqueta.
 - Posición en el contenedor (si un contenedor tiene n unidades, qué número ocupa entre ellas ocupa la concreta).
 - Si es contenedor de términos, cuántos tiene; si lo es de unidades, cuántas contiene.
 - Niveles de unidades estructurales.
 - Los atributos de una unidad se consideran un tipo especial de etiqueta no recuperable.
- Relaciones de composición de unidades estructurales.
 - Para cada unidad que contenga otras unidades, cuáles son las que contiene, y el número de éstas.
 - Para cada unidad, quién es la unidad que los contiene.
- Relaciones (hiperenlaces) de unidades estructurales.
 - Para cada unidad que contenga enlaces, quiénes son las unidades enlazadas.
 - Para cada unidad, de dónde vienen los enlaces hacia ella.

Área Textual

- Términos.
 - Identificador.
 - Cadena de caracteres.
 - Relación de pesos genéricos (por ejemplo, idf en la colección).
 - Número de unidades donde aparece.
 - Número de apariciones totales.

²Se asume que una unidad conteniendo a la vez texto y otras unidades estructurales, es duplicada, de tal forma que la unidad originaria contiene las unidades estructurales más una virtual, donde se asignan los términos. Así, esta unidad pasaría a ser sólo unidad contenedora de unidades. La nueva unidad virtual no sería recuperable y sólo es un contenedor auxiliar.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- Relaciones de ocurrencia de términos en unidades.
 - Para cada término, y un tipo de etiqueta, qué unidades lo contienen, junto con su frecuencia de aparición.
 - Para cada unidad, qué términos, junto con la frecuencia de aparición, contiene.
 - Para cada unidad, y cada término, posición de aparición del término en la unidad (Comentario de implementación: en una estructura de datos diferente).

4.2.3. Requisitos detallados sobre las operaciones

En cuanto a las operaciones con las que deberíamos dotar el sistema, éstas podrían ser las siguientes:

- En el área estructural:
 - Dada una colección, devolver su descripción.
 - Dada una colección, devolver su DTD o lista de ellos.
 - Dada una colección, devolver el número de documentos que la componen.
 - Dada una colección, devolver el número de unidades que contiene.
 - Dada una colección, devolver el número de términos que contiene.
 - Dada una colección, devolver la información referente a la indexación.
 - Dado un tipo de etiqueta mediante su cadena de caracteres, devolver su identificador.
 - Dado un tipo de etiqueta mediante su identificador, devolver su cadena de caracteres.
 - Dado un tipo de etiqueta (id), devolver el grado de importancia.
 - Dado un tipo de etiqueta (id), devolver la relación de unidades (ids) que son de ese tipo.
 - Dada una unidad, identificada con su camino XPath, devolver su identificador.
 - Dada una unidad, identificada por su clave única, devolver su camino XPath.
 - Dada una unidad (id), devolver el tipo de etiqueta al que pertenece.
 - Dada una unidad (id), devolver la colección a la que pertenece.

- Dada una unidad (id), devolver el tipo de etiqueta al que pertenece.
 - Dada una unidad (id), devolver la posición del contenedor donde está incluida.
 - Dada una unidad (id), devolver el número de términos que contiene o unidades, según el tipo de contenedor que sea.
 - Dada una unidad (id), devolver la relación de unidades (ids) que contiene (si es un contenedor de unidades).
 - Dada una unidad (id), devolver la unidad donde está contenida (id), o nulo si es la raíz.
 - Dada una unidad (id) contenida y otra contenedora, devolver el peso de la primera en la segunda
 - Dada una unidad (id), devolver la relación de hermanos
 - Dada una unidad (id), devolver las unidades a las que apunta (hacia las que existe una referencia).
 - Dada una unidad (id), devolver las unidades que la apuntan (desde donde referencian a la primera).
- En el área textual:
- Dado un término (cadena de caracteres), devolver su identificador.
 - Dado un término (identificador), devolver su cadena de caracteres.
 - Dado un término (id), devolver su peso genérico i-ésimo.
 - Dado un término (id), devolver el número de unidades donde aparece
 - Dado un término (id), devolver la relación de las unidades de ocurrencia directa y su frecuencia de aparición.
 - Dado un término (id), devolver la relación de las unidades de ocurrencia indirecta y su frecuencia de aparición.
 - Dado un término (id) y un tipo de etiqueta (id), devolver la relación de las unidades de ese tipo que lo contienen, y su frecuencia de aparición.
 - Dada una unidad (id), devolver la relación de términos que contiene y su frecuencia, de manera directa.
 - Dado un término (id), y una unidad, devolver la relación de términos incluidos en la misma unidad que el original, con su posición en la unidad.
 - Dados dos términos (ids) y una unidad (id), devolver la distancia mínima entre las posiciones de éstos en la unidad.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- Dado un término (id) y una unidad (id), devolver la relación de posiciones.

4.3. Diseño del Sistema: Nivel lógico

4.3.1. Introducción

En este documento se expondrá el proceso seguido en el diseño del sistema propuesto en la especificación anterior. Los pasos que se han seguido son:

- Se han identificado las principales clases del sistema.
- Se ha hecho lo mismo con las relaciones entre clases.
- Se han añadido las operaciones correspondientes a cada clase, procurando tener siempre en mente un *diseño orientado a responsabilidades* (esto está por hacer).

Una introducción rápida a los diagramas de clases y su notación puede encontrarse en [36]. Para un desarrollo más profundo de esta metodología, pueden consultarse [4] y [20].

Debido a los requisitos críticos de eficiencia en nuestro sistema, tanto en tiempo como en espacio, no se ha seguido ningún proceso clásico de diseño de forma estricta ya que un buen diseño, en este caso, podría empobrecer el desempeño del sistema por lo que hemos optado por un compromiso intermedio entre diseño y un sistema utilizable.

4.3.2. Niveles y vistas del Sistema de RI

En esta sección tratamos de enumerar diferentes niveles de abstracción desde los cuáles observaremos el funcionamiento de un sistema de RI genérico. Simplemente se tratan de abstracciones para tratar de ayudar en el proceso de diseño.

Se proponen tres niveles distintos (del más alto al más bajo):

1. *Nivel conceptual*: se considera el sistema como una caja negra (vemos sólo las entradas y salidas).
2. *Nivel de estructuras de datos*: se considera el funcionamiento del sistema a nivel conceptual (*qué* hace, pero *no cómo* lo hace).
3. *Nivel físico*: clases y estructuras a más bajo nivel que implementan las diferentes técnicas de indexación y búsqueda (del tipo de las que implementa el Sistema mg).

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

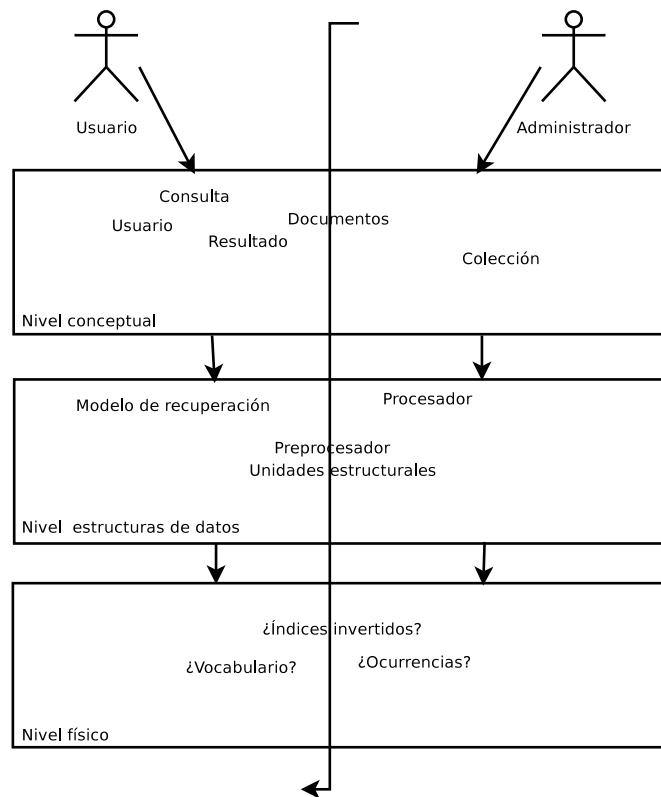


Figura 4.1: Niveles y vistas de un SRI

A partir de cada uno de estos niveles obtendremos un grupo de clases de menor nivel de abstracción cada grupo, según descendamos de nivel.

Además, en cada nivel podemos describir dos vistas del sistema:

1. *Vista de usuario:* usuario normal que accede al sistema para realizar consultas y recibir resultados (accede sin posibilidad de modificar el contenido del sistema).
2. *Vista de administrador:* usuario que accede al sistema pudiendo variar su contenido (accede con la posibilidad de modificar el contenido del sistema), por ejemplo: incluyendo o quitando documentos, obteniendo estadísticas, cambiando modos de funcionamientos, ... Fundamentalmente nos interesará la faceta referida a la inserción/borrado de documentos.

La figura 4.1 puede ayudar a la comprensión de esta división por niveles y vistas.

Al igual que en una especificación de casos de uso (pero sustituyéndola) proponemos un funcionamiento concreto del sistema a los tres niveles (desde dos

perspectivas distintas cada uno) de donde podemos extraer información sobre las clases del sistema.

4.3.3. Nivel conceptual

Vista de usuario

Como se ha comentado anteriormente, este es el uso típico del sistema de RI: un usuario introduce una o más consultas en el sistema y éste le devuelve un resultado (conjunto de documentos -unidades-) ordenados según su relevancia, por cada consulta.

En principio, pueden hacer falta clases para: las consultas, el resultado, el sistema en particular y tal vez los usuarios (si existieran diferentes perfiles de búsqueda, etc). Además, añadimos clases que representaran los elementos que componen cada resultado (documentos/unidades) aunque en este nivel no observaremos relaciones entre documentos y/o sus partes.

Vista de administrador

Este es el uso atípico del sistema de RI. Un administrador introduce documentos en el sistema, añadiéndolos a una colección concreta, con objeto de que queden almacenados y listos para la búsqueda.

En esta ocasión harían falta las siguientes clases: el documento³ (o unidad) y la colección. No tendría demasiado sentido incluir la clase administrador por su falta de contenido a menos que se quiera especificar una serie de políticas de seguridad a este nivel.

4.3.4. Nivel de estructuras de datos

Vista de usuario

En este punto, trataremos de refinar el anterior. Un usuario produce una consulta (de uno de los tipos predefinidos) que es procesada (para eliminar *stopwords*, realizar stemming, case-folding ...) tras lo que se obtiene la “consulta indexada”. Hecho esto, se calcula la relevancia que tiene cada unidad (en las que se descompone cada documento) según un modelo de recuperación. El modelo decide qué

³La discusión sobre si el documento (documento como “archivo XML”) tiene entidad en este tipo de sistemas es algo que surge de forma natural. Como ya se ha comentado en el anterior capítulo, a la hora de tratar documentos estructurados, la colección se puede considerar como otra unidad estructural compuesta de las unidades raíz de cada documento. Con esta visión, ¿qué sentido tiene el documento? En definitiva, es una clase que, como tal, pierde entidad.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

unidades serán devueltas y en qué orden, consultando un índice invertido. Finalmente, se construye un resultado para la consulta con la lista de unidades que devolvió el modelo y se devuelve al usuario que formuló la consulta.

Nuevas clases encontradas: modelo de recuperación, consulta indexada, pre-procesador, stopword, unidades estructurales, índice invertido (y todo lo que refleje las relaciones de las unidades entre sí).

Vista de administrador

Siguiendo la estructura anterior, un administrador puede introducir nuevos documentos (o colecciones completas) en el sistema. Cada documento es preprocesado de forma parecida a las consultas y posteriormente para serles extraídas las diferentes unidades (compuestas por unidades o términos). Cada nuevo término se añade al vocabulario.

Nuevas clases encontradas: procesador de documentos, vocabulario.

4.3.5. Nivel físico

No comentamos este nivel, puesto que de su eficiencia depende el desempeño completo del sistema. Así, su diseño tendrá que ser supeditado a las necesidades de rendimiento del conjunto en global y será realizado en la segunda parte de este documento.

En este nivel encontraremos estructuras como el vocabulario, las tablas de ocurrencias, árboles,... a nivel de funcionamiento. También quedarán definidas las estructuras de ficheros que se usarán para los índices invertidos.

4.4. Identificación de las clases del sistema

Para facilitar este proceso, hemos navegado por los dos niveles que se describen en este documento y hemos descrito cada una de las posibles clases.

Para realizar la identificación, se ha seguido el documento de requerimientos y se han buscado los sustantivos más repetidos que designan entidades con sentido en el sistema, es decir: entidades, ocurrencias o eventos, papeles o roles, etc.

Los sustantivos o locuciones más encontrados son los siguientes⁴. Para cada uno se da una descripción más o menos detallada de a qué realidad corresponde, con objeto de validar su identificación:

4.4.1. Clases del nivel conceptual

- *Usuario* (*User*) esta clase simbolizaría a un usuario que realiza una serie de consultas sobre el sistema. Si hay varios usuarios conectados a la vez, sería una forma de distinguir a quién hay que devolver los resultados.
- *Administrador* (*Admin*) entidad que puede hacer cambios (inserciones/borrados/actualizaciones) en el sistema.
- *Sistema*: (*System*) entidad superior contenedora de todas las demás⁵.
- *Colección*: (*Collection*) conjunto de documentos sobre los que se pueden realizar consultas.
- *Resultado*: (*QueryResult*) un resultado de una consulta es una lista de elementos recuperables proporcionado tras introducir en el sistema una consulta determinada.

4.4.2. Clases que representan abstracción sobre estructuras del sistema

El proceso de identificar estas clases no es tan claro, puesto que los requerimientos del sistema aún están parcialmente especificados. En función de las necesidades

⁴Aunque se describen por su nombre en español, la posible utilización de la biblioteca de clases en ámbitos más internacionales hace casi necesario que le demos el nombre a cada clase en inglés. Éste aparece entre paréntesis y con letra *typewriter* detrás del sustantivo.

⁵Esta clase no participaría de la biblioteca de clases, sino que es la que contendría a cada sistema específico. De esta manera no debe tener operaciones ni métodos que sean “generales”. Debe construirse de cero para cada sistema específico, heredando de una clase base que se proporcione desde nuestra biblioteca con algunas operaciones elementales.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

del sistema (y las nuestras futuras), este subconjunto de clases puede variar, o refinarse.

Aunque en principio existe la referencia [26], no seguiremos su notación puesto que se trata de un modelo demasiado a “alto nivel” y no precisamente orientado a la recuperación estructurada (presenta un diseño de clases más bien general para un sistema de RI cualquiera).

- *Consulta*: (Query) se trata de una representación de la información que el usuario necesita. Es una clase de tipo *virtual* (es decir, no instanciable), existiendo una subclase para cada tipo de consulta que propongamos (p. ej.: por proximidad, por contenido, por estructura, ...), aún por definir.
- *Consulta indexada*: (IndexedQuery)
- *Ocurrencia*: (Occurrence) clase que modela la relación uno-a-muchos entre término y unidad estructural. Relaciona cada término con las unidades estructurales en las que se encuentra.
- *Término*: (Term) cada una de las unidades de más alto nivel de las que se componen las unidades del penúltimo nivel.
- *Stopword*: (Stopword) subclase de Término que tiene la particularidad de ser una palabra bastante frecuente y que no merece la pena ser indexada (tanto en documentos como en consultas).
- *Modelo de recuperación*: (RetrievalModel) clase que debe proporcionar operaciones para poder obtener la relevancia de cualquier unidad, dada una consulta. Su funcionamiento es transparente al resto del sistema de forma que sea posible intercambiarlo por otro modelo (p. ej.: siendo esta una clase virtual con los métodos ya definidos y que cada modelo concreto herede de ésta).
- *Unidad estructural*: (Unit) entidad estructural en la que se encuentra información. Puede estar compuesta de términos o de otras unidades de más bajo nivel.
- *DTD*: (DTD) entidad que representa la estructura que puede seguir un documento. No es el DTD en sí mismo, sino que incluiría los distintos tipos de etiquetas.
- *Vocabulario* (Lexicon) conjunto de términos presente en el sistema. Encapsula mecanismos eficientes de búsqueda y compresión (si existen).
- *Índice invertido* (InvertedIndex) Encapsula a las tablas de ocurrencias.

4.5. El nivel físico

4.5.1. Introducción

Este documento complementa al anterior. Su finalidad es proporcionar una visión global del funcionamiento del sistema **a un nivel físico**, describiendo las estructuras de datos que se utilizan para proporcionar eficiencia al sistema, distinguiendo entre las que se encuentran en memoria principal y las que están en almacenamiento secundario.

Inicialmente, describiremos la estructura del índice invertido, estudiando los algoritmos de búsqueda sobre el mismo. En documentos siguientes daremos un conjunto de clases sobre las que se apoyará este sistema.

4.5.2. Relaciones entre términos y unidades estructurales

Introducción al índice invertido básico

En los sistemas de RI tradicionales se utiliza un índice invertido representado como

$$Ind = \{ \{T_k, (D_j, \dots, D_x)\}_{k=1 \dots |\mathcal{T}|} \},$$

donde T_k es el término k -ésimo, los D_j simbolizan documentos donde aparece el término T_k ⁶ (pudiendo cambiarse por el par documento-frecuencia y/o añadir información del punto del documento donde se encuentra dicho término) y $|\mathcal{T}|$ es el número total de términos del sistema⁷.

En la práctica se suele descomponer el índice invertido en *vocabulario* (el conjunto de términos T_k) y *ocurrencias* (para cada término T_k la lista de apariciones D_j).

El vocabulario y los punteros a las ocurrencias correspondientes pueden ser albergados en memoria al tener un tamaño más reducido que las ocurrencias. Así, en nuestro sistema vamos a almacenar la totalidad del vocabulario en memoria y las ocurrencias trataremos de colocarlas en memoria lo máximo posible (dentro de la capacidad del equipo que se disponga).

Para el caso concreto de documentos estructurados, pasamos del concepto de documento al de *unidad estructural* (y por tanto, en vez de hacer mención a que

⁶Utilizamos la notación $D_j, \dots, D_x$ porque a priori no sabemos cuántos habrá ni qué identificadores tendrán.

⁷De ahora en adelante, todo término T_i , documento (o más genéricamente unidad estructural U_j) consideraremos que está representado por un entero correspondiente a su identificador único en el sistema.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

“un término aparece en un cierto documento”, en una ocurrencia almacenaremos que “un término aparece en una cierta unidad estructural”).

El índice invertido: vocabulario

El conjunto de palabras del sistema conceptualmente se trata de un array L ⁸ de cadenas donde cada una de ellas lleva asociada información sobre cómo encontrar su lista de ocurrencias y algunos contadores estadísticos (según sean necesarios). Además, cada elemento del array también contiene un identificador único (un entero) que se añade como un campo más a la estructura. Aunque podría pensarse que basta con tomar la posición en el array para obtener el identificador, no se ha hecho esto para permitir la posibilidad de una actualización en la colección (que pudiera introducir nuevos términos en el vocabulario con el consiguiente problema para los identificadores de los nuevos términos). Por otra parte, el tamaño del array L será, obviamente, $|\mathcal{T}|$.

Como este array reside completamente en memoria, la búsqueda por cadena (binaria) se realizará en tiempo $O(\log|\mathcal{T}|)$. Puesto que el tamaño del array puede ser suficientemente grande, consideraremos la posibilidad de implementar una búsqueda con interpolación, cuyo desempeño es $O(\log(\log|\mathcal{T}|))$, mejor que la búsqueda binaria (aunque puede resultar compleja su programación para el caso de cadenas).

Suponiendo un tamaño medio de las palabras (en el idioma español) de 6 caracteres (7 realmente, contando el terminador de cadena) y que para cada término se almacena un entero de 4 bytes⁹ para la dirección de la lista de ocurrencias, y otros 4 para el identificador, el tamaño de una unidad del array sería de $7 + 1 + 4 + 4 = 16$ bytes por término. Suponiendo una colección con 10^6 términos distintos (un límite muy superior al encontrado normalmente en colecciones de tamaño medio), el tamaño total del vocabulario sería de unos 15.26 MB. Esto es más que aceptable para los estándares de memoria de hoy en día.

Nótese que en principio no se ha utilizado ningún método de compresión, y a priori continúa sin ser necesario (aunque su uso puede ser discutible en el futuro si se necesita más espacio).

Para poder acceder a un término por su identificador hasta ahora estamos obligados a recorrer el vocabulario completo. Esta operación, de orden $O(|\mathcal{T}|)$ (es decir lineal en tiempo), puede ser tediosa si trabajamos con $|\mathcal{T}|$ bastante grande (que es el caso que nos ocupa). Para minimizar el impacto de esta operación, se

⁸De “lexicon”. Todas las variables se nombrarán según abreviaturas del término concreto en inglés.

⁹Asumimos que estamos tratando con una máquina de 32 bits. De lo contrario, si trabajamos con una máquina de $c \cdot 32$ bits, siendo c un entero mayor que 1, multiplíquese por c cualquier resultado final dependiente de la arquitectura mostrado en este documento.

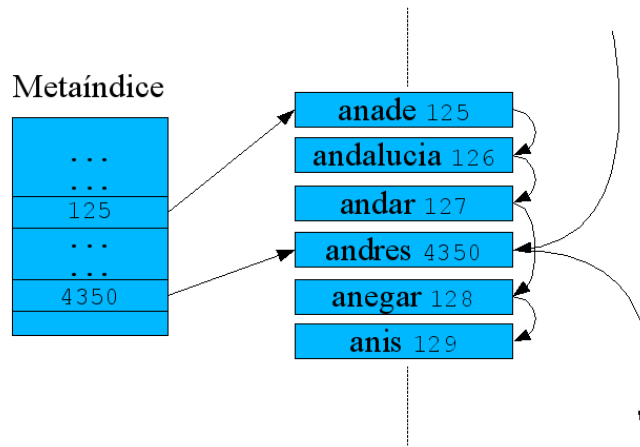


Figura 4.2: Fragmento de un vocabulario de ejemplo obviando los punteros a ocurrencias. Los términos representativos se han tomado de 25 en 25. Nótese cómo el término “Andrés” se introdujo posteriormente (en una ampliación de la colección) y por eso su identificador no es correlativo con los términos anterior y posterior según el orden lexicográfico.

propone añadir una dirección de memoria a cada término, apuntando al término cuyo identificador es el siguiente. Adicionalmente, una estructura (interna al vocabulario) que llamaremos *metaíndice* apuntará a unos términos (representativos) del vocabulario, tomados a intervalos regulares (por ejemplo, podemos tomar los términos 25, 50, 75, ..., es decir, cada 25 elementos). Para localizar un término por el identificador basta con localizar el término representativo más cercano, e ir recorriendo los punteros al siguiente. En el peor de los casos, se recorrería linealmente la mitad del intervalo elegido para los términos representativos (el intervalo es 25 en el caso anterior). La figura 4.2 presenta un ejemplo del funcionamiento de la búsqueda por identificador.

Las modificaciones anteriores, siguiendo con el ejemplo de la colección con 10^6 términos sólo añaden 32 bits por cada término más un array de punteros (32 bits cada uno) de tamaño $10^6/25$ (el metaíndice). En total resultan $0.25 + 0.15 = 0.4$ MB, lo cual incrementa en un 2.5 % el tamaño del vocabulario (de la estructura completa) pero soluciona el problema de la búsqueda por identificador, pasando de un tiempo lineal $O(|\mathcal{T}|)$ a recorrer 13 punteros en el peor caso (cuyo orden de eficiencia puede considerarse como de tiempo constante $O(1)$).

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

El índice invertido: ocurrencias y posiciones de aparición

Debido a la gran cantidad de accesos que va a soportar el índice, nos interesa encontrar un formato sencillo para almacenar las ocurrencias de un término en las diferentes unidades estructurales donde aparece. Una ocurrencia debe incluir (explícita o implícita, según su posición) información sobre:

- *Término* al que se refiere la ocurrencia (no aparece explícitamente, pero la ocurrencia es referenciada desde la entrada en el vocabulario).
- *Unidad(es)* en la(s) que aparece dicho término (cuántas en total) y *veces* en las que aparece en cada unidad.
- *Posiciones* donde está el término dentro de cada unidad en la que aparece.

Si no hubiera que almacenar información sobre la posición del término dentro de la unidad¹⁰, la estructura de una ocurrencia para un término cualquiera T_k , sería la siguiente:

$$\{m_k, \{\text{unit}_i, \text{freq}_i\}_{i=1\dots m_k}\}$$

siendo m_k el número de unidades distintas en las que aparece T_k y verificándose obligatoriamente que

$$\sum_{i=1}^{m_k} \text{freq}_i = n_k,$$

donde n_k es la frecuencia de aparición del término T_k en la colección.

El problema al introducir las posiciones dentro de la unidad es que, a diferencia del caso anterior, desconocemos el tamaño total del registro conociendo sólo el primer valor. La mejor solución que podemos dar es la siguiente, donde consideramos una ocurrencia con el siguiente formato (conceptual):

$$\{m_k, \{\text{unit}_i, \text{freq}_i, (\text{pos}_j)_{j=1\dots \text{freq}_i}\}_{i=1\dots m_k}\},$$

pero que realmente quedaría implementada en dos registros físicos distintos (dos archivos separados): uno con los datos de las ocurrencias (con un formato prácticamente igual que el primero que se ha propuesto) y una lista de posiciones para cada aparición del término dentro de una unidad concreta. Obviamente, desde el primer archivo habría que incluir información (ptr_i) de dónde está la lista de posiciones en el segundo archivo. La estructura de ambos registros para cada término sería la siguiente:

1. Archivo con los datos de las ocurrencias: $\{m_k, \{U_i, \text{freq}_i, \text{ptr}_i\}_{i=1\dots m_k}\}$

¹⁰Para permitir, por ejemplo, la realización de consultas basadas en proximidad de términos.

2. Archivo con los datos de las posiciones: $\{\{\text{pos}_j\}_{j=1\dots\text{freq}_i}\}_{i=1\dots m_k}$

En las descripciones anteriores, las variables significan:

- Número de apariciones, m_k , (número de unidades estructurales en las que aparece el término T_k).
- En la tripleta (U, freq, ptr) encontramos que “U” es el entero (identificador) de una unidad donde aparece el término, “freq” la frecuencia absoluta del término en dicha unidad, y “ptr” un desplazamiento para buscar en el archivo de posiciones.
- Finalmente pos son las posiciones de cada término en las que aparece en una unidad concreta. Para evitar problemas (por ejemplo, para no tener que reindexar la colección), almacenaremos desplazamientos absolutos desde el comienzo del documento.

Por extensión, los dos archivos mencionados se compondrán de una serie de registros consecutivos de los dos tipos descritos. A partir de ahora, y en lo sucesivo, nos referiremos al archivo de posiciones como F_{occ} y al archivo de posiciones (el que contiene la lista de los pos) como F_{pos} ¹¹.

Para finalizar, faltaría incluir la información referida al peso del término en la unidad. Simbolizamos por $w(T_k, U_i)$ el peso del término k en la unidad i de entre todas las que aparece. Generalmente el peso será un número real (de tipo float) y equivale a la importancia del término k en la unidad i . Cuanto más bajo sea el peso, menos importancia tendrá el término. Este peso será introducido en cada registro de la ocurrencia, como cuarto elemento.

Aunque sólo se trabaja a la vez con un peso, se permitirá que el modelo tenga precalculados una serie de pesos, existiendo una operación de “intercambio de pesos”, donde se pudieran reescribir los pesos de los términos (pertenecientes a un juego de pesos W_0) por otros distintos (pertenecientes a otro juego W_1 , por ejemplo).

Nótese que, aunque hemos especificado la existencia de estos elementos (pesos), no hemos precisado su forma de calcularlo. Generalmente, en sistemas no estructurados, se sigue un esquema de tipo $tf \cdot idf$, pero aquí podemos precalcular varios pesos, cada uno con su fórmula, e intercambiarlos (con la operación anteriormente mencionada) cuando sea necesario. El archivo encargado de almacenar el conjunto de pesos de cada término para cada unidad de último nivel lo notaremos como F_{weight} . En la siguiente subsección se explica con más detalle su estructura.

¹¹De “Occurrence file” y “Position file”, respectivamente.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

Almacenamiento de los pesos precalculados

Puesto que el cálculo del peso de un término en una unidad final implica bastantes operaciones de coma flotante (las más costosas) y tal vez incluso algún acceso a disco (si no tenemos en memoria los parámetros correspondientes) optaremos por precalcular y almacenar los pesos para evitar tener que calcularlos en tiempo de consulta.

Además, de esta forma se permite tener almacenados varios esquemas (juegos) diferentes de pesos, y cambiarlos para repetir una batería de experimentos con objeto de comparar cuál produce mayor desempeño.

Más concretamente, cuando hablamos de *peso* nos estamos refiriendo a dos tipos distintos:

1. Peso de un término T_i en una unidad U_j : $w(T_i, U_j)$.
2. Peso de una unidad U_i en su unidad hijo U_j : $w(U_i, U_j)$.

En ambos casos se trata de un número real que verifica algunas propiedades (ver [9] para más detalles).

Supongamos que se tienen precalculados y almacenados ω juegos de pesos $W_1 \dots W_\omega$ diferentes. Como puede observarse, en los pesos del tipo 1 (término en una unidad), tenemos que almacenar, para cada término, por cada unidad en la que aparece, ω pesos diferentes. Alternativamente, en el tipo 2, para cada unidad estructural (no raíz) hay que almacenar sólo ω pesos. Como puede verse, ambos casos tienen unas necesidades de almacenamiento bastante distintas.

La solución que proponemos trata de desaprovechar el menor espacio posible, haciendo el acceso a cada juego de pesos razonablemente eficiente, y posibilitando el eventual intercambio entre conjuntos de pesos distintos W_i y W_j (con $i \neq j$).

Inicialmente, tendremos ω ficheros de pesos, F_{weight_i} , cada uno de ellos internamente dividido en dos partes. La primera, apuntada por un archivo de acceso directo IF_{wterm} , contiene para cada término la dirección donde se encuentra un registro con el siguiente formato:

$$\{m_k, \{U_j, w(T_k, U_j)\}_{j=1 \dots m_k}\}$$

Donde m_k es el número de unidades en las que aparece el término T_k , y las U_j representan dichas unidades. En la segunda parte, apuntada por un archivo de acceso logarítmico IF_{wunit} , se encuentran registros que contienen los pesos una unidad en su hijo (tantos como unidades no finales). Este archivo de pesos genérico, corresponde a un sólo archivo de pesos. De esta forma, al mantener cada juego de pesos en un archivo, es posible eliminar un juego concreto W_i tan sólo borrando su archivo correspondiente. Además, los dos archivos índice, sirven para cualquiera de los diferentes juegos, por tener igual tamaño.

Sobra decir que estos archivos no serán de acceso normal en tiempo de consulta (esto es, existe una copia del peso “actual” en los archivos índice anteriormente mencionados) y que, como se ha dicho antes, existen tantos archivos F_{weight_i} como cada uno de los ω pesos precalculados.

El índice directo

Como acabamos de ver, se ha modelizado eficientemente la relación entre términos y unidades estructurales, aunque únicamente de la siguiente forma: la clase `Occurrence` simboliza que un término aparece en muchas unidades estructurales (uno a muchos).

Realmente, la relación entre términos y unidades es de tipo *muchos a muchos* (un término puede aparecer en varias unidades distintas, pero en una misma unidad pueden aparecer varios términos). Actualmente, con las estructuras que hemos definido para nuestro sistema, no es posible obtener, de forma eficiente, la segunda relación; es decir, no se puede responder de forma eficiente a la siguiente pregunta:

Para una unidad concreta, ¿cuál es la lista de términos que aparecen en ella?

Para evitar tener que recorrer la lista de ocurrencias e ir construyendo el resultado requerido (por cada término aparecido en la unidad, añadirlo a la lista), proponemos la creación de otra clase que modele esa misma relación. Por representar semánticamente el opuesto de `Occurrence`, la denominaremos `AntiOccurrence` y su contenido será, para cada unidad final, una lista de los términos que aparecen en ella.

Obviamente, ni esto es una necesidad prioritaria para muchos sistemas ni se requiere que se almacenen tantos detalles como en las ocurrencias. Por eso, sólo almacenaremos una lista de identificadores de términos (sin frecuencias ni información relativa a la posición), obviando términos repetidos y utilizando archivos índice propios. Estos archivos pueden ser bastante grandes (su tamaño es del orden del tamaño de la colección) y por eso, debe incluirse el dar la posibilidad, en tiempo de indexación, de crearlos o no.

La estructura de archivos sería sencilla. Por una parte, un archivo índice para buscar en otro mayor, con la misma forma que hemos descrito antes, IF_{anti} . Por otra parte, un archivo con las listas de **términos distintos** que contiene la unidad, precedidos de un entero con el número de elementos de la lista. Nótese que sólo se “indexarán” unidades de último nivel (`FinalUnit`).

Para evitar que este índice tenga un tamaño demasiado grande, se propone un esquema sencillo de compresión. Con él, almacenaremos los términos de cada lista por orden (y sin duplicar). De esta forma, se puede almacenar un entero “base”

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

(a partir del cuál, todos los identificadores de términos son mayores que él), y tras él, almacenar sólo diferencias entre el término anterior y el que estamos tratando en ese momento. Así, se reduciría el número de bits necesario para almacenar dichas diferencias, y la lista completa podría ocupar menos tamaño.

En definitiva, la lista de términos para una unidad U_i sería un registro en un archivo con la siguiente estructura:

$$\{\text{length}_i, nt_i, T_{i1}, \text{bits}_i, et_{i2}, \dots, et_{int_i}\}$$

Donde las variables significan:

- length_i (entero): longitud del registro completo, en bytes, incluyendo este entero.
- nt_i (entero): número de términos distintos que tiene la unidad i -ésima.
- t_{i1} (entero): término primero (base) que tiene la unidad i -ésima.
- bits_i (byte): número de bits que ocupan las diferencias entre términos.
- et_{ji} (entero de 'bits' bits): número que hay que sumarle al términos anterior en la lista (el resultado de calcular $et_{(j-1)i}$) para obtener el entero del término j -ésimo de esta unidad.

Los archivos que contienen esta información serán IF_{dir} y F_{dir} . Para aclarar este punto, vemos el siguiente ejemplo:

Supongamos una unidad estructural que contiene los siguientes términos distintos (por orden): $\{1, 3, 7, 8, 15, 20, 22, 25, 28, 30\}$. Para construir el registro, haríamos lo siguiente:

- Calculamos el **número de términos**: 10.
- Calculamos las **diferencias** entre un término y el anterior, almacenando el primero como base: $\{1, 2, 4, 1, 7, 5, 2, 3, 3, 2\}$
- Calculamos el **número de bits** necesario para almacenar la mayor de las diferencias, en este caso es 7, haciendo $\lceil \log_2 7 \rceil = 3$.
- Construimos el registro añadiéndole al principio la longitud de términos y de bits: $\{14, 10, 1, 3, 2, 4, 1, 7, 5, 2, 3, 3, 2\}$

Como vemos, en este caso hemos pasado de 40 bytes a 14 (quedando el espacio utilizado en menos de la mitad).

4.5.3. Estructuras en memoria

A pesar de que el uso de archivos invertidos mejora la velocidad global del sistema, los accesos a disco son el cuello de botella de un SRI, por lo que hay que minimizarlos. La solución tiene que ser mantener el mayor número posible de estructuras en memoria. Para esto debemos primero distinguir entre dos tipos diferentes de estructuras:

- Estructuras de tamaño variable: por ejemplo, cada una de las ocurrencias de un término en unidades estructurales. No hay ninguna forma, a priori, de obtener una estimación de su tamaño (salvo, claro está, consultándola).
- Estructuras de tamaño fijo: por ejemplo, la información referida a cada unidad. No se trata realmente de objetos o estructuras con tamaño fijo, pero sí podemos situarlos con cierta certeza en un cierto intervalo de tamaño. De esta forma, su tratamiento sería más flexible que en el primer caso.

Para el primer caso mantendremos un sistema de cachés estáticas (no se actualiza con las consultas) debido a la disparidad de tamaño entre los registros. Para el segundo caso, mantendremos una caché actualizable según consultas (con objeto de responder mejor a la localidad temporal).

Sistema de cachés estáticas

El principal objetivo de este sistema es el de mantener parte del índice invertido (las listas de ocurrencias) en memoria principal, para agilizar su acceso.

Un enfoque podría ser colocar el archivo invertido completamente en memoria (vocabulario y ocurrencias). Desgraciadamente, encontramos el problema de que la memoria principal suele ser de menor capacidad que el tamaño del índice (pudiendo tener éste el mismo tamaño que la colección). Además, se delegaría en el sistema operativo el intercambio de páginas de memoria a disco, y por tanto, el trasvase de información entre memoria principal y almacenamiento secundario. El segundo problema es que cada vez que existiera una falta de página, se produciría una excepción (tomando, por tanto el control el sistema operativo) con el consiguiente gasto asociado por sobrecarga debido a los cambios de contexto. Además, otro problema adicional es que no se tiene en cuenta el hecho de que haya partes del índice más utilizadas que otras (pudiéndose tener en memoria las primeras para buscar en menor tiempo).

En contraposición al enfoque anterior y a caballo entre aquél y el no mantener la tabla de ocurrencias en memoria (nuestra primera idea), proponemos usar una tabla *caché software* de ocurrencias, con objeto de llenar parte de la memoria con las listas de ocurrencias más utilizadas. Esto se conseguiría con las siguientes estructuras:

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- Tabla de ocurrencias T_{occ} (conjunto de ocurrencias para ciertos términos, inicialmente escogidos de manera razonable¹²). Se organizarían con un árbol para búsqueda.
- Archivo con un conjunto de *contadores* (F_{cnt}) que indican el número de consultas en las que se ha usado dicha entrada en el sistema. Se trata de un entero por término que inicialmente vale 0, y con cada consulta que se realiza al sistema, su valor se incrementa en 1 si dicho término pertenece a la consulta. Al ser un archivo con un entero por término, el acceso es directo y puede residir en almacenamiento secundario.
- Mapa de bits, M , para localizar rápidamente si la ocurrencia de un término concreto reside en memoria o no. Solamente haría falta 1 bit por término que estaría a 1 si la ocurrencia correspondiente está cargada en memoria (con lo que no accederemos a disco). Su valor estaría puesto a 0, en caso contrario.

Periódicamente se procesarían los datos del archivo de contadores para traer a memoria un nuevo “conjunto de caché” usando información basada en el uso que se le ha dado al sistema (los términos que más han aparecido en las consultas).

Sistema de cachés dinámicas

Este sistema se utilizará, fundamentalmente, para mantener en memoria objetos (seguramente de la clase “unit”), que no caben totalmente en memoria.

El funcionamiento es sencillo: utilizaremos la misma idea de la que se aprovechan las cachés asociativas por conjuntos ([29]). Para cada clase que utilice este tipo de almacenamiento, proponemos la siguiente estructura (dos arrays):

- Un array de punteros (ptrs) a objetos que contiene una serie de objetos en memoria.
- Un array de enteros (ids), que contiene el identificador del objeto alojado en la misma posición del array de punteros a objetos.

Ambos tendrían tamaño fijo y estarían complementados con una función *hash* cuidadosamente diseñada para proporcionar el mayor aprovechamiento posible del array (además de ser eficiente). A cada casilla del array le corresponderían todos aquellos objetos distintos cuyo hash del identificador fuera el índice de dicha casilla. El algoritmo para recuperar un objeto de esta clase sería el siguiente:

¹²Por ejemplo, mantener las ocurrencias de los términos más frecuentes en el vocabulario

ALGORITMO recuperar

```
    entrada: identificador i
    salida: objeto cuyo identificador es "i"

    h = hash(i)

    SI ids[h] != i ENTONCES
        ptrs[h] = recuperar_de_disco_objeto(i)
        ids[h] = i
    FIN_SI

    devolver ptrs[h]
```

FIN

Para el correcto funcionamiento de este sistema, las cachés deben estar inicialmente “cargadas” con objetos de disco. Por eso, se necesita un algoritmo de “precarga” con el que se inicia el sistema.

El único requisito necesario para poder especificar este algoritmo es tener una función que sea la inversa de la función hash. Como generalmente, el cardinal del codominio de dicha función es mucho menor que el del dominio y la función es inyectiva, pero no sobreyectiva, no se puede dar una definición de función inversa (para cada elemento de la imagen existen varios elementos posibles en el dominio). Nuestra función *ihash* del algoritmo simplemente tomará uno de dichos elementos calculado aleatoriamente. El valor inicial no importa, puesto que la efectividad de esta caché se hace patente cuando el sistema lleva ya funcionando un cierto tiempo y se han producido bastantes substituciones.

ALGORITMO precarga

```
    entrada: ninguna
    salida: ninguna
    efecto: precarga las cachés con elementos
            seleccionados aleatoriamente

    PARA i=0 HASTA tamaño(ids)

        h = ihash(i)
        ids[i] = h
        ptrs[i] = recuperar_de_disco_objeto(h)
```

FIN_PARA
FIN

4.5.4. Índice estructural

Relaciones entre unidades

Con este índice, tratamos de cubrir el hueco existente por ahora en el sistema: proporcionar de forma eficiente relaciones entre unidades estructurales.

Inicialmente, queremos poder *proporcionar la lista de unidades estructurales hasta la raíz desde una de último nivel*¹³. Para ello, usaremos un archivo que contenga esta serie de listas de identificadores de unidades, precedidos de la longitud del camino. Este archivo se conocerá como F_{desc} ¹⁴.

Para poder acceder eficientemente, usaremos un índice invertido auxiliar (IF_{desc}) en disco donde para cada unidad estructural de último nivel se dará el desplazamiento dentro de F_{desc} . Obviamente, IF_{desc} tendrá huecos que no será necesario llenar por ahora (referidos a unidades no estructurales).

Nótese que para mantener la efectividad de la caché estática, es necesario que las listas de descendientes correspondientes a las ocurrencias aparecidas en T_{occ} residan también en memoria (en una estructura que llamaremos por analogía a F_{desc} , T_{desc}).

Por otra parte, pretendemos también resolver el problema de, *dada una unidad estructural de nivel distinto del último, recuperar la lista de unidades que “cuelgan” de ella* (sus padres). Para ello, mantendremos en otro archivo un conjunto de listas de unidades contenidas en esta (“padres” según el modelo de recuperación) de una forma similar a como lo hacíamos en F_{desc} . Este archivo se llamará F_{par} y puesto que la intersección de las unidades por las que se consultará en éste con las del archivo anterior es nula, podemos utilizar IF_{desc} para apuntar a este índice, en los huecos que antes no habíamos llenado. Llamaremos entonces a IF_{desc} como $IF_{desc,par}$.

Información de cada unidad

La información sobre cada unidad (tipo, padre de la unidad, identificador, ...) no puede residir en memoria. Para poner un ejemplo, en la colección de prueba INEX,

¹³Es una exigencia derivada de las operaciones realizadas en el modelo BNR extendido.

¹⁴De “descendants file”. Aunque esta y otra notación parece cambiada respecto al concepto padre/hijo de las etiquetas en XML, realmente se refiere a la nomenclatura utilizada en el modelo BNR y superiores, a la que estamos más acostumbrados

cuyo tamaño total no llega a 1 GB, existen 18.5 millones de unidades, cada una de ellas conteniendo una serie de metadatos (esta y otras estadísticas sobre la colección pueden consultarse en [21]).

Como es obvio que esta colección podría ser mucho mayor (tal y como lo son las colecciones de hoy en día) y ya de por sí costaría almacenar esa información en memoria, hemos optado por almacenar cada uno de los objetos “unit” (véase diagrama de clases) en disco en un archivo F_{unt} . Para buscar en este archivo necesitaremos, obviamente un índice invertido que complementa a éste, y que llamaremos IF_{unt} . Nótese que el contenido de cada uno de los objetos “unit” será especificado más tarde, y por ahora sólo nos preocupamos en dónde almacenarlos. Adelantamos que se almacenará como un miembro más de dicha clase, la unidad hijo de esta (si la hay). De esta forma, podemos navegar eficientemente por las diferentes unidades y responder a todos los requisitos planteados anteriormente.

Información referida a los hipervínculos

Los hipervínculos (o enlaces) relacionan de forma directa unidades que no tienen por qué ser “familiares” unas de otras. Existen dos tipos de informaciones sobre las que tenemos que llevar cuenta:

- Hipervínculos *directos*: dada una unidad, a qué otras unidades apunta.
- Hipervínculos *invertidos*: dada una unidad, qué unidades le apuntan.

El tratamiento de ambos tipos no va a ser el mismo por una razón sencilla: dada una unidad estructural, a menos que se modificase su contenido (algo que no se va a permitir), es fácil saber a qué unidades apunta. Contrariamente, la lista de unidades que apuntan a otra puede variar conforme vaya creciendo la colección (si ésta es dinámica), por lo que debe manejarse con cuidado.

Inicialmente, proponemos dos archivos F_{dhyp} y F_{ihyp} (apuntados por sus correspondientes índices IF_{dhyp} e IF_{ihyp}) que almacenarán, respectivamente, las listas de hipervínculos directos e invertidos.

Debido a que la cantidad de unidades que enlazan o son enlazadas respecto al total serán más bien pocas, los archivos índice IF_{dhyp} e IF_{ihyp} no serán de acceso directo, sino de búsqueda binaria en disco.

DTDs y distintos tipos de etiquetas

Una de las características intrínsecas de un documento XML (véase [23], cap. 9) es la posibilidad de incluir diferentes tipos de etiquetas, a la par que unas reglas de inclusión de unas con otras. Esto es lo que se conoce como DTD (*Document Type Definition*): es una declaración en un documento XML que especifica restricciones

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

en la estructura del documento y proporciona la lista de los diferentes tipos de unidades del documento.

Como suponemos que los documentos proporcionados a nuestro sistema son XML bien formado (véase [23], cap. 2) y no contienen tipos de etiquetas no especificados en su DTD, sólo tendremos que almacenar una lista con los diferentes tipos de etiquetas y su identificador.

Esto no daría problemas si no se permitiera trabajar con colecciones que tuvieran varios DTD (heterogéneas). Por ejemplo: supongamos una colección con revistas científicas de biología y matemáticas (dos tipos de revistas distintas). Podemos suponer que cada una de ellas sigue un DTD distinto: por ejemplo, la etiqueta *section* puede significar, en las revistas de matemáticas, una sección de un artículo, y en las de biología una fotografía de una sección realizada a una muestra con un microtomo.

Si nos replanteamos el problema con este añadido (varios DTDs), tenemos que ser capaces de:

- A cada tipo de etiqueta de cada DTD, asignarle un identificador distinto (en el ejemplo anterior, ambos *section* tendrían un identificador distinto).
- Dado un identificador, recuperar la cadena del tipo de etiqueta que le corresponde (y saber también, por ejemplo, a qué DTD pertenece).
- Dada una cadena de un tipo de etiqueta, obtener la lista de identificadores que la apuntan, y para cada identificador, obtener el DTD al que pertenece.

Para recuperar la cadena de un identificador podríamos tener un archivo de registros, donde cada registro sería una cadena de un tipo de etiqueta F_{id-tag} . El archivo estaría apuntado por un índice ordenado por identificadores IF_{id-tag} .

Para el caso contrario, recuperar la lista de identificadores de una cadena, se podría mantener una tabla en memoria de cadenas, que para cada cadena T_{tag-id} diera la lista de identificadores.

Finalmente, para obtener el DTD de cada unidad, se podría mantener un archivo en disco $F_{tag-dtd}$ donde, para cada identificador de etiqueta, se diera el entero del DTD que le corresponde. En este caso, al ser una correspondencia de entero a entero, no hace falta un archivo índice para buscar en él.

4.5.5. Posibles optimizaciones

Compresión

Esta subsección ofrece un esbozo de los posible esquemas que podrían seguirse para presentar un mecanismo de compresión sencillo que permita recuperación sin descomprimir todo.

Compresión del vocabulario

En varias fuentes ([2] y [3]) se cita que una buena forma de compresión es almacenar el vocabulario en un árbol *trie*. Esta forma no nos ha parecido apropiada en nuestro estudio, sobre todo por el gran espacio desperdiciado en direcciones de memoria (punteros).

En [2] se ofrece un enfoque para la búsqueda en vocabulario basado en *árboles de prefijo* (árboles-*B* que contienen prefijos en vez de valores, de longitud igual al nivel de profundidad donde estemos). Por otra parte, en [43], se propone como mecanismo de compresión el almacenar prefijos y sufijos comunes para grupos de palabras. Aprovechando lo mejor de ambos, podemos colocar un árbol de prefijos (de un nivel razonable: 3, por ejemplo), y almacenar sólo el sufijo para cada palabra (o el caracter terminador de cadena si la palabra era de tres letras. Esto nos da un esquema más flexible para la búsqueda (usa un árbol-*B* para restringir el espacio del array en el que buscar) y ahorra espacio (pues cada prefijo se almacena una sólo vez). El único problema es que se restringe la longitud mínima de una palabra al número de niveles del árbol.

Compresión de las ocurrencias

Para poder almacenar más ocurrencias en memoria, se puede implementar compresión (además, de esta forma nos quedaría un índice invertido más pequeño y que tardaría menos en cargarse desde disco). Sin utilizar ningún algoritmo conocido, la idea es bien sencilla, siendo la estructura de una ocurrencia la anteriormente mencionada:

- Primeramente, si los identificadores de las distintas unidades se almacenan seguidos, podemos almacenar en el inicio el valor del primero, e ir colocando en los siguientes pares el valor de la diferencia con aquél. La diferencia entre el primero y el último es cota superior para los demás, por lo que podemos usar el número de bits del último para almacenar todos los demás.
- Segundo, aplicar la misma idea a las apariciones dentro de una unidad estructural en su fichero: almacenamos el primer valor, y hacemos la diferencia con los otros desplazamientos. El valor de la diferencia del último es cota superior para los demás y por tanto, el número de bits de la última diferencia sirve para almacenar los demás.

Tres notas a tener en cuenta sobre este método de compresión:

1. En el caso peor, en el que sólo haya una aparición ($m = 1$), se sobreentende que no se aplica compresión). De igual modo, cuando un término sólo aparece una vez dentro de una unidad estructural, tampoco se aplica compresión. Así, casi podemos asegurar que en ningún caso se perderá espacio por almacenar el primer valor.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

2. Aplicar este sistema de compresión también en disco puede acarrear problemas si la colección es dinámica, puesto que con la llegada de un nuevo documento, el número de bits anteriormente seleccionados puede no ser suficiente. Una solución a este problema es mantener el índice en disco sin comprimir y comprimirlo al llevarlo a memoria.
3. En el caso de comprimir las ocurrencias, hace falta un campo más en memoria que indique la longitud, en bytes (o múltiplo) de las ocurrencias comprimidas.

Uso de políticas de reemplazamiento basadas en un *conjunto de trabajo*

En los sistemas operativos modernos que implementan paginación, al traer una página a memoria, se suele traer un pequeño grupo de páginas (llamado “conjunto de trabajo” que están alrededor suya, teniendo en cuenta dos características del sistema:

- El principio de *localidad*, que asegura que en tiempos cercanos y con alta probabilidad se accederá a una página cercana.
- La gran posibilidad de que el resto de páginas ya se haya leído anteriormente o se lea con ésta en el mismo bloque de disco y por tanto, resida en caché. Hemos tratado la localidad espacial, pero ahora trataremos la temporal.

De esta forma, se puede acelerar el acceso a direcciones de memoria cercanas. Para más información sobre esta técnica, puede consultarse [16].

Desde el punto de vista de los sistemas de recuperación de información, se puede pensar que un usuario que realiza una consulta con un término, es posible que refine dicho término en consultas siguientes (p.ej: pasar de buscar “congreso” a “congresistas”) y por tanto, habría que volver a acceder a una zona muy cercana a la accedida anteriormente (en el caso en el que las ocurrencias estuvieran ordenadas alfabéticamente). Continuando con el ejemplo anterior, si en la primera vez que se hizo la consulta, se trajo también las ocurrencias de “congresistas” a memoria, es posible acelerar el tiempo de respuesta de la segunda consulta y reduciendo el tiempo total de la suma de ambas.

Otra posibilidad es que este tipo de “realimentación” fuera implementada de forma automática por el sistema y esta mejora sirviera para poder tratarla.

4.5.6. Tabla-Resumen sobre las distintas estructuras

Símbolo	Descripción	Localización
V	Vocabulario. Array de términos	Memoria
F_{occ}	Archivo de ocurrencias	Disco
F_{pos}	Archivo de posiciones	Disco
T_{occ}	Caché software de ocurrencias	Memoria
T_{desc}	Caché software de listas de hijos (complementaria a la anterior).	Memoria
M	Mapa de bits para comprobar si una ocurrencia está en caché	Memoria
F_{cnt}	Archivo de contadores para la caché	Disco
F_{desc}	Archivo conteniendo caminos a la raíz para una unidad	Disco
F_{par}	Archivo conteniendo padres de una unidad	Disco
$IF_{desc,par}$	Archivo invertido auxiliar para uso con los dos anteriores	Disco
F_{unt}	Archivo conteniendo información sobre cada unidad	Disco
IF_{unt}	Archivo invertido para buscar en F_{unt}	Disco
F_{dhyp}	Archivo con las unidades a las que apunta una unidad concreta	Disco
IF_{dnyp}	Archivo invertido para buscar en F_{dhyp}	Disco
F_{ihyp}	Archivo con las unidades que apuntan a una concreta	Disco
IF_{ihyp}	Archivo invertido para buscar en F_{dhyp}	Disco
F_{dir}	Archivo con los términos que tiene una unidad	Disco
IF_{dir}	Archivo invertido para buscar en F_{dir}	Disco
F_{id-tag}	Archivo que enlaza ids. de etiquetas con su cadena	Disco
IF_{id-tag}	Archivo invertido para buscar en F_{id-tag}	Disco
T_{tag-id}	Tabla con la lista de ids. que corresponde a cada etiqueta (cadena)	Memoria
$F_{tag-dtd}$	Archivo que enlaza a cada id. de etiqueta su id. de DTD	Disco
F_{anti}	Archivo de antiocurrencias	Disco
IF_{anti}	Archivo invertido para buscar en F_{anti}	Disco
F_{weight_i}	Archivo con el peso i -ésimos precalculado	Disco
IF_{wunit}	Archivo invertido para buscar en $F_{weight_i}, \forall i$	Disco
IF_{wterm}	Archivo invertido para buscar en $F_{weight_i}, \forall i$	Disco

4.6. Diseño de las clases (nivel lógico)

4.6.1. Introducción

Esta parte cierra el apartado de diseño. Desde él se proponen un contenido (atributos y operaciones) desde cada clase y se contrasta con los requisitos del primero. Para su realización (aunque aquí no se contempla), se ha iterado desde una propuesta inicial hasta la aquí obtenida, añadiéndose en cada paso los atributos y los métodos correspondientes a cada clase.

En la sección primera, se da una visión del contenido de las clases, describiendo cada campo lo necesario, y posteriormente, se expresan los algoritmos no triviales con la notación descrita anteriormente.

Sobre la notación empleada

Para los tipos de los atributos y los devueltos por los métodos (si los hubiera) se ha empleado la misma notación usada por el C++. Se ha preferido utilizar `char *` en vez de `string` por su mayor posibilidad de control (y para no depender de una cierta implementación) además de que sospechamos que `string` ocupa mayor cantidad de memoria, lo que no nos interesa. Para los tipos no primitivos, se utiliza, para cada clase, el mismo nombre que se indica en este documento.

Los atributos son todos privados, salvo que se especifique lo contrario. Para cada atributo de cada clase existen métodos `get...()` y `set...()` que obtienen o ponen a un valor, respectivamente, atributos de un objeto. Por otra parte, es especialmente planteable el cuándo hacer la actualización hacia el disco duro si se hace algún `set`. Por esto, para cada clase existirá también un método `store` para re-escribirla en disco si se actualiza alguno de sus datos (en el caso en el que se considere factible la posibilidad de actualizar). Nótese que, en la mayoría de los casos, una actualización es un proceso bastante costoso porque implica, en la mayoría de los casos, más de un acceso a disco, además de tener que cuidar la no aparición de inconsistencias.

4.6.2. Atributos y operaciones

AntiOccurrence

- **Descripción:** representa las apariciones de los diferentes términos en una unidad.
- **Atributos:**
 - **num** (int): número de términos (distintos) que contiene la unidad.

- **sum** (int): número total de apariciones de términos indexables en la unidad.
- **termId** (int): identificador del término i-ésimo distinto que aparece en la unidad.
- **termFreq** (int): veces que aparece el término i-ésimo en la unidad.

■ **Métodos:** Ninguno.

Collection

■ **Descripción:** es la abstracción sobre el conjunto de documentos que la compone.

■ **Atributos:**

- **description** (char *): texto significativo con la descripción de la colección.
- **dtdList** (int *): lista de identificadores de los DTDs de los documentos que contiene la colección.
- **files** (char *[]): array de nombres de los archivos que componen la colección (el único objetivo es utilizarlo como información “consultable” *ad hoc*).
- **id** (int): identificador de la colección en el sistema.
- **indexInfo** (info (int)): entero que codifica (de alguna forma definida posteriormente) las diferentes opciones de la indexación.
- **num** (int): entero con el número total de documentos que componen la colección.
- **numDtd** (int): entero con el número total de DTDs de la colección.
- **numUnits** (int): entero con el número total de unidades estructurales de la colección.
- **numFinalUnits** (int): entero con el número total de unidades estructurales finales de la colección.

■ **Métodos:**

- + char *getDescription: devuelve la descripción de la colección.
- + int *getDtdList(): devuelve el array con los identificadores de los DTDs.
- + int getFinalNumUnits(): devuelve numFinalUnits.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- + `int getId()`: devuelve el identificador de la colección.
- + `int getIndexInfo()`: devuelve la información sobre la indexación de la colección.
- + `int getNum()`: devuelve `num`.
- + `int getNumDtd()`: devuelve `numDtd`.
- + `int getNumUnits()`: devuelve `numUnits`.
- + `void printInfo()`: imprime (por la salida estándar) un resumen con las principales características de la colección.

DTD

- **Descripción:** esta clase recubre la correspondencia entre los tipos de etiquetas y su cadena, además de otras organizaciones de estructura, vistas en el nivel físico.

Esta clase encapsula a los archivos F_{id-tag} , IF_{id-tag} y la estructura en memoria T_{tag-id} , así como el archivo $F_{tag-dtd}$, definidos en el apartado referido al nivel físico.

- **Atributos:**

Ninguno. Toda la información necesaria está contenida en los archivos índice que encapsula.

- **Métodos:** todos los métodos son de clase, no se crearán objetos de tipo DTD.

- + `char *getStringFromId(int id)`: devuelve la cadena de un tipo de etiqueta, dado su identificador.
- + `int getImportance(int id)`: devuelve la importancia de un tipo de etiqueta (definida “a mano”).
- + `list <int>getUnitsFromType(int id)`: devuelve la lista de unidades de la colección cuyo tipo es `id` (ver apunte sobre la eficiencia en la siguiente sección).

FinalUnit

Inicialmente, no añade nada a la definición de `Unit`.

Lexicon

- **Descripción:** contiene el conjunto de términos del sistema.
- **Atributos:**
 - **num** (int): número de términos del léxico. A la hora de introducir una nueva palabra, el valor de su id será $\text{num} + 1$.
 - **metaIndex** (Term *): punteros a ciertos términos (metaíndices), separados interval unidades.
 - **interval** (int): intervalos sobre los cuales se hace apuntar cada uno de los metaíndices.
- **Métodos:**
 - + **getIdFromString**(char *str): devuelve el identificador correspondiente a un término (si está en el sistema). Si no, devuelve -1 . Este método realiza una búsqueda binaria en el vector de términos que está en memoria.
 - + **getTermFromId**(int id): devuelve el puntero a un término correspondiente al identificador que le pasamos (debe ser un identificador válido). Este método aprovecha los punteros next del conjunto de términos y el metaIndex (ver parte referida al nivel físico) con objeto de realizar esta operación en tiempo $O(1)$.

Occurrence

- **Descripción:** abstracción de las apariciones de un término en distintas unidades estructurales.
- **Atributos:**
 - **num** (unsigned): número de unidades distintas en las que aparece el término al que corresponde esta ocurrencia.
 - **unit** (unsigned *): array con los id. de las unidades donde aparece el término.
 - **freq** (unsigned *): array con el número de veces que aparece en cada unidad este término.
 - **offset** (unsigned *): offset en el archivo F_{app} (no consultable ni modificable).

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- **weight** (float *): peso del término en la unidad i-ésima donde aparece

■ Métodos:

- + int getNum(): devuelve el número de unidades distintas en las que aparece el término que corresponde esta ocurrencia.
- + pair <int, int> getDirectUnits(): devuelve la lista de unidades de aparición directa (finales) y las frecuencias de aparición.
- + list <int>getPositions(int unt): devuelve la lista de posiciones de un término en una unidad unt pasada como parámetro. Si el término no aparece en dicha unidad, devuelve una lista vacía.
- + list <int>getUnitsOfType(int type): devuelve la lista de unidades que son del tipo cuyo identificador es pasado como parámetro.
- + pair <int, int> getIndirectUnits(): devuelve la lista de unidades de aparición indirecta (no finales) y las frecuencias de aparición.

Term

- **Descripción:** representa a un término del sistema (por tanto, aparece en alguna unidad). Todos los términos se mantienen en memoria como tales (dentro del Lexicon.)

■ Atributos (todos private):

- **termString** (char *): cadena que representa al término.
- **offsetFileOcc** (int): desplazamiento del término en el archivo de ocurrencias (permite localizar su lista de ocurrencias asociada).
- **id** (int): identificador único del término en el sistema.
- **next** (Term *): puntero al término cuyo identificador es el siguiente.

■ Métodos:

- + Occurrence retrieveOccurrence(): devuelve la lista de ocurrencias del término que recibe este mensaje.
- + int getId(): devuelve el identificador de un término.
- + int getLength(): devuelve la longitud del término.
- + char *getTermString(): devuelve la cadena de un término.
- + int getFreq(): devuelve la frecuencia de aparición de un término en la colección (recuperada desde la correspondiente ocurrencia).

- + `int getMinimumDistance(int unt, int id2)` devuelve la mínima distancia entre las apariciones del término que recibe el mensaje y el representado por `id2` en la unidad cuyo identificador es `unt`.
- + `byte *dump()`: produce un volcado en binario de este objeto.

Unit

- **Descripción:** representa una unidad estructural, que bien puede ser un contenedor de unidades, bien de términos (en cuyo caso será una clase derivada de ésta llamada `FinalUnit` [de todas formas, ambas comparten estos atributos y operaciones]).

Esta clase encapsula a las estructuras F_{unt} , IF_{unt} , F_{desc} , F_{par} , $IF_{desc,par}$, IF_{ihyp} , F_{ihyp} , IF_{dhyp} , F_{dhyp} , IF_{dir} , F_{dir} y F_{anti} .

- **Atributos:**

- **id** (int): identificador único de la unidad en el sistema.
- **route** (char *): ruta Xpath completa de la unidad.
- **type** (int): tipo de la unidad, que se corresponderá con un tipo de un cierto DTD.
- **position** (int): posición en el contenedor padre. Para el caso de ser el único hijo, la posición es 1. Igualmente, para el caso de ser una etiqueta raíz, la posición es 1.
- **container** (int): identificador de la unidad que contiene a esta. Si es raíz, su valor será 0.
- **numContained** (int): número de descendientes¹⁵, si es un contenedor. Número de términos, si es un objeto de la clase `FinalUnit`.
- **weight** (float): peso de la unidad en su contenedora.

- **Métodos:**

- + `char *getRoute()` devuelve la ruta XPath de la unidad.

¹⁵Esta información realmente está duplicada, pero tiene que ser así. Esto es, si tan sólo queremos saber cuántos descendientes tiene una unidad, y la unidad es de tipo contenedor, se puede obtener el número de hijos mirando en F_{par} (archivo de padres). Si el objeto es una unidad final, no hay otra forma de hacerlo, pero por razones de homogeneidad, se incluye este dato en la clase padre y en la derivada. Obviamente, en el caso de existir duplicada esta información, se usará la de más rápido acceso.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- + `Unit(int id)` constructor a partir del identificador (accede a F_{unt} y devuelve el objeto con dicho identificador). Reposa sobre la caché de objetos unidad gestionados mediante una función hash y un array de objetos de tamaño prefijado.
- + `int getType()` devuelve el tipo de la unidad.
- + `Unit.getUnitType(int id)` devuelve el tipo de la unidad cuyo identificador pasamos como parámetro (método de clase).
- + `getContainer()` devuelve el identificador de la unidad que contiene a ésta.
- + `getPosition()` devuelve la posición de la unidad en su contenedora.
- + `list <int>getDescendants()` devuelve la lista de descendientes de la unidad.
- + `list <int>getBrothers()` devuelve la lista de “hermanos” de la unidad.
- + `list <int>getDirectLinks()` devuelve la lista de unidades enlazadas por la unidad (hace uso de IF_{dhyp} y F_{dhyp}).
- + `list <int>getInverseLinks()` devuelve la lista de unidades enlazadas por la unidad (hace uso de IF_{ihyp} y F_{ihyp}).
- + `list <int>getPathToRoot()` devuelve la lista de unidades hasta la unidad raíz del documento, dando la secuencia de unidad-unidad contenedora y así sucesivamente.
- + `int getNumContained()` devuelve el número de elementos contenidos en la unidad (pueden ser términos u otras unidades, según el tipo de esta).
- + `float getWeight()` devuelve el peso de la unidad en su contenedora (un valor no determinado en caso de que sea unidad raíz).
- + `AntiOccurrence retrieveAntiOccurrence()` devuelve la antiocurrencia (índice directo) relativo a una unidad (este método se vale de los archivos IF_{dir} , F_{dir} y F_{anti}).
- + `int getIdFromXPath(char *str)` devuelve el identificador de una unidad estructural dada la ruta XPath que la identifica (método de clase).

4.6.3. Especificación de los algoritmos requeridos

Esta sección trata de especificar con la notación de clases y métodos anteriormente propuesta, los algoritmos requeridos en la especificación inicial. Además, se trata

de dar una aproximación sobre su eficiencia puesto que no sólo es necesario que un algoritmo sea factible, sino que sea eficiente.

Notación: El tiempo de posicionamiento, acceso y transferencia de una serie de datos con *acceso directo* a disco será representado por la constante k . Cuando una operación requiera t accesos directos a disco, escribiremos $k \cdot t$.

Área estructural

Requisitos referidos a la colección

Nota: todos los algoritmos son $O(1)$.

- Dada una colección, devolver su descripción: `collection.getDescription()`.
- Dada una colección, devolver su DTD o lista de ellos: `collection.getDtdList()`.
- Dada una colección, devolver el número de documentos que la componen: `collection.getNum()`.
- Dada una colección, devolver el número de unidades que contiene: `collection.getNumUnits()`.
- Dada una colección, devolver el número de términos que contiene: `collection.getNumFinalUnits()`.
- Dada una colección, devolver la información referente a la indexación: `collection.getIndexInfo()`.

Requisitos referidos a los tipos de etiqueta

- Dado un tipo de etiqueta mediante su identificador, devolver su cadena de caracteres: `Dtd.getStringFromId(id)` (un sólo acceso a disco, $O(k)$).
- Dado un tipo de etiqueta (id), devolver el grado de importancia: `Dtd.getImportance(id)` (un sólo acceso a disco, $O(k)$).
- Dado un tipo de etiqueta (id), devolver la relación de unidades (ids) que son de ese tipo `Dtd.getUnitsFromType(id)` (**muy ineficiente**: recorre todas las unidades del sistema haciendo dos accesos directos a disco por cada una: $O(2 \cdot U \cdot k)$).

Requisitos referidos a las distintas unidades

En todo caso en el que se requiera una unidad concreta y no se tenga “a mano”, habrá que recuperar el objeto unidad si no se tiene ya (2 accesos a disco, $O(2 \cdot k)$), mediante el constructor específico que proporciona la clase `Unit`.

- Dada una unidad, identificada con su camino XPath, devolver su identificador: `Unit.getIdFromXPath(char* str)` (*bastante ineficiente*, del orden del nivel de ramificación medio del árbol XML, multiplicado por el número de niveles de la ruta, por k [accesos a disco]).

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- Dada una unidad, identificada por su clave única, devolver su camino XPath: devolver el atributo `route` con `unit.getRoute()` ($O(1)$).
- Dada una unidad (`id`), devolver el tipo de etiqueta al que pertenece: devolver `unit.getType()` ($O(1)$).
- Dada una unidad (`id`), devolver el tipo de etiqueta al que pertenece: podemos realizarlo de 2 maneras distintas:
 - + Si tenemos la unidad en memoria, basta retornar el tipo con `unit.getType()` ($O(1)$).
 - + Si no queremos recuperar la unidad, podemos utilizar el método `Unit.getUnitType(in id)` (de clase) que sería más rápido que recuperar el objeto al completo y luego devolver el entero correspondiente ($O(k)$).
- Dada una unidad (`id`), devolver la unidad donde está contenida (`id`), o nulo si es la raíz: `unit.getContainer()` ($O(1)$).
- Dada una unidad (`id`), devolver la posición del contenedor en donde está incluida aquella: `unit.getPosition()` ($O(1)$).
- Dada una unidad (`id`), devolver la relación de unidades (`ids`) que contiene (si es un contenedor de unidades): `unit.getDescendants()` ($O(2 \cdot k)$). Hace uso de $IF_{desc,par}$ y F_{par} .
- Dada una unidad (`id`), devolver la relación de hermanos: `unit.getBrothers()`. El algoritmo pregunta por el contenedor de la unidad, y devuelve una lista de unidades contenidas de la misma, eliminando la unidad receptora del mensaje. Sólo requiere dos accesos a disco ($O(2 \cdot k)$).
- Dada una unidad (`id`), devolver las unidades a las que apunta (hacia las que existe una referencia): `unit.getDirectLinks()`. Siendo U el número de unidades, el orden de eficiencia es $O(k + k \cdot \log_2 U)$ (búsqueda binaria en disco).
- Dada una unidad (`id`), devolver las unidades que la apuntan (desde donde referencian a la primera): `unit.getInverseLinks()`. El orden de eficiencia es $O(k + k \cdot \log_2 U)$ (búsqueda binaria en disco).
- Dada una unidad (`id`), devolver el número de términos que contiene o unidades, según el tipo de contenedor que sea: `unit.getNumContained()` ($O(1)$).
- Dada una unidad (`id`) contenida y otra contenedora, devolver el peso de la primera en la segunda: `unit.getWeight()` ($O(1)$).

Área textual:

- Dado un término (id), y una unidad, devolver la relación de términos incluidos en la misma unidad que el original, con su posición en la unidad.
- Dado un término (id) y una unidad (unt), devolver la relación de posiciones: `term.retrieveOccurrence().getAppearances(int unt)`.
- Dado un término (id), devolver el número de unidades de último nivel donde aparece `term.retrieveOccurrence().getNum()` (un acceso a disco, eficiencia: $O(k)$).
- Dado un término (cadena de caracteres), devolver su identificador: utilizamos el método `Lexicon.getIdFromString(char *str)`, con eficiencia $O(\log_2 T)$, siendo T el número de términos del sistema.
- Dado un término (identificador), devolver su cadena de caracteres (que lo representa): `Lexicon.getTermFromId(int id).getTermString()` (eficiencia: $O(1)$).
- Dado un término (id), devolver la relación de las unidades de ocurrencia directa y su frecuencia de aparición: recuperamos el término (`Lexicon.getTermFromId(int id)`) ó directamente hacemos `term.retrieveOccurrence().getDirectUnits()` (un acceso a disco, $O(k)$).
- Dado un término (id), devolver la relación de las unidades de ocurrencia indirecta y su frecuencia de aparición: `term.retrieveOccurrence().getIndirectUnits()`. El algoritmo es igual que el `getDirectUnits`, sólo que además, va construyendo un árbol en memoria con los caminos a la raíz (F_{desc}) y sumando las frecuencias. Finalmente, devolvemos el árbol por niveles.
- Dada una unidad (id), devolver la relación de términos que contiene y su frecuencia, de manera directa: `unit.retrieveAntiOccurrence()`
- Dado un término (id) y un tipo de etiqueta (id), devolver la relación de las unidades de ese tipo que lo contienen, y su frecuencia de aparición: primero recuperamos el término y posteriormente, la ocurrencia, con la que hacemos `occurrence.getUnitsOfType(int type)`. El algoritmo es el siguiente: para cada unidad de que aparece en la ocurrencia, comprobamos el tipo en F_{unt} (si no está en caché) y añadimos dicha unidad a la lista que se devolverá si el tipo es el buscado. Finalmente devolvemos dicha lista. Siendo u el número de unidades en las que aparece el término correspondiente, el orden de eficiencia es $O(k \cdot (u + 1))$.

CAPÍTULO 4. DESARROLLO DE UN SISTEMA PARA RECUPERACIÓN DE DOCUMENTOS ESTRUCTURADOS

- Dados dos términos (ids) y una unidad (id), devolver la distancia mínima entre las posiciones de éstos en la unidad: `term.getMinimumDistance(int unt, int id2)`

Capítulo 5

Diseño de los algoritmos

5.1. Introducción

Este capítulo se centra en el proceso de diseño y construcción de los algoritmos más relevantes del sistema que hemos diseñado. Como ya se ha comentado en repetidas ocasiones, debido a las necesidades de alto desempeño en tiempo de este tipo de sistemas, hemos hecho especial hincapié en realizar un diseño a nivel físico muy detallado.

Por este mismo motivo, vamos a cuidar también de realizar un diseño de los algoritmos que sea capaz de aprovechar las estructuras anteriormente diseñadas, y que sea capaz de proporcionar un buen funcionamiento.

En resumidas cuentas, los dos algoritmos fundamentales que tendremos que diseñar son:

- El algoritmo de indexación para una colección completa.
- El algoritmo de recuperación.

5.2. Diseño de un algoritmo de indexación para documentos estructurados

5.2.1. Operaciones necesarias: programas de indexación

En esta sección se especifican las diferentes funciones que son necesarias para la gestión del conjunto de índices de una colección (y otras operaciones relacionadas) que serán invocadas mediante programas. Las funciones más básicas son las siguientes:

1. Crear una colección nueva con una serie de archivos (creación de una colección desde sus archivos componentes).
2. Eliminación del sistema de una colección completa y de todos sus archivos.
3. Indexar un conjunto de archivos que pertenezcan a una colección, especificando “manualmente” la colección (añadir más documentos a la colección).
4. Eliminación de un archivo de una colección (los antecesores no comunes de una unidad de primer nivel).
5. Intercambiar pesos desde los archivos de pesos a los índices invertidos.

Las funciones 1, 2 y 5 son básicas. 3 puede implementarse provisionalmente a partir de 1 y de 2. De la misma forma, si la indexación es rápida, puede considerarse la no realización de 4, puesto que 1 y 2 la substituyen, y posteriormente implementarla de forma específica¹.

Pueden proponerse algunas otras para completar la lista, según vayan haciendo falta. La más compleja es la indexación de una colección (1), y es a la que le dedicaremos esta parte. El resto son fácilmente implementables (eliminando directamente archivos índice ó reindexando la colección) a partir de ésta.

5.2.2. Métodos de inversión

Esta sección da un esbozo de los algoritmos que hacen la inversión. Principalmente existen dos tipos de inversión: *en memoria* y *mediante ordenación*. Los demás que pudieran surgir, son simplemente derivados de aquéllos dos.

¹De todas formas, siempre será más fácil eliminar algunos documentos de la colección, que añadir nuevos, porque el índice resultante ocupará menor o igual espacio en el primer caso, y seguramente mayor en el segundo

5.2. DISEÑO DE UN ALGORITMO DE INDEXACIÓN PARA DOCUMENTOS ESTRUCTURADOS

Inversión en memoria

El método de inversión más sencillo que puede plantearse es realizar la lectura secuencial de los archivos a memoria e ir rellenando estructuras en ésta. De esa forma, si existiera suficiente espacio para realizar este proceso, el orden de eficiencia sería prácticamente lineal respecto al tamaño de los documentos.

Nada más lejos de la realidad. Se estima que, para un texto sin estructurar (caso en que el índice es más sencillo) según [3], el índice puede ocupar de un orden de $O(n^\beta)$ ($\beta \in [0.4, 0.6]$) para el vocabulario, más $O(n)$ para las ocurrencias. Teniendo en cuenta que estamos tratando con un caso estructurado y que almacenamos cada posición de cada término en la unidad contenedora, esta cifra seguramente se disparará muy por encima. Además, algunas colecciones de prueba estándar empiezan a llegar a un tamaño de varios GB, lo cuál es, a día de hoy, imposible de manejar en memoria.

El algoritmo tal y como lo encontramos en [43] es el siguiente:

1. `/* Initialization */`
Create an empty dictionary structure S
2. `/* Phase one: collection of term appearances */`
For each doc Dd in the collection ($1 \leq d \leq N$)
 - (a) Read Dd, parsing it into index terms
 - (b) For each index term $t \in Dd$
 - i. Let $fd;t$ be the frequency in Dd of term t
 - ii. Search S for t
 - iii. If t is not in S, insert it
 - iv. Append a node storing (d; $fd;t$) to the list corresponding to term t
3. `/* Phase two: output of inverted file */`
For each term $1 \leq t \leq n$
 - (a) Start a new inverted file entry
 - (b) For each (d; $fd;t$) in the list corresponding to t, append (d; $fd;t$) to this inverted file entry
 - (c) If required, compress the inverted file entry
 - (d) Append this inverted file entry to the inverted file

Podemos concluir que, dada la amplia necesidad de espacio requerido para los índices de una colección estructurada, este algoritmo **no servirá** para nuestro propósito.

Inversión basada en ordenación

La idea que nos propone este algoritmo está un poco más elaborada que el anterior. A grandes rasgos, da **dos pasadas**: una que obtiene registros $\langle T_i, U_j, f_{i,j} \rangle$ (frecuencia de aparición del término T_i en la unidad U_j), volcándolos a un archivo temporal por lotes (ordenados por T_i), para posteriormente ir haciendo mezcla de lotes de dos en dos (ordenando por U_j) y finalmente, al quedar un solo lote, construir el índice.

El algoritmo es el siguiente:

1. /* Initialization */
 Create an empty dictionary structure S
 Create an empty temporary file on disk
2. /*Proces text and write temporary file */
 For each document Dd in the collection, $1 \leq d \leq N$
 - (a) Read Dd, parsing it into index terms
 - (b) For each index term t in Dd
 - i. Let fd;t be the frequency in Dd of term t
 - ii. Search S for t
 - iii. If t is not in S, insert it
 - iv. Write record (t; d; fd;t) to the temporary file, where t is represented by its term number in S
3. /* Internal sorting to make runs */
 Let k be the number of records that can be held in memory
 - (a) Read k records from the temporary file
 - (b) Sort into nondecreasing t order, and for equal values of t, nondecreasing d order
 - (c) Write the sorted run back to the temporary file
 - (d) Repeat until there are no more runs to be sorted
4. /* Merging */ Pairwise merge runs in the temporary file until it is one sorted run
5. /* Output inverted file */
 For each term $1 \leq t \leq n$
 - (a) Start a new inverted file entry
 - (b) Read all triples (t; d; fd;t) from the temporary

5.2. DISEÑO DE UN ALGORITMO DE INDEXACIÓN PARA DOCUMENTOS ESTRUCTURADOS

```
file and form the inverted file entry for term t
(c) If required, compress the inverted file entry
(d) Append this inverted file entry to the inverted
file c
```

Obviamente, el algoritmo tendría que ser adaptado al caso estructurado, construyendo en disco de las estructuras de datos propias de nuestro sistema. Además, en el algoritmo original, no se tratan relaciones entre documentos (unidades estructurales) que, en nuestro caso sí hace falta tratar.

La inversión basada en ordenación, tal y como se describe, tarda unas 20 horas en invertir 5 GB de texto [43], usando 40 MB de memoria. Como disponemos de bastante memoria principal en los computadores de hoy en día, suponemos que este tiempo será bastante menor.

Otros métodos

El principal inconveniente del enfoque anterior es que, en cierto momento de la indexación, *requiere un tamaño total en disco igual al doble del tamaño que tendrá finalmente el índice*. Uno de los algoritmos que mejora esto, reduciendo el tiempo anterior a 11 horas es el “Multiway-merging” (discutido en [43]), que es una modificación de este. El enfoque “Multiway in-place” ocupa bastante menos disco que el “Multiway merging” (para este caso, del orden de 150 MB frente a los 540 MB del anterior) y tarda lo mismo, así que, en caso de falta de espacio, merecería la pena consultar este algoritmo.

Como la disponibilidad de almacenamiento secundario a bajo precio es total (podemos a día de hoy fácilmente encontrar discos duros de varias centenas de GB), supondremos que es virtualmente imposible que la falta de espacio sea un problema real, al menos con el tamaño de las colecciones que se manejan a día de hoy, y con las que seguramente se manejarán.

Otro algoritmo, el *FAST-INV*, debido a Fox y a Lee ([43]), se aprovecha de dos puntos:

- Los ordenadores de hoy en día pueden tener bastante memoria; no tanta para que quepa el índice en ella, pero más que la necesaria para los métodos clásicos de indexación.
- Los datos están inherentemente ordenados.

Estos dos puntos vienen a decirnos que el cuello de botella de la indexación es el alto número de accesos a discos que hay que realizar. Por eso, se propone

un método que aproveche mejor la memoria, más allá de almacenar un lote más grande y tardar más en escribirlo al archivo temporal.

El tiempo total de ejecución es de unas 12 horas para el mismo caso anterior de los 5 GB de texto. Esto quiere decir que **no merece la pena** realizar dicho esfuerzo para no reducir demasiado un tiempo de una operación que sólo tiene que realizarse una vez.

Conclusiones

Puesto que la indexación no parece ser un proceso crítico en tiempo (sólo tiene que hacerse una vez), escogemos un algoritmo cuyo desempeño sea relativamente bueno: el algoritmo de *indexación basada en ordenación* comentado anteriormente.

Hecha esta elección, es necesario que adaptemos dicho algoritmo para nuestras necesidades (colección de documentos estructurados, DTDs heterogéneos y estructuras de datos predefinidas), punto que desarrollaremos en el siguiente apartado.

Suponemos que a día de hoy, la disponibilidad de computadores con memoria principal de tamaño del orden de GB, no resulta necesario afinar tanto el método de indexación, puesto que se puede ir procesando la información en “bloques grandes”, para luego volcar el resultado temporal al fichero, y finalmente, procesar éste.

5.2.3. Adaptación de un algoritmo de indexación para el caso estructurado

NOTA: las variables notadas con circunflejo ($\hat{\cdot}$) son temporales, residen en memoria, y su ámbito se extiende sólo durante la ejecución del algoritmo.

Variables (sólo las principales)

$\hat{\mathcal{U}}$, lista de objetos `Unit`

$\hat{F}_{tag-dtd}$, array de enteros, subconjunto de $F_{tag-dtd}$

$\hat{F}_{id-tag}$, array de cadenas, subconjunto de F_{id-tag}

$\hat{T}_{tree}$, árbol de enteros

$\hat{T}_{tag-id}$, array de hashes de cadenas (array de tablas que asocian a cada cadena un entero).

id_{DTD} , entero identificador del DTD actual

1. Inicialización

F = archivo (temporal) vacío

$L \leftarrow \emptyset$

$\hat{\mathcal{U}} \leftarrow \emptyset$

5.2. DISEÑO DE UN ALGORITMO DE INDEXACIÓN PARA DOCUMENTOS ESTRUCTURADOS

U_{lim} = número máximo de objetos *Unit* que queremos procesar en memoria

T_{lim} = número de registros que caben en memoria

2. Procesamiento

PARA cada archivo D_l de la colección

$\hat{T}_{tree} \leftarrow \emptyset$

Obtener el id_{DTD} correspondiente. Si no existe, crearlo.

PARA cada unidad U_i de D_l

ObtainProperties(U_i)

SI U_i es unidad final

processFinalUnit(U_i)

SI NO

processUnit(U_i)

FIN SI

FIN PARA

processParents()

FIN PARA

2.1 ObtainProperties

Construir el objeto *Unit* correspondiente. Utilizar un nuevo identificador. $\hat{\mathcal{U}} = \hat{\mathcal{U}} \cup U_i$

Asociarle el identificador id_{DTD}

SI en $\hat{T}_{tag-id}[id_{DTD}]$ no hay una entrada, para el tipo de U_i

Crearla y asociarle un nuevo identificador

Actualizar $\hat{F}_{id-tag}$

FIN SI

Asociar a U_i su identificador de tipo de unidad

SI $|\hat{\mathcal{U}}| = U_{lim}$

Añadir $\hat{\mathcal{U}}$ al final de F_{unt} e IF_{unt}

Añadir $\hat{F}_{tag-dtd}$ al final de $F_{tag-dtd}$

$\hat{\mathcal{U}} = \emptyset$

FIN SI

2.2 ProcessUnit

Añadir U_i al árbol $\hat{T}_{tree}$ en su lugar correspondiente

2.3 ProcessFinalUnit

Añadir su camino a la raíz en F_{desc} e IF_{desc} con $\hat{T}_{tree}$

Leer el contenido de U_j , extrayendo términos indexables

CreateDirectIndexEntry()

PARA cada término T_l indexable en U_j

Sea $f_{l,j}$ la frecuencia de T_l en U_j
 Buscar T_l en L . Si no está, insertarlo
 Escribir² la lista de las m posiciones de aparición $\{pos_1, \dots, pos_n\}$ en F_{pos}
 Escribir registro $(T_l, U_j, f_{l,j}, ptr_{l,j})$ al archivo temporal
 F , donde T_l y U_l están representados por su identificador, y
 $ptr_{l,j}$ es el desplazamiento en el archivo de posiciones.

FIN PARA

2.3.1 CreateDirectIndexEntry

Crear una entrada del índice directo y almacenarla en F_{dir} e IF_{dir}

2.4 ProcessParents

Utilizando $\hat{T}_{tree}$, rellenar F_{par} y $IF_{desc,par}$
 recorriendo el árbol **en orden de identificadores** (en profundidad)

3. Ordenación para crear lotes

REPETIR

Leer hasta T_{lim} registros del archivo F .

Ordenar por orden no decreciente de T_i , y para iguales valores de
 éste, por orden no decreciente de U_j

Escribir los T_{lim} registros de vuelta al archivo F

HASTA que no haya más registros que ordenar

4. Mezcla

Mezclar por parejas lotes en F , hasta que sólo quede un lote ordenado.

5. Volcado del archivo invertido

PARA cada término $1 \leq T_l \leq |\mathcal{T}|$

Crear una nueva entrada en F_{occ} e IF_{occ}

Leer todos los registros $(T_l, U_j, f_{l,j}, ptr_{l,j})$ de F
 y crear la entrada para F_{occ}

Comprimir (si se realiza compresión)

Escribir la entrada a F_{occ}

FIN-PARA

Experimentación

El algoritmo planteado no es susceptible de variar demasiados parámetros, pero aún así es susceptible de recibir algunas variaciones como pueden ser:

5.2. DISEÑO DE UN ALGORITMO DE INDEXACIÓN PARA DOCUMENTOS ESTRUCTURADOS

- Ajuste del valor T_{lim} según el tamaño de la memoria principal (un tamaño mayor, dará un menor tiempo de ejecución).
- Establecimiento de otros valores límite para mantener algunas estructuras en memoria, antes de escribirlas a disco (cada cuál ajustado al posible tamaño de dicha estructura en función del número de elementos).
- Utilización de cachés explícitamente proporcionadas por el sistema operativo en lugar de utilizar un umbral (relegamos en el sistema la escritura del bloque de memoria, cuando éste no cabe).
- Utilización de otras facilidades proporcionadas por los sistemas operativos, como puede ser el mapeo de archivos en memoria (para reducir el acceso a disco).

No hace falta decir que todas estas mejoras son en gran parte dependientes de la máquina concreta donde vaya a ejecutarse el sistema RI, y por tanto tendrán que ser ajustadas expresamente para ésta. Así, si el sistema va a implantarse en una máquina concreta, y quiere sacarse el máximo rendimiento a las características de ésta durante el proceso de indexación, podría plantearse un estudio de los parámetros, comparando diferentes valores de éstos, la influencia o no de ciertas acciones, o incluso, diferentes sistemas operativos, con objeto de obtener el máximo rendimiento.

5.3. Diseño de un algoritmo de recuperación para documentos estructurados

5.3.1. Introducción

La presente sección trata de cubrir los posibles aspectos de la implementación del algoritmo de recuperación de un modelo concreto. El modelo que vamos a tratar es el modelo estructurado BNR-SD [9] (que posteriormente podría ser ampliado con la utilización de un diagrama de influencia, modelo IDR-SD).

5.3.2. Implementación del modelo BNR-SD

Introducción

Una vez obtenidos de la consulta el conjunto de términos que interesan al usuario (supondremos sencillamente que se trata de un conjunto $Q = \{T_j \wedge \dots \wedge T_x\}$), tenemos que calcular, para Q consulta, las unidades que resulten relevantes.

En términos de probabilidad, se trata de obtener las unidades U_i para las que la probabilidad a posteriori $p(u_i^+|Q) > 0$. Ordenar por relevancia dicha lista significará ordenar por el valor concreto de probabilidad, de mayor a menor.

Durante toda esta sección, se realizará un desarrollo *independiente del juego de pesos considerado*, W_i . Todo lo expuesto aquí puede funcionar para cualquier conjunto de pesos siempre y cuándo verifique las mismas propiedades que los propuestos en [9].

Algoritmo

Debido a que no podemos confiar en tener una estructura estricta por niveles, **estamos obligados a recorrer el grafo en profundidad** tal y como se hace en el algoritmo mostrado en [10] (concretamente en la segunda propuesta, que es la que aquí desarrollamos).

```
PARA cada término  $T_i$  de la consulta  $Q$  HACER
  PARA cada unidad  $U_j$  hija de  $T_i$  HACER
    level =  $l$ ;
    prod =  $w(T_i, U_j)$ ;
    SI existe el acumulador acum[ $U_j$ ] ENTONCES
      acum[ $U_j$ ] += prod;
    DE LO CONTRARIO
      crear acum[ $U_j$ ]; acum[ $U_j$ ] = prod;
  FIN-SI
```

5.3. DISEÑO DE UN ALGORITMO DE RECUPERACIÓN PARA DOCUMENTOS ESTRUCTURADOS

```
MIENTRAS level <> 1 HACER
     $U_h$  = unidad hija de  $U_j$ ;
    prod * =  $w(U_j, U_h)$ ;
    SI existe el acumulador acum[ $U_h$ ] ENTONCES
        acum[ $U_h$ ] += prod;
    DE LO CONTRARIO
        crear acum[ $U_h$ ]; acum[ $U_h$ ] = prod;
    FIN-SI
     $U_j = U_h$ ;
    level--;
FIN-MIENTRAS
FIN-PARA
FIN-PARA

PARA cada unidad  $U_i$  para la que exista acum[ $U_i$ ] HACER
    acum[ $U_i$ ] =  $p_0 + (1 - p_0)acum[U_i]$ ;
FIN-PARA
```

Tras la ejecución, basta devolver la lista de unidades estructurales U_i , ordenadas por el valor de su acumulador (que una vez terminada la ejecución del algoritmo coincide con la probabilidad a posteriori $p(u_i^+ | Q)$) de mayor a menor.

Cuestiones sobre la implementación

Como hemos visto, la idea del algoritmo es la siguiente: construir el subbosque³ de unidades que tendrán cierta relevancia (aquellas descendientes de las unidades de último nivel donde aparece un término de Q). En cada nodo, se almacenará una unidad y un acumulador (float) cuyo valor inicial será 0.0.

Para el último nivel, obtenemos el valor de la probabilidad de relevancia como la suma de los pesos de los términos de Q que contiene. Uno por uno, vamos recorriendo dichas unidades de último nivel, explorando todos sus hijos hasta llegar al nodo de nivel inferior, sumando al acumulador de cada unidad el producto del peso de su padre por la probabilidad de relevancia del mismo. Tras repetir esta operación para todas las unidades de último nivel, disponemos de un subbosque donde cada nodo tiene asociado la unidad y el valor de la probabilidad de relevancia.

³Subconjunto del bosque que representa todos los términos y unidades.

Debido a que el algoritmo tendrá que ser implementado haciendo uso de nuestro entorno, merece la pena comentar algunos aspectos concretos de implementación:

- Inicialmente, como ya decíamos antes, construiremos el grafo representando a la red explícitamente en memoria. Para ello, tendremos una **lista** de objetos de clase *Node*, N , en memoria, mientras dure la ejecución.
- La clase *Node* contendrá, al menos, un puntero a un objeto *Unit* y un valor *float* que representará el acumulador. En el constructor de la clase se debe permitir iniciar objetos *Node* con un par *Unit*, valor *float* (que es asignado directamente como valor al acumulador). Debe existir también facilidad para incrementar dicho valor con una cantidad *float* pasada como parámetro (método *increment*).
- Aunque el algoritmo parece reposar irremediabilmente en una estructura jerárquica rígida de niveles, esto no es del todo cierto ya que, con ligeras modificaciones, esta dependencia desaparece. Así, en vez de utilizar un criterio por niveles en el bucle más interno, comprobaremos que la unidad correspondiente (el objeto *Unit*) no sea la unidad raíz para seguir ciclando. Alternativamente, los objetos *Node* deberán obligatoriamente disponer de un puntero a objetos de su mismo tipo, con el fin de permitir la navegabilidad del grafo. Mediante el método *getDescendant()* es posible obtener el “hijo”⁴ de un *Node*, ó *NULL* si aquél no existe. Alternativamente, otro método *setDescendant()* debe permitir “enlazar” los diferentes objetos de la clase.
- Inicialmente, para cada unidad final, recuperaremos (con la ayuda de F_{desc}) la lista de identificadores de unidades descendientes hacia la raíz. Para cada identificador, en caso de no estar presente en algún *Node* de la lista N , se traerá a memoria su correspondiente objeto *Unit*, y será introducido en la lista N . Puesto que esta va a ser una operación realizada repetidas veces, **interesa razonablemente** mantener la lista ordenada por identificadores de unidad ya que la inserción y búsqueda será mucho más rápida. Además, siguiendo el sistema de numeración de las unidades tal y como se ha propuesto en el apartado de indexación, como para cada lista de descendientes además de existir un orden por niveles, existe un orden por identificadores que se verifica seguro⁵, la búsqueda quedará reducida a regiones aún más concretas de la lista N . Esto hace bastante eficiente la tarea de construir en

⁴El hijo de un *Node* N_i es el *Node* N_j cuya *Unit* U_j sea hija de la *Unit* de N_i , U_i .

⁵El identificador de un hijo ó descendiente directo de una unidad siempre será **menor** que el de la unidad considerada.

5.3. DISEÑO DE UN ALGORITMO DE RECUPERACIÓN PARA DOCUMENTOS ESTRUCTURADOS

memoria el grafo y reduce casi todo el tiempo de procesamiento a los bucles de cálculo de los acumuladores.

Una representación de la estructura en memoria de los elementos de N puede verse en la figura 5.1.

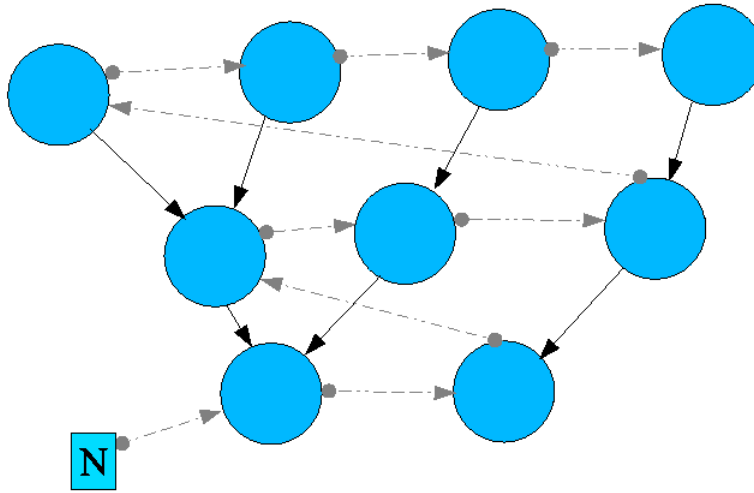


Figura 5.1: Estructura en memoria de la lista de nodos N . Obsérvese que las flechas continuas y los Node coinciden con la estructura de la red bayesiana, mientras que las flechas con círculo punteadas dan una ordenación de los identificadores de las unidades de los nodos, comenzando por el nodo raíz.

Adaptación del algoritmo a nuestro entorno

Notación: la construcción $N(U_j)$ significa el Node de N que contiene un puntero a la unidad de identificador U_j (si existe).

El algoritmo adaptado será pues, el siguiente:

```

lista<Node> N;
PARA cada término  $T_i \in Q = \{T_1 \wedge \dots \wedge T_n\}$  HACER
    PARA cada identificador de unidad  $U_j \in \text{ocurrencias}(T_i)$  HACER
        float prod =  $w(T_i, U_j)$ ;
        SI existe el Node  $N(U_j)$  ENTONCES
             $N(U_j).increment(prod)$ ;
        DE LO CONTRARIO

```

```

    Unit  $U_j(U_j)^6$ ;
    Node  $n_{tmp}(U_j, prod)$ ;
    insertar( $N, n_{tmp}$ );
FIN-SI

Node* actual =  $N(U_j)$ ;
MIENTRAS actual.getUnit().getParent() <> NULL
    Unit  $U_h$  = identificador de la unidad hija de actual.getUnit()7;
    prod * =  $w(actual.getUnit(), U_h)$ ;
    SI existe el Node  $N(U_h)$  ENTONCES
         $N(U_h).increment(prod)$ ;
    DE LO CONTRARIO
        Unit  $U_h$ ;
        Node  $n_{tmp}(U_h, prod)$ ;
        insertar( $N, n_{tmp}$ );
        enlazar  $N(U_h)$  y actual;
    FIN-SI
    actual =  $N(U_h)$ ;
FIN-MIENTRAS
FIN-PARA
FIN-PARA

PARA cada Node  $N_i$  de  $N$  HACER
     $N_i.acum = p_0 + (1 - p_0) \cdot N_i.acum$ ;
FIN-PARA
Ordenar  $N$  por valor de acum;
return  $N$ ;

```

Casos especiales

A partir de este algoritmo pueden implementarse fácilmente otros:

- Versión “espacio vectorial”: en la que sólo se recuperan unidades de último nivel. Simplemente basta con eliminar el bucle “MIENTRAS” interno, dejando el resto del algoritmo como está. De esta forma, la lista N que se construya sólo estará compuesta de unidades de último nivel y serán éstas las únicas que sean devueltas.
- Versión “máxima granularidad” si se quiere especificar un tipo de etiqueta a partir de la cuál no se quiere recuperar unidades más grandes, podemos, o bien introducir un criterio de parada adicional en el bucle interno, o bien

5.3. DISEÑO DE UN ALGORITMO DE RECUPERACIÓN PARA DOCUMENTOS ESTRUCTURADOS

procesar la lista N a posteriori. Generalmente, preferiremos el último criterio por motivos que explicamos en el siguiente punto.

- Generalizando al anterior, podemos implementar una poda genérica de cualquier tipo de unidades, según requisito de problemas, simplemente procesando la lista N a posteriori, antes de devolverla. Este procesamiento, al estar los nodos unidos por punteros (por una parte) y ordenados por identificadores (por otra) se puede realizar de forma bastante sencilla y eficiente.

Capítulo 6

Aplicación al caso del Parlamento de Andalucía

6.1. Introducción

Este capítulo trata de abordar el proceso y la problemática específica de la adaptación e implantación de un sistema de RI genérico para una colección particular: la de las sesiones plenarias del Parlamento de Andalucía.

Aunque el sistema diseñado en los dos capítulos anteriores no está totalmente terminado (se encuentra en su última fase, desarrollo y pruebas), explicamos el proceso que se seguirá para la puesta en marcha del mismo cuando esté totalmente terminado, añadiendo además de que algunos puntos pueden realizarse independientemente de estar el sistema terminado.

Describiremos, primeramente, generalidades sobre los documentos del Parlamento de Andalucía, qué tamaño tiene la colección, con qué tipo de archivos estamos tratando... Tras esta sección, estudiaremos a fondo la estructura de las sesiones plenarias, proponiendo un DTD para las mismas, puesto que la colección debe ser adaptada a un formato estructurado (como veremos no se dispone de las sesiones plenarias en formato XML, sino en un formato no estructurado).

La última parte de este capítulo versa sobre la implantación (esto es, instalación y puesta en marcha) del sistema físicamente.

6.2. Los Diarios de Sesiones del Parlamento de Andalucía

El día 21 de junio de 1982 se celebraba en los Reales Alcázares de Sevilla la sesión constitutiva del Parlamento de Andalucía. Desde entonces, han transcurrido seis legislaturas y parte de la séptima, en las que se han celebrado casi 1000 plenos recogidos cada uno en su correspondiente “Diario de Sesiones”¹. Cada uno de ellos tiene una media de unas 50 páginas de texto.



Figura 6.1: Salón de plenos del Parlamento de Andalucía

En total, la colección asciende a casi 50000 páginas de documentos relatando lo sucedido en cada una de las sesiones. La situación actual de los documentos es la de un archivo, en formato PDF con el contenido completo de dicha sesión. Dichos documentos son accesibles desde la web del Parlamento de Andalucía ([30]), de forma directa (es posible descargarse un diario de sesiones concreto de una cierta legislatura), o mediante el uso de buscador. Este buscador implementa un sistema de consulta basado consultas de tipo CO, esto es, construidas a base de términos.

Además, no incorpora ningún tipo de función de relevancia (por lo que se considera que un documento es relevante si aparece en el diario completo) y, como era previsible, sólo devuelve como resultado una lista de diarios (sin especificar ni

¹Documento que contiene la transcripción de lo acontecido en dicha sesión plenaria.

una parte concreta del diario, ni un punto concreto donde se encuentre el término buscado).

Por poner un ejemplo, si buscamos un término que no tiene por qué ser frecuente, “ley del suelo” (suponiendo que queremos comprobar cuándo se han aprobado dichas leyes y qué modificaciones han sufrido), obtenemos 118 plenos. Esto hace un total de unas 6000 páginas, y unos 60MB que descargar. Seguramente, muchos de estos documentos no tratarán específicamente sobre la aprobación (o modificación) de la ley del suelo, y su revisión puede ser un trabajo tedioso.

Si los diarios de sesiones estuvieran estructurados, se podrían devolver sólo fragmentos de los mismos que fueran relevantes (por ejemplo, intervenciones concretas, preguntas, réplicas...). Además, la implementación de un criterio de relevancia (que permita mostrar gráficamente y según un orden qué documentos son más importantes) podría reducir el tiempo necesario para encontrar información por parte de un usuario que, a día de hoy, resulta poco más que tedioso.

Obviamente, si la búsqueda pudiera realizarse en términos aproximados, incluyendo elementos opcionales, o restringiendo a unos ciertos descriptores temporales, los resultados serían muchísimo más precisos. De ahí la necesidad también de contar con un lenguaje de consulta más potente que el actual.

Otro problema añadido es el formato actual de las sesiones plenarias. Dicho formato (PDF) no es flexible en el sentido en el que pueden extraerse partes del mismo automáticamente, o que pueda mostrarse con diferentes estilos. El formato PDF está orientado a la representación final de datos, con objeto de visualizarlos como si de un texto impreso en papel se tratase. Dicho formato es un formato binario que incluye además, las imágenes embebidas. Generalmente no se permite la modificación de estos archivos, por lo que no son adecuados en absoluto para nuestro propósito.

Además, en dichos archivos, el texto de las sesiones se encuentra a dos columnas, con lo que cualquier intento de extraerlo puede ser fallido debido a la confusión de líneas provocada por la división en columnas.

CAPÍTULO 6. APLICACIÓN AL CASO DEL PARLAMENTO DE ANDALUCÍA

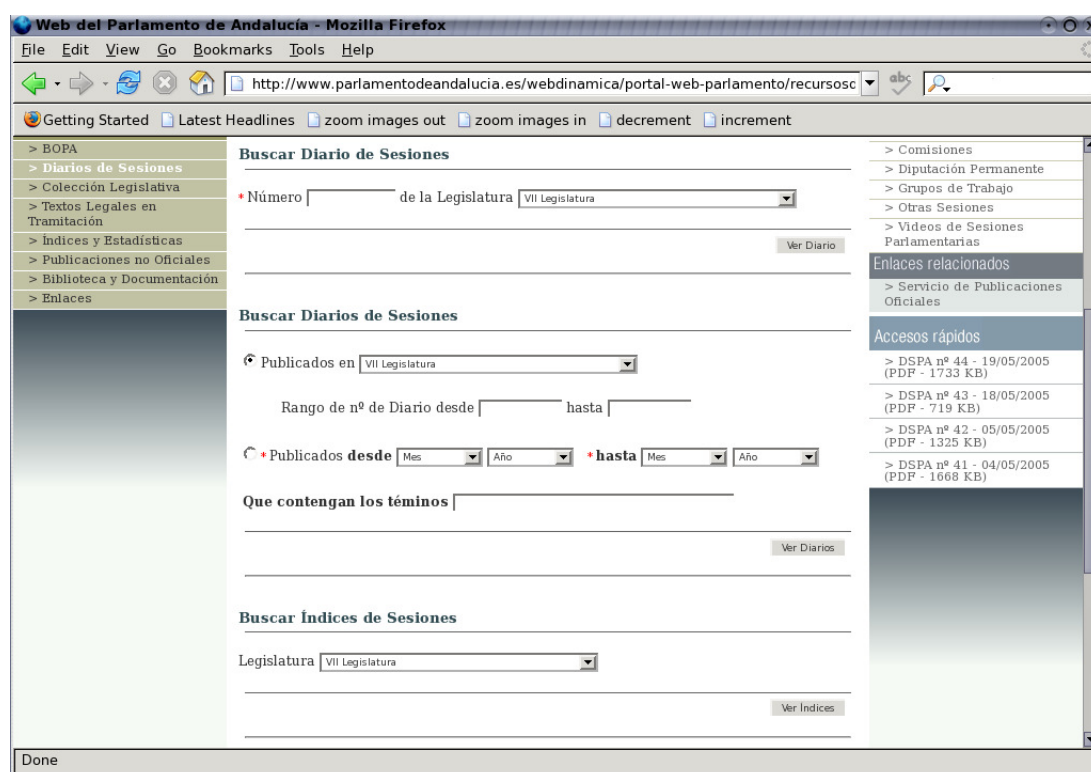


Figura 6.2: Buscador de diarios de sesiones de la web del Parlamento

6.3. Estructura de las sesiones plenarias. Adaptación de la colección

Una vez situados en el problemático marco del formato de las sesiones del Parlamento, constatamos que dichos documentos deberán ser transformados a un formato más manejable. La conversión a un formato estructurado, como XML quedará justificada con la inclusión en el siguiente punto de un documento DTD que resume todas las posibles estructuras de los plenos² del Parlamento.

En el siguiente punto a la definición del DTD, incluye un esquema de cómo podría convertirse la colección a formato XML. A día de hoy hemos estudiado diferentes alternativas, aunque todavía no hemos hallado la más apropiada (todas tienen algún problema). Por ello, el esquema es general (aunque no por ello se deja fuera algún paso).

²Al menos de los celebrados hasta ahora, aunque no hay demasiada previsión de que esta cambie.

6.3.1. Estructura de las sesiones

Mostramos en este punto el documento DTD con la estructura de las sesiones.

```
<!DOCTYPE diario_sesiones_pa PUBLIC "-//dtd/diario_sesiones_pa.dtd">

<!ELEMENT diario_sesiones_pa (impresion, issn,
depositolegal, legislaturas)>

<!ELEMENT impresion (#PCDATA)>
<!ELEMENT issn (#PCDATA)>
<!ELEMENT depositolegal (#PCDATA)>

<!ELEMENT legislaturas (numero_legislatura, anio)+>
<!ELEMENT numero_legislatura (#PCDATA)>

<!ELEMENT anio (fecha_anio, (comision+ | sesion+ )>

<!ELEMENT comision (numero_comision, serie,
nombre_comision, presidente, fecha_celebracion,
pagina_inicio, pagina_fin, (orden_del_dia |
unica_iniciativa), sumario, desarrollo)>

<!ELEMENT numero_comision (#PCDATA)>
<!ELEMENT nombre_comision (#PCDATA)>
<!ELEMENT serie (#PCDATA)>

<!ELEMENT sesion (numero_pleno, presidente,
numero_sesion_plenaria, fecha_celebracion, pagina_inicio,
pagina_fin, (orden_del_dia | unica_iniciativa),
sumario, desarrollo)>

<!ELEMENT numero_pleno (#PCDATA)>
<!ELEMENT presidente (#PCDATA)>
<!ELEMENT numero_sesion_plenaria (#PCDATA)>
<!ELEMENT fecha_celebracion (fecha_dia,
    fecha_mes, fecha_anio) (#PCDATA)>
<!ELEMENT fecha_anio (#PCDATA)>
```

CAPÍTULO 6. APLICACIÓN AL CASO DEL PARLAMENTO DE ANDALUCÍA

```
<!ELEMENT fecha_mes (#PCDATA)>
<!ELEMENT fecha_dia (#PCDATA)>
<!ELEMENT pagina_inicio (#PCDATA)>
<!ELEMENT pagina_fin (#PCDATA)>

<!ELEMENT orden_del_dia (tipo_iniciativa, iniciativas+)>
<!ELEMENT tipo_iniciativa (#PCDATA)>

<!ELEMENT iniciativas (numero_expediente?, extracto,
proponente? )>
<!ELEMENT numero_expediente (#PCDATA)>
<!ELEMENT extracto (#PCDATA)>
<!ELEMENT proponente (#PCDATA)>

<!ELEMENT unica_iniciativa (#PCDATA)>

<!ELEMENT sumario (hora_inicio, fecha_inicio, contenido,
hora_fin, fecha_fin)>
<!ELEMENT hora_inicio (#PCDATA)>
<!ELEMENT hora_fin (#PCDATA)>
<!ELEMENT fecha_inicio (#PCDATA)>
<!ELEMENT fecha_fin (#PCDATA)>

<!ELEMENT contenido (punto_orden+ | tema+)>
<!ELEMENT punto_orden (descripcion_punto, tema)>
<!ELEMENT descripcion_punto (#PCDATA)>
<!ELEMENT tema (extracto, intervienen, votacion?)>
<!ELEMENT extracto (#PCDATA)>
<!ELEMENT intervienen (interviniente+)>
<!ELEMENT interviniente (#PCDATA)>
<!ELEMENT votacion (#PCDATA)>

<!ELEMENT desarrollo (materia, intervencion+ )>
<!ELEMENT materia (#PCDATA)>
<!ELEMENT intervencion (interviniente, discurso)>
<!ELEMENT discurso (parrafo | acotacion)+>
<!ELEMENT acotacion (#PCDATA)>
<!ELEMENT parrafo (#PCDATA)>
```

6.3.2. Adaptación de la colección

Como ya se ha comentado antes, el estado actual de la colección es en documentos en formato PDF. Cualquier uso de los mismos, implicaría convertirlos a XML. Dicha conversión, debido al amplio volumen de información, no podría hacerse “a mano”, teniéndose que desarrollar un software específico para realizar la tarea. Para evitar que la conversión de la colección suponga una tarea más costosa que la construcción del propio sistema de recuperación, procuraremos utilizar en algunas partes de la conversión herramientas existentes³.

De una forma general, la conversión de PDF a XML puede plantearse de dos formas diferentes:

1. Realizar una conversión *de PDF a XML directamente*, y luego procesar el xml resultante para dar como salida un archivo en el formato adecuado. Esta opción fue desestimada puesto que considerábamos imposible que la salida fuera parecida a la estructura propuesta en el DTD anterior.
2. Realizar primero una conversión *a texto plano*, y posteriormente, convertir este texto a formato XML. Este enfoque tiene más trabajo que el anterior, puesto que implica la realización de una herramienta de análisis de texto para aplicar un “marcado” sobre el archivo. A pesar de todo, sí es más probable encontrar un conversor de PDF a texto de relativamente buena calidad, con lo que la primera de las tareas correría a cuenta de una herramienta externa.

La única herramienta encontrada por ahora que es capaz de realizar la segunda tarea es `pdftotext`, del paquete `xpdf`. Los experimentos realizados muestran que no funciona del todo bien y que, por ejemplo, no es capaz de detectar que el documento está escrito en dos columnas, alternando renglones de ambas.

El estado final de la colección debería ser el de un archivo XML por diario de sesión. Hecho esto, podríamos trabajar con la colección de diarios de sesiones con el sistema anteriormente implementado de forma normal.

³Procuraremos también, que las mismas sean software libre.

6.4. Implantación del sistema

6.4.1. Localización del sistema y la colección

El sistema y la colección deberían estar localizados en una máquina dedicada únicamente a tal efecto. Esto es así debido a la gran cantidad de estructuras en memoria que se mantienen, y de la mayor disponibilidad de ésta, si el uso no está compartido con otras aplicaciones.

La misma máquina, además del programa de indexación y el de recuperación, es conveniente que almacenase el programa de conversión a XML. Así, cada vez que fuera introducido un diario de sesiones, sería procesado automáticamente, y estaría listo para la indexación.

Otro dato a tener en cuenta es el hecho de que la colección del parlamento es una *colección dinámica*. Esto quiere decir que el número de documentos que la componen va a ir aumentando poco a poco. De esta forma, se requiere que la función de indexación admita la posibilidad de añadir un nuevo documento al índice de la colección. Esto último puede realizarse de dos maneras:

- *Reconstruyendo el índice completo* desde el principio. Puede ser una buena opción puesto que no requiere casi esfuerzo adicional de desarrollo, pero sólo puede ser tenida en cuenta si el algoritmo de indexación es considerablemente rápido. En caso de que así fuera⁴, cada vez que se quisiera añadir un nuevo documento a la colección, el proceso de indexación sería sencillo:
 - Añadir el nuevo documento a la colección.
 - Eliminar todos los índices existentes.
 - Reindexar la colección desde el principio.
- Otro procedimiento, quizás más ortodoxo, puede ser el realizar la indexación por separado del (de los) nuevo(s) documento(s), y realizar un proceso de *mezcla* del índice anterior con el actualmente obtenido. Requiere más esfuerzo de desarrollo que el anterior (nuevas funciones que añadir en el programa de indexación), pero debería realizarse en bastante menos tiempo. El proceso esquemático sería el siguiente:
 - Sea el índice de la colección I .
 - Añadir el nuevo documento a la colección.
 - Indexar, obteniendo un índice I' .

⁴En su momento realizaremos la comprobación, aunque presumiblemente no será así.

- Mezclar los índices, haciendo $I = I \cup I'$.
- El procedimiento que menos tiempo computacional requiere es utilizar un *conjunto de índices preparados para la posterior ampliación*. Este conjunto de índices ocuparía desde un primer momento más espacio en almacenamiento secundario, debido fundamentalmente a la presencia de una serie de “huecos” en su interior (destinados a las entradas futuras). Es el método que más esfuerzo por parte nuestra requeriría, y sólo sería considerado en el caso en el que los dos anteriores no proporcionasen resultados eficientes en tiempo.

De esta forma, el esquema gráfico del funcionamiento del sistema sería el presentado en la figura 6.3.

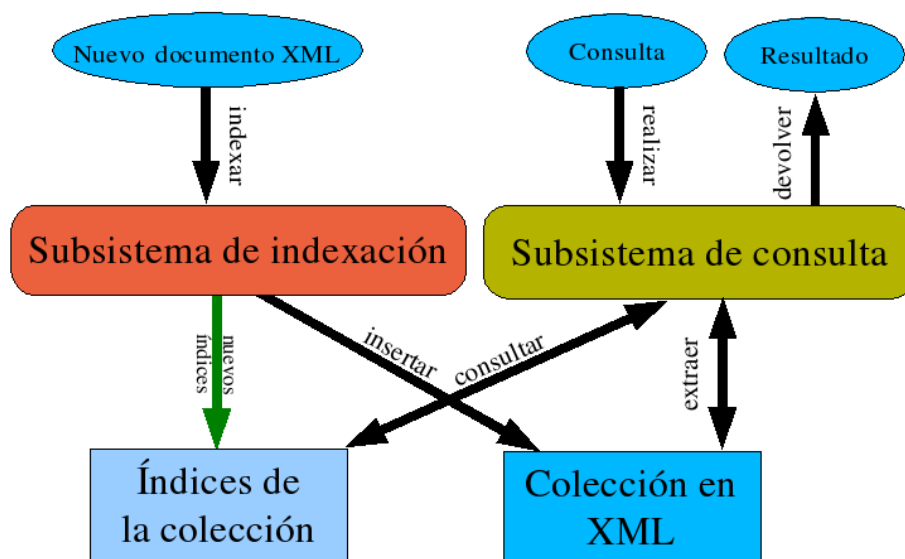


Figura 6.3: Esquema gráfico de funcionamiento general del sistema.

6.4.2. Interacción con los usuarios: interfaz del sistema

Para su uso conveniente, debería habilitarse una interfaz para el tratamiento cómodo de la colección. Básicamente, la única función que debería poder realizarse es la de añadir un nuevo diario de sesiones a la colección (conforme van celebrándose los plenos).

Una opción bastante cómoda es la de realizar una web (como Apache) mediante la que poder realizar la operación de indexación. Además, este mismo servidor web actuaría de plataforma para la recepción de consultas y devolución de resultados.

La disponibilidad de gran cantidad de lenguajes interpretados (PERL, PHP, Python) con amplia capacidad para generación dinámica de HTML y transformación de XML, además de presentar integración completa con el servidor web Apache, hace de esta opción una de las más directamente considerables.

El único problema podría surgir debido a la carga adicional añadida al sistema por el uso de un servidor web. Si esto fuera así⁵, podría considerarse una *arquitectura a tres niveles*, donde el servidor contendría tan sólo el sistema mostrado en la figura 6.3 (menos la colección en XML) y un pequeño programa servidor, mientras que el nivel intermedio contendría tan solo el servidor web propiamente dicho (y los archivos XML que compusieran la colección), y los clientes serían los diferentes usuarios (o administradores) a través de navegadores web. Las numerosas alternativas para realizar la distribución de una aplicación presentes en los sistemas de hoy en día hacen esta alternativa bastante factible.

6.4.3. Experimentación y pruebas

Se han comentado ya numerosos puntos donde se puede realizar una variación de parámetros, en función de la historia de uso del sistema, con objeto de ajustar su desempeño para que éste sea óptimo.

Entre otros detalles comentados, respecto al *nivel físico* se habló de variar el tamaño de la tabla de ocurrencias que residía en memoria, así como de mantener la lista de descendientes de las unidades estructurales más requeridas. Ambos tamaños de caché también pueden variarse.

Durante la fase de experimentación pueden recogerse datos del tiempo de respuesta del sistema, en función de la acción realizada. Podrían estudiarse los tipos de consulta que más se realizan, los términos más buscados, longitud media de la lista de resultados... Todos ellos pueden servir para ajustar el funcionamiento del sistema.

⁵Dependería en gran medida del número de usuarios del sistema. Número que, en este caso pronosticamos que no va a ser demasiado alto como para tener que realizar esta alternativa.

Además, la amplia presencia de programas de *profiling* de libre disposición (como GNU Prof) hace muy fácil la generación de estadísticas del tiempo medio de ejecución de cada función, y la posible detección de cuellos de botella en el sistema. Así, tras varias ejecuciones habiendo utilizado este tipo de programas, la obtención de las estadísticas de funcionamiento del programa es inmediato.

Conclusiones y trabajo futuro

Aunque el trabajo realizado no ha sido poco, aún no ha llegado a su fin. A pesar de ello, el logro más importante es la realización del diseño (y en un futuro cercano, la implementación) de un sistema de recuperación de documentos estructurados suficientemente general como para ser aplicado a problemas concretos, como a investigación en el área de la Recuperación de Información.

Durante el desarrollo de la memoria se han visto expuestos numerosos puntos donde se podía mejorar/variación/ampliar el funcionamiento del sistema. Muchos de los mismos formarán parte de un proceso de ajuste y experimentación con el sistema, por lo que requieren de un cierto uso del mismo para poder establecer conclusiones sólidas.

El sistema asimismo, podría utilizarse para investigación sobre recuperación estructurada. En particular, el grupo de investigación “Tratamiento de la Incertidumbre en Sistemas Inteligentes” (TIC-103) del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada, (en concreto los Prof. Dr. D. Luis M. de Campos –director de este proyecto–, Dr. D. Juan M. Fernández-Luna y Dr. D. Juan F. Huete) han mostrado su interés en el mismo como plataforma en la que poder realizar cierto tipo de experimentos relacionados con su investigación. Por esto, sería interesante la adaptación del sistema para el trabajo con la colección INEX ([21]), además de dotarle de una interfaz apropiada para este tipo de trabajo (añadiendo, entre otros, un módulo de juicios de relevancia).

Por otra parte, he mostrado especial interés en la publicación con licencia libre de este software (el sistema en general). La no disponibilidad de una herramienta libre de este tipo para el trabajo con documentos XML hace necesaria la división de esfuerzos de este tipo de investigadores, donde cada uno implementa sus herramientas propias. La necesidad de aunar esfuerzos, y de realimentarnos de otros proyectos implicaría casi automáticamente la liberación de dicho código. Por supuesto, esto se realizaría cuando estuviera totalmente desarrollado y probado (y si hay conformidad por parte de todas las autoridades competentes).

Finalmente, la implantación de este tipo de sistema en un entorno real, es, como ya se comentó anteriormente, una fuente inagotable de datos sobre el uso de

un sistema de RI. Dichos resultados pueden ser útiles también, para algún trabajo futuro, o para otras versiones de este software. Asimismo, la utilización por usuarios reales podría permitir la apertura de algún medio mediante el cuál ellos mismos pudieran sugerir cuestiones de funcionamiento del sistema (tipos de consultas, aplicación del lenguaje de consulta...).

Bibliografía

- [1] Silvia Acid, Luis M. de Campos, Juan M. Fernández-Luna, Juan F. Huete, *An information retrieval model based on simple Bayesian networks*. Int. J. Intell. Syst. **18**(2), 2003.
- [2] R. Baeza-Yates, W. B. Frakes (eds.), **Information Retrieval. Data Structures and Algorithms**, Prentice Hall, 1992.
- [3] R. Baeza-Yates, B. Ribeiro-Neto, **Modern Information Retrieval**, Addison Wesley Longman, 2002.
- [4] G. Booch, J. Rumbaugh, I. Jacobson, **El Lenguaje Unificado de Modelado**, Addison Wesley Iberoamericana, 1999.
- [5] E. Castillo, J.M. Gutierrez, A.S. Hadi, **Expert Systems and Probabilistic Networks Models**, Springer-Verlag, 1997.
- [6] Yves Chiaramella, *Information Retrieval and Structured Documents*. ESSIR 2000.
- [7] G. Cooper, *The computational complexity of probabilistic inference using Bayesian belief networks*. Artificial Intelligence, **42**, 1990.
- [8] Fabio Crestani, Mounia Lalmas, C. J. van Rijsbergen, Iain Campbell, “*Is This Document Relevant? ... Probably*”: A Survey of Probabilistic Models in Information Retrieval, ACM Comput. Surv. **30**, 1998.
- [9] Fabio Crestani, Luis M. de Campos, Juan F. Huete, Juan M. Fernández-Luna, A Multi-layered Bayesian Network Model for Structured Document Retrieval, *ECSQARU* (2003), 74–86.
- [10] Luis M. de Campos, *Modelos de Recuperación de Información Basados en Redes de Creencia*, Memoria de Habilitación, 2003.

- [11] Luis M. de Campos, Juan M. Fernández-Luna, Juan F. Huete, *Bayesian networks and information retrieval: an introduction to the special issue*, Inf. Process. Manage. **40**(5): 727-733, 2004.
- [12] Luis M. de Campos, Juan M. Fernández-Luna, Juan F. Huete, *The BNR model: foundations and performance of a Bayesian network-based retrieval model*. Int. J. Approx. Reasoning **34** (2-3), 2003.
- [13] Luis M. de Campos, Juan M. Fernández-Luna, Juan F. Huete, *Improving the Efficiency of the Bayesian Network Retrieval Model by Reducing Relationships Between Terms*. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems **11** (Supplement-1), 2003.
- [14] Luis M. de Campos, Juan M. Fernández-Luna, Juan F. Huete, *Clustering terms in the Bayesian network retrieval model: a new approach with two term-layers*. Appl. Soft Comput. **4** (2), 2004.
- [15] Luis M. de Campos, Juan M. Fernández-Luna, Juan F. Huete, *A Layered Bayesian Network Model for Document Retrieval*. Proc. of ECIR 2002, 2002.
- [16] P. J. Denning, *Working Sets Past and Present*, IEEE T. Software Eng. **SE-2**, (1980), 64-84.
- [17] S. J. DeRose, *Navigation, access, and control using structured information*. American Archivist , **60** (3), 1997.
- [18] F. J. Díez, **Introducción al Razonamiento Aproximado**, UNED 2003 (disponible *on-line* en <http://www.ia.uned.es/~fjdiez/libros/razaprox.html>).
- [19] Juan M. Fernández-Luna, **Modelos de Recuperación de Información Basados en Redes de Creencia**, Tesis Doctoral, Universidad de Granada, 2001.
- [20] M. Fowler, K. Scott, **UML Distilled**, Addison-Wesley, 1999.
- [21] N. Fuhr, N. Göbert, *INEX: Initiative for the Evaluation of XML Retrieval*, disponible en www.is.informatik.uni-duisburg.de/bib/fulltext/ir/Fuhr_etal:02a.pdf.
- [22] David A. Hull, *Stemming Algorithms: A Case Study for Detailed Evaluation*, JASIS **47**, 1996.
- [23] David Hunter, Andrew Watt, Jeff Rafter, et al, **Beginning XML, (Third Edition)**, Wrox Press, (2004).

- [24] F. V. Jensen, **An Introduction to Bayesian Networks**, UCL Press, 1996.
- [25] F. V. Jensen, **Bayesian networks and decision graphs**. Berlin: Springer-Verlag, 2001.
- [26] Joemon M. Jose, David G. Hendry, David J. Harper, *An Object-Oriented Framework for Developing Information Retrieval Applications*. OOIS 2001.
- [27] Yong Kyu Lee, Seong-Joon Yoo, Kyoungro Yoon, P. Bruce Berra, *Index Structures for Structured Documents*, Digital Libraries 1996: 91-99.
- [28] Gonzalo Navarro, Ricardo A. Baeza-Yates, *Proximal Nodes: A Model to Query Document Databases by Content and Structure*, ACM Trans. Inf. Syst, **15**(4), 1997.
- [29] J. Ortega, M. Anguita, A. Prieto, **Arquitectura de Computadores**, Thompson, 2004.
- [30] Página web del Parlamento de Andalucía, www.parlamentodeandalucia.es.
- [31] J. Pearl, **Probabilistic Reasoning in Intelligent Systems : Networks of Plausible Inference**, San Mateo CA: Morgan Kaufmann, 1986.
- [32] S. L. Pfleeger, **Ingeniería del Software: Teoría y Práctica**, Prentice Hall, 2002.
- [33] Eric T. Ray, **Learning XML (2nd edition)**, O'Reilly, 2003.
- [34] B. A. Ribeiro-Neto, R. R. Muntz, *A Belief Network Model for IR*, Proc. of the 19th Annual International ACM-SIGIR Conference on Research and Development in Information Retrieval SIGIR'96, ACM, 1996.
- [35] C. J. van Rijsbergen, **Information Retrieval**, second ed., Butter Worths, London, 1979.
- [36] P. Salinas, *Tutorial de UML: Modelo de Clases*, disponible en www.dcc.uchile.cl/~psalinas/uml/modelo.html.
- [37] Biren Shah, Abhilash Gummadi, Jong P. Yoon y Vijay Raghavan, *Efficient Dynamic Indexing and Retrieval of XML Documents using Three-Dimensional Quasi-BitCube*, Proc. of the First International Workshop on High Performance XML Processing.
- [38] Doug Tidwell, *Introduction to XML*, IBM developerWorks, 1999.

BIBLIOGRAFÍA

- [39] Andrew Trotman, *Searching Structured Documents*, Inf. Process. Manage. **40**(4), 2004.
- [40] Andrew Trotman, *Choosing Document Structure Weights*, Inf. Process. Manage. **41**(2), 2005.
- [41] Andrew Trotman, Börkur Sigurbjörnsson, *Narrowed Extended XPath I (NE-XI)*. INEX 2004, 2004.
- [42] H. R. Turtle, W. B. Croft, *Evaluation of an Inference Network-Based retrieval model*. Information Systems, **9**(3), 1991.
- [43] I. H. Witten, A. Moffat, T. C. Bell **Managing Gigabytes**, Morgan Kaufmann, 1999.
- [44] W3C XML Core Working Group, *Extensible Markup Language (XML) 1.0 (Third Edition)*, *W3C Recommendation 04 February 2004*, disponible en www.w3.org/TR/2004/REC-xml-20040204/.
- [45] L. Zadeh, *Fuzzy Sets*, Information and Control, **8**, 1965.